

MSc in Cybersecurity Engineering

Embedded Operating Systems and Architecture

01WMQWQ · A.Y. 2026/27

Professor: Stefano Di Carlo

Notes by Emanuele Tocci

Personal study notes, not official course material

Generated on 05/11/2026

Contents

Part I – Foundations **4**

1 Introduction **5**

1 What Is an Embedded System 5

2 Design Constraints of Embedded Systems 8

3 Inside the Hardware: Processor, Memory, and Buses 10

4 Operating Systems 15

5 Kernel Architectures 20

6 Embedded and Real-Time Operating Systems 22

2 ARM Cortex M3 Architecture **24**

1 ARM Cortex-M3 Core 24

2 Programmer's model: registers and ISA 25

3 Address Space and Memory-Mapped Registers 36

4 Execution Modes, Privilege, and Stacks 44

5 Exception Model 47

6 Configuring the Core: SCB, NVIC, and SysTick 53

7 Memory Protection Unit 58

3 Toolchain **62**

1 Cross-Compilation Toolchain 62

2 The ELF Format 67

3 Inspecting the Image 71

4 Running and Debugging the Image 80

5 Build Automation with Make 83

Part II – ScratchOS **87**

4 Boot CPU Config **88**

1 The Boot Problem 88

2 Linker Script 89

3 The Vector Table 96

4 The Startup Code: boot/startup.s 98

5 Reset Handler: From Reset to main() 103

6 Freestanding C and Semihosting 106

5 Device Interrupts Syscalls **109**

1 Peripherals and Buses of the MPS2-AN385 Platform 109

2 Memory-Mapped I/O 114

3 Device Drivers and Polled UART I/O 118

4 Polling and Interrupts 126

5 The SysTick Timer and Wraparound-Safe Time Keeping 127

6 The NVIC and Interrupt-Driven UART Transmission 131
7 System Calls and the Kernel Boundary 138

6 Memory Management 148

1 Why an Operating System Manages Memory 148
2 Paging and Virtual Memory 151
3 Static Task Partitions 155
4 Starting and Ending Tasks 159
5 Protecting Tasks with the MPU 162

Part I

Foundations

1 What Is an Embedded System

The word *"system"* comes from physics: a system is a set of elements inside a boundary. Everything inside the boundary is the system; everything outside is its *environment*. To describe an embedded system, the first step is therefore to decide **what is inside the boundary** (components, functions) and what is **outside** (the physical world it observes and controls).

There is no single official definition of an embedded system. Three common definitions stress different aspects, and together they give a complete picture.

View	Definition	Key idea
Hardware and software	A combination of hardware and software (firmware) where the software is embedded within the hardware	The software cannot be written without knowing the hardware
Dedicated computer	A system based on a microcontroller or microprocessor that performs one specific task	Hardware and software are designed together for one job
Cyber-physical system	Computer hardware, software, and physical parts that perform a dedicated function	The computer interacts with the physical world

The third view explains what an embedded system usually does. *Sensors* measure physical quantities such as temperature, speed, or pressure. The processor runs the firmware and computes a response. *Actuators* turn that response into a physical action, such as moving a motor, opening a valve, or changing a display. Many embedded systems are also connected to other systems through a **network**.

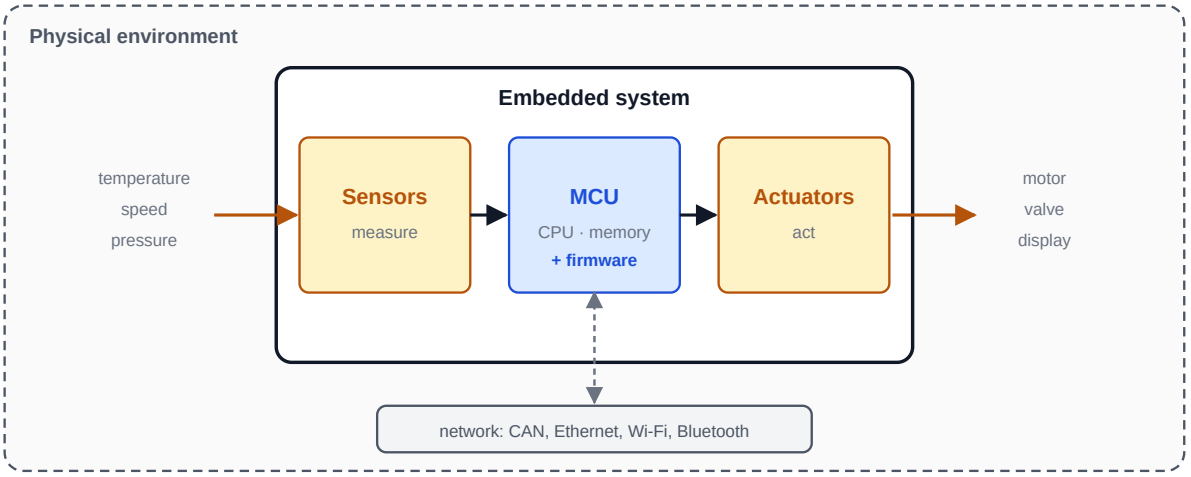


Figure 0.1: *An embedded system inside its environment. Sensors bring physical quantities in, the microcontroller runs the firmware, and actuators act on the world. A network may connect it to other systems.*

1.1 Classes of Embedded Systems

Embedded systems are used in very **different fields**, so they are **classified** in several ways. The three most useful classes are based on autonomy, timing, and connectivity. They are not exclusive: a car's brake controller is both real-time and networked.

Class	Main property	Example
Standalone	Works alone, without a network	Early washing machines, microwave ovens, thermostats
Real-time	Must complete each task before a deadline	Electronic brake control, drone stabilisation
Networked	Exchanges data with other nodes over a network or field bus	Engine, brake, and lighting controllers on a car's CAN bus

1.2 Embedded Versus General-Purpose Computers

1. A **general-purpose computer**, such as a laptop, runs any program the user installs, supports many users and tasks, and has plenty of memory and power.
2. An embedded system runs **one fixed job** and works under **strict limits**.

Aspect	Embedded system	General-purpose computer
Workload	One fixed task	Anything the user installs
Resources	KB to a few MB of RAM and Flash	GB of RAM, TB of disk
Timing	Often hard deadlines	Best effort
User model	No interactive user, no shell	Multi-user, multi-tasking
Power and cost	Milliwatts, cents to a few euros	Tens to hundreds of watts, hundreds of euros

These limits change **how software is written**. Embedded software cannot rely on an operating system to create a process, assign an address space, discover devices, or provide standard input and output. It is **built for one known processor** and one known memory map. The hardware details are visible and cannot be ignored, but this does not make the software simple: the developer must handle many tasks that a desktop OS normally hides.

The border between the two worlds is becoming less sharp. The same processor families, especially ARM, now range from tiny sensor nodes to smartphones, laptops, and cloud servers.

1.3 A Short History

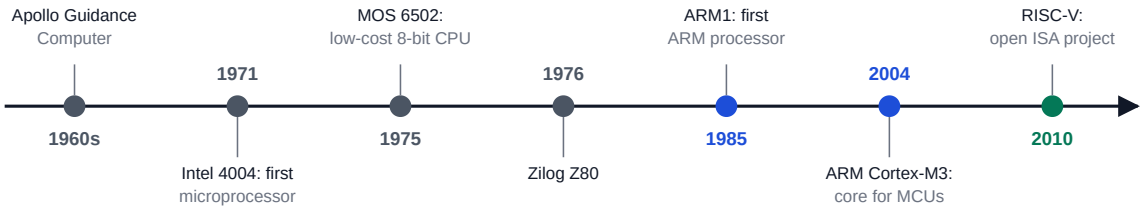


Figure 0.2: *Milestones of embedded computing, from the Apollo Guidance Computer to the open RISC-V instruction set (not to scale).*

Embedded computing grew from dedicated machines such as the Apollo Guidance Computer to cheap single-chip

processors, and today it is dominated by ARM cores, with the open RISC-V instruction set as a newer alternative. Embedded products also have a long lifecycle: a satellite or an industrial machine may keep the same processor for ten or twenty years, so reliability and **long-term support** often **matter more than performance**.

2 Design Constraints of Embedded Systems

Embedded systems are designed under **constraints** that rarely matter on a desktop. Each one shapes both the hardware and the software.

Constraint	Why it matters	Example
Real-time	Results must arrive be**fore a deadline**	A brake controller must react within a few milliseconds
Size and weight	Extra mass costs energy and space	A heavier controller shortens a drone's flight time
Cost	Small extra costs multiply over millions of units	A 50-cent security chip rejected for a car because of the total cost
Energy	Many devices run on batteries	A wearable or remote sensor must last months on one battery
Reliability and safety	Failures can damage machines or hurt people	Medical devices and robots must *fail safe*
Development time	Low-level code is slow to write and test	Hardware debugging and testing in all operating conditions

Fail safe means that when a fault is detected, the system moves to a **controlled and predictable state** instead of acting in an uncontrolled way. In a safety-critical system, every failure scenario must be considered at design time.

2.1 Real-Time Means Bounded Response

Real-time does not mean fast. A real-time system guarantees a **bounded and predictable** response time: every reaction to an event must complete before its **deadline**. A correct embedded system must therefore meet three kinds of requirements.

Requirement	Example
Functional	Sample a sensor correctly
Temporal	Sample it every 1 ms, before the deadline
Robustness	Continue safely when one input is missing

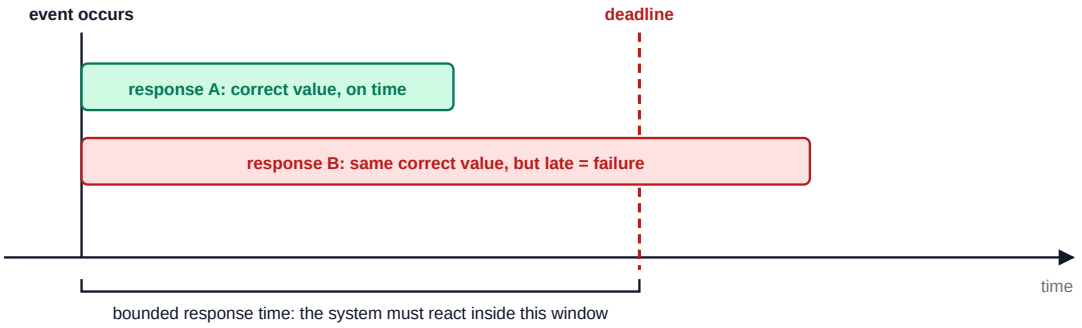


Figure 0.3: *Two responses to the same event. Both compute the correct value, but only the first arrives before the deadline; the second is a failure.*

In a *hard* real-time system, missing a deadline is a failure of the system, as in a braking controller. In a *soft* real-time system, a late result is still useful but lowers the quality of service, as in video playback.

> ⚠ **CORRECT VALUE, WRONG TIME**
>
> A result computed after its deadline can be a failure even if its value is correct. A fast system is not necessarily a real-time system, because average speed says nothing about the worst case.

3 Inside the Hardware: Processor, Memory, and Buses

A computer has four basic parts:

1. Central processing unit (CPU),
2. memory,
3. input units,
4. output units.

An embedded developer works much closer to these parts than an application developer does, so a basic model of each is needed.

3.1 Microprocessors and Microcontrollers

The **core** of a microprocessor and the core of a microcontroller execute instructions in the **same way**. The difference is what surrounds the core.

A ***microprocessor*** is mainly the CPU. It needs external RAM, external storage, and external peripheral chips to form a working computer. At power-on it runs a small firmware stored in a boot memory. This firmware starts a *boot loader*, which copies the operating system from storage (a disk or an SD card) into RAM and jumps to it.

A ***microcontroller*** **(MCU)** puts the CPU core, RAM, Flash, and peripherals (timers, converters, serial interfaces) on a **single chip**. Its internal Flash is mapped at the address the CPU reads after reset, so the firmware starts running directly from Flash, without a boot loader.

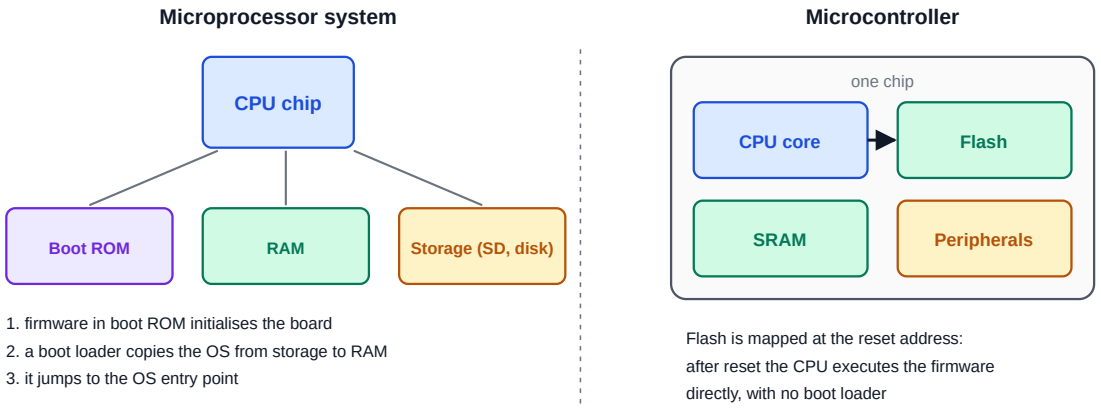


Figure 0.7: *A microprocessor needs external memories and a boot loader; a microcontroller integrates everything and executes its firmware straight from Flash after reset.*

3.2 The CPU and the Pipeline

A CPU repeats the same cycle forever:

1. **Fetch**: it reads the next instruction from memory, at the address held in the program counter.
2. **Decode**: it works out what the instruction asks for: which operation, and which registers or memory address it uses.

3. **Execute**: it performs the operation, for example an addition in the ALU, a load or store to memory, or a jump to another address.

Inside the CPU, the *control unit* drives this cycle, the *arithmetic logic unit* (ALU) performs arithmetic and logic operations, and *registers* hold the values being processed.

Registers are the fastest storage in the system, and a few of them have special roles:

1. **Program counter** (`PC`): holds the address of the next instruction to fetch. It advances automatically after each instruction; a branch or a function call changes it to jump elsewhere.
2. **Stack pointer** (`SP`): holds the address of the top of the stack, the memory area that stores local variables, return addresses, and saved registers during function calls and interrupts.
3. **Status register**: holds flags that describe the result of the last operation (for example zero, negative, carry, overflow), which conditional branches use to implement `if` statements and loops. It can also hold information about the processor state, such as whether interrupts are enabled.

Modern CPUs overlap these steps with a ***pipeline***. Each step is handled by a different part of the hardware, so while one instruction executes, the next one is decoded and a third one is fetched, like cars in a car wash with one worker per station.

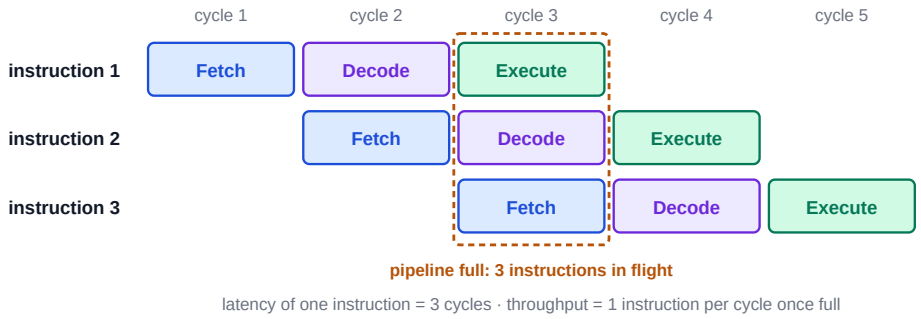


Figure 0.4: *A three-stage pipeline. From cycle 3 on, three instructions are in flight and one instruction completes every cycle.*

With n stages of duration t, a single instruction still needs $n \cdot t$ (its *latency* does not change), but once the pipeline is full one instruction completes every t. A sequence of m instructions takes

$$T_{total} = (n + m - 1) \cdot t$$

So pipelining increases *throughput*, not the speed of each instruction. The Cortex-M3 uses a three-stage pipeline of this kind.

3.3 Von Neumann and Harvard Architectures

CPU architectures differ in how they reach instructions and data. In the *von Neumann* architecture, instructions and data share one memory and one bus. It is simple and flexible, but the **shared bus limits** how fast data can move (the *von Neumann bottleneck*).
In the *Harvard* architecture, instructions and data use separate memories and buses, so the CPU can fetch an instruction and access data at the same time.

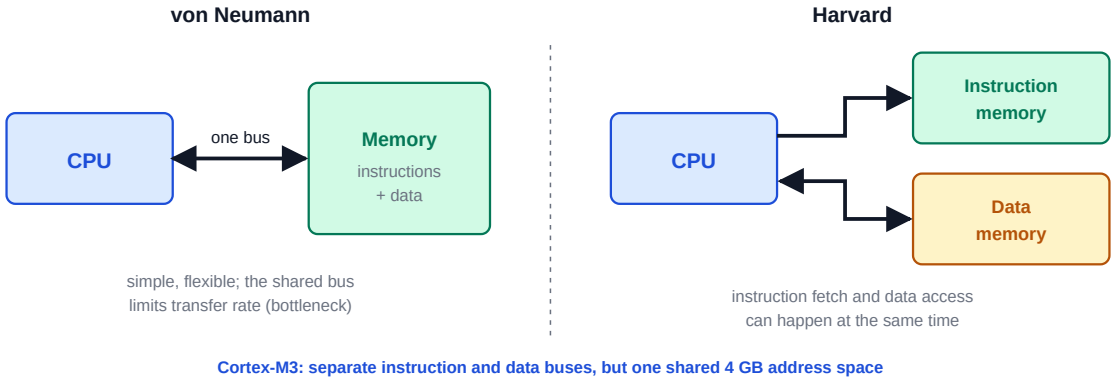


Figure 0.5: *von Neumann (one memory, one bus) and Harvard (separate instruction and data paths). The Cortex-M3 mixes the two: separate buses, one address space.*

Real processors often mix the two models. The Cortex-M3 has separate buses for instruction fetches and data accesses, which gives Harvard-like performance, but code and data share one 4 GB address space, as in von Neumann.

3.4 RISC and the Load/Store Model

ARM processors follow the **RISC (Reduced Instruction Set Computer)** philosophy: a small set of simple instructions, each executed quickly, combined with good compilers.
A key RISC rule is the ***load/store*** model: arithmetic and logic work **only on registers**, so a value must be loaded from memory into a register, processed, and stored back.
CISC (Complex Instruction Set Computer) processors, such as Intel *x86*, can instead **operate directly on memory**. Compilers hide this difference in C, but it becomes visible in assembly and in operating-system code.

3.5 Memory Hierarchy and Memory Technologies

Memories differ in speed, size, and cost, so every computer arranges them in a hierarchy. Small and fast memories sit close to the CPU; large and slow memories sit further away.

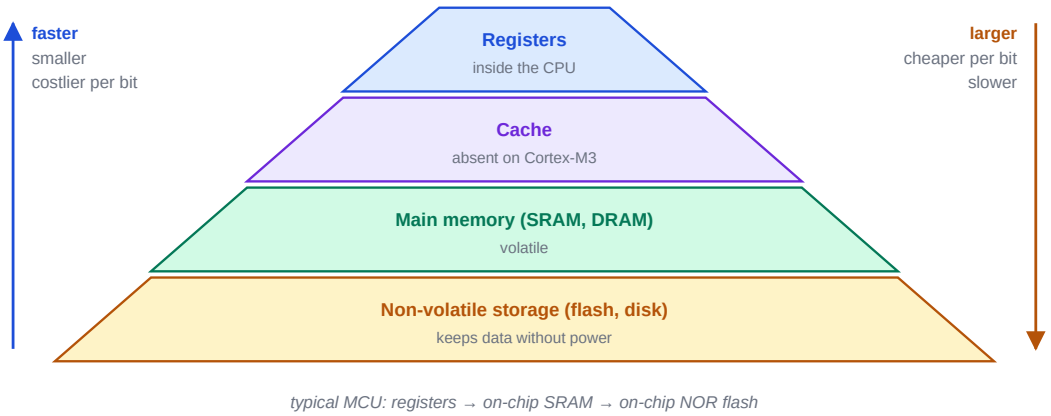


Figure 0.6: *The memory hierarchy. Going down, memories become larger and cheaper per bit but slower.*

A *cache* is a small, fast memory that automatically keeps recently used data close to the CPU. It improves

average speed, but its access time depends on what happened before, so the **same code can take different times** on different runs. This makes **timing hard to predict**, which is why many **real-time systems disable caches** or configure them carefully.

> **IMPORTANT**
>
> The Cortex-M3 core has no cache at all.

3.6 Buses and Peripherals

A *bus* is a set of **shared wires** that moves information between parts of a computer. In the textbook model, one system bus joins the CPU and the memories and is made of **three groups of lines**:

1. **address lines** select the memory location or device,
2. ***data lines*** carry the value being read or written,
3. ***control lines*** say whether the operation is a read or a write and when the data is valid.

A real microcontroller uses a **hierarchy of buses** instead of a single one. A fast on-chip bus (on ARM chips, the **AHB**) connects the CPU core with Flash and SRAM, which must be reached at every cycle.
Peripherals are slower and far more numerous, so they sit on a separate, simpler peripheral bus (the **APB**), reached through a *bridge*. This split keeps the fast bus small and saves power. The buses of the `mps2-an385` board are described in the chapter on devices, interrupts, and system calls.

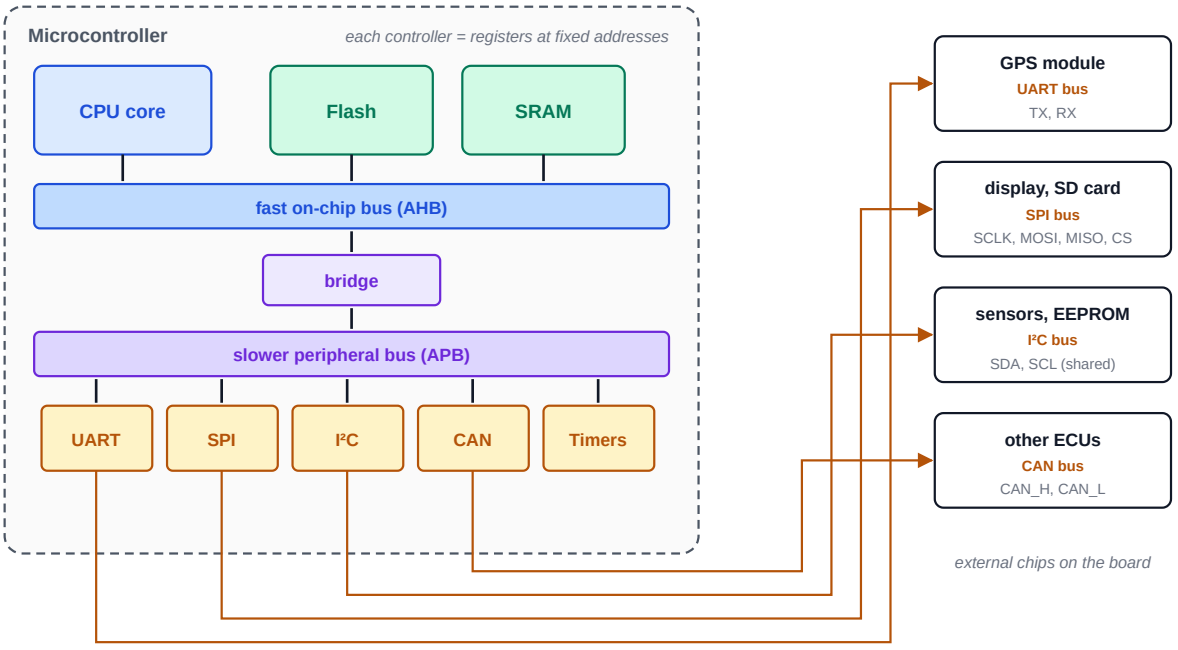


Figure 0.7: *Buses of a microcontroller. A fast on-chip bus connects the CPU and memories, a bridge leads to a slower peripheral bus, and each peripheral controller drives a serial bus to external chips.*

Each on-chip peripheral is a *controller* that the CPU programs through its *registers*, placed at fixed addresses in the memory map. Firmware configures a controller by writing these registers and checks its state by reading them, following the rules in the chip's datasheet. Many controllers drive a *serial bus*, which sends bits one after another over a few wires, to talk to other chips on the board.

On a desktop, device drivers inside the operating system hide all of this behind files and system calls. In an embedded system the firmware usually talks to the controllers directly, which is why register-level programming is a central skill of embedded development.

4 Operating Systems

An **operating system (OS)** has **several** classic **definitions**, each stressing a different role.

Definition	Emphasis
Intermediary	A middle layer between hardware and applications that hides hardware details
Resource manager	Software that shares CPU, memory, I/O devices, and storage among applications
Master control program	The first software loaded at power-on; its core, the *kernel*, stays in memory

ROLE OF AN OPERATING SYSTEM

An OS mediates between application software and hardware. It gives controlled access to processors, memory, devices, and time, and it decides how competing work is scheduled.

The OS has two main purposes:

1. First, it **makes the hardware usable**: without an OS, every application would have to initialise the CPU, program timers, share communication links, and handle interrupts on its own.
2. Second, it **manages resources** that are **limited** and **shared**, even on a small board: CPU time (which task runs now), memory (which code and data occupy RAM), devices (who may use a timer or read a received byte), and events (how an interrupt becomes work for the right task).

Internally, an OS can be seen as a set of *managers*, one per resource, placed above a layer of hardware-specific services. Applications never call the managers directly: they go through a single *system-call interface*.

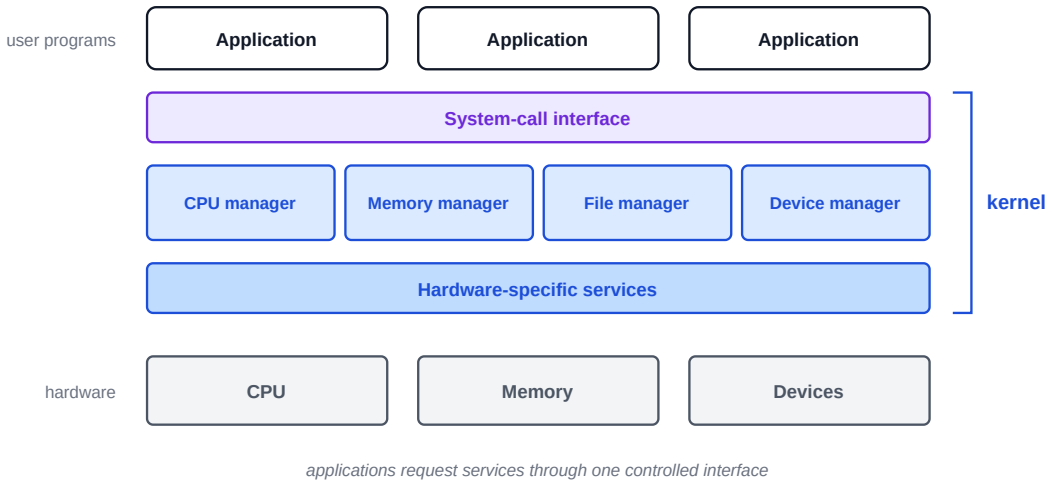


Figure 0.8: *The structure of an operating system. Applications use the system-call interface; the kernel's managers coordinate the hardware through hardware-specific services.*

4.1 Operating System Services

The services of a general-purpose OS fall into **three groups**.

Group	Services
For users	User interface (command line, graphical, touch, batch); program execution and termination
For programs	Uniform I/O operations through device drivers; file-system operations; communication between processes
For safe sharing	Resource allocation; logging and accounting; protection and security; error detection and handling

An **embedded OS** usually offers only a **small part of this list**. A microcontroller OS may provide only task scheduling, synchronisation, and time services, while the application keeps its own device drivers.

> **THE OS BOUNDARY IS A DESIGN CHOICE**
>
> What belongs to the OS is not a fixed feature list. It depends on what the product needs.

4.2 Privilege Levels, Traps, and System Calls

To protect itself and the hardware, an OS relies on CPU *execution modes*:

- **User mode (restricted)**: the mode of application code. A program in user mode can compute and access its own memory, but it cannot program I/O controllers, change memory-protection settings, or disable interrupts. If it tries, the CPU blocks the operation: depending on the processor and on the instruction, it raises an exception or silently ignores it.
- **Kernel mode (privileged)**: the mode of the kernel, where every instruction and every register is allowed. Only code in kernel mode can configure devices, memory protection, and interrupts.

> **IMPORTANT**
>
> The CPU switches from user to kernel mode only in controlled ways, through an **exception**, an **interrupt**, or a **system call**, so an application can never decide by itself to run with privileges.

When an application needs a privileged operation, it makes a ***system call***. A system call is implemented with a ***trap***, a special instruction that moves execution from user mode to kernel mode in a controlled way. The CPU switches mode, **saves the context** of the program (for example the program counter and some registers), and jumps to a fixed kernel entry point. The kernel checks the arguments, performs the service, prepares a return value, restores the context, and returns to user mode.

A SYSTEM CALL FROM C

The following program, written for a PC running Linux, prints a message on the terminal with a single system call.

```
#include <unistd.h>   /* write() */
#include <string.h>   /* strlen() */


int main( void )
{
    const char *msg = "Greetings\n";
    ssize_t n = write( 1, msg, strlen( msg ) );  /* 1 = standard output */
    return ( n < 0 ) ? 1 : 0;
}
```

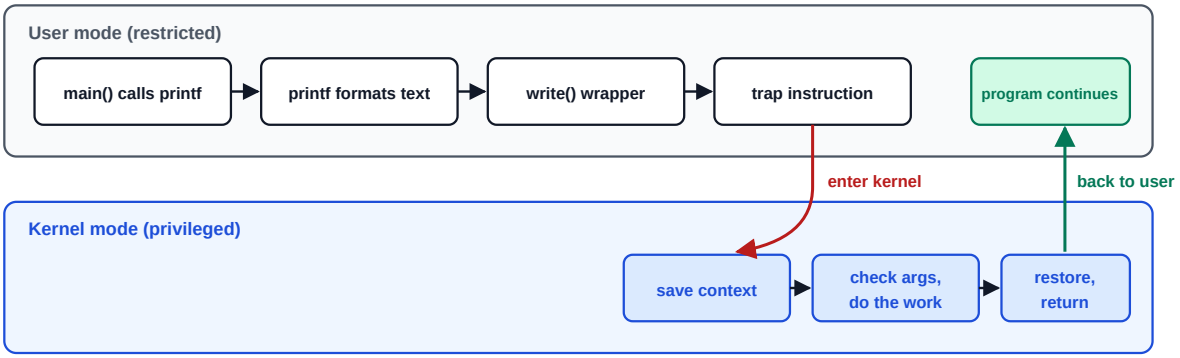
The call `write( 1, msg, strlen( msg ) )` asks the kernel to write some bytes to an output. Its three arguments say *where* to write, *what* to write, and *how many* bytes:

- `1` is a *file descriptor*, a small number that identifies an open file or device; descriptor 1 is the standard output, normally the terminal;
- `msg` is the address of the bytes to write;
- `strlen( msg )` is their number, here 10.

The program never touches the terminal itself, because in user mode it is not allowed to. The function `write` from the C library is only a thin *wrapper*: it places the service number and the three arguments in the registers where the kernel expects them and executes the trap. The kernel checks the request, its terminal driver prints the bytes, and control comes back to the program.

`write` returns the number of bytes actually written, or −1 if the operation failed. The program uses this value as its exit code: 0 means success, 1 means error.

Programs rarely execute the trap instruction themselves. They call library functions such as `printf`, which formats the text in user mode and then calls `write`, a thin *wrapper* that places the arguments where the kernel expects them and executes the trap. The return value tells the program whether the operation succeeded.



the same shape on Cortex-M: svc enters a privileged handler, exception return goes back

Figure 0.9: *A system call. The program reaches the trap through the C library, the kernel does the work in privileged mode, and control returns to the program in user mode.*

The Cortex-M3 implements the same mechanism with the `svc` instruction and the privileged Handler mode, as described in the chapter on the ARM Cortex-M3 architecture.

4.3 From Source Code to a Running OS

A program becomes **executable** in three steps:

1. The *compiler* turns each source file into a *relocatable object file*, whose final addresses are not fixed yet.
2. The *linker* combines the object files and libraries into one executable image (an ELF file), resolves the references between them, and assigns final addresses.
3. Finally, the image must be placed in memory and started: on a PC this is the job of a *loader*, which can also relocate the program to a different address.

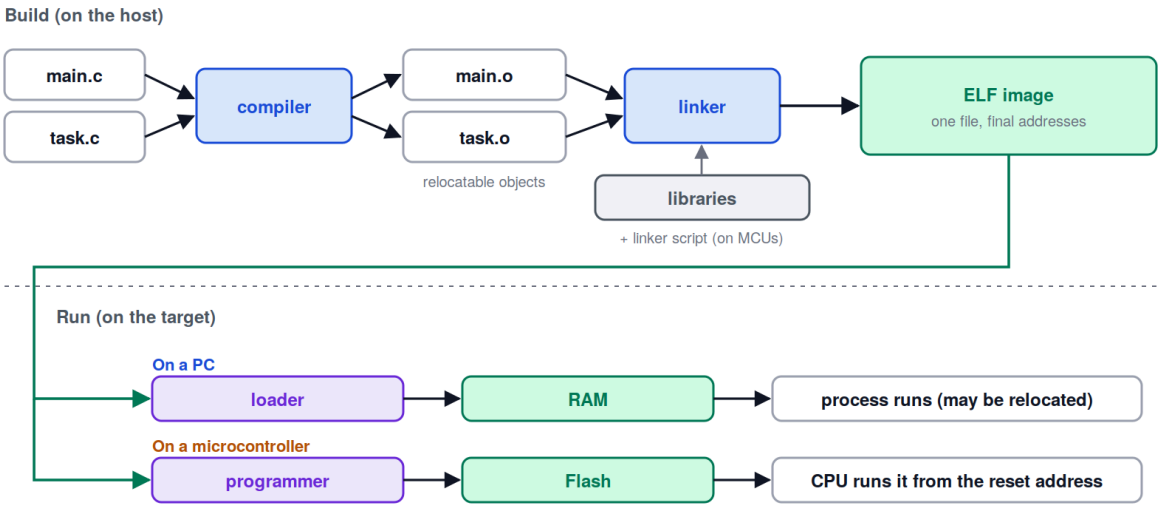


Figure 0.11: *From source code to a running program. The compiler and linker run on the host and produce one ELF image; a PC loads it into RAM with a loader, while a microcontroller has it written into Flash and runs it from the reset address.*

On a microcontroller the OS image is written into Flash with an external programmer during development, and the CPU runs it from its reset address.

Applications can be added to such a system in two ways:

- With ***fixed tasks***, the tasks are compiled and linked together with the OS into one image, and **no loader is needed**.
- With ***programmable tasks***, applications are separate images loaded after the OS starts; this allows updates without rebuilding the OS, at the cost of a loader, memory management, and security checks.

Most small embedded systems use **fixed tasks**.

5 Kernel Architectures

Operating systems differ in *where* their services run and *how strongly* they are isolated from each other and from applications.

Architecture	Where services run	Advantage	Cost	Examples
Flat	Everything in one image and one address space	Small and fast	Little or no isolation	FreeRTOS, MS-DOS
Monolithic	All services in one privileged kernel	Fast direct calls between services	Large trusted core; a bad driver can crash the kernel	Linux, UNIX
Microkernel	Minimal kernel; services as user-mode servers	Small trusted base, faults contained	IPC overhead, more interfaces	seL4, QNX
Hybrid	Mix of the above	Balance of speed and structure	Complexity	macOS (Mach + BSD), Linux modules

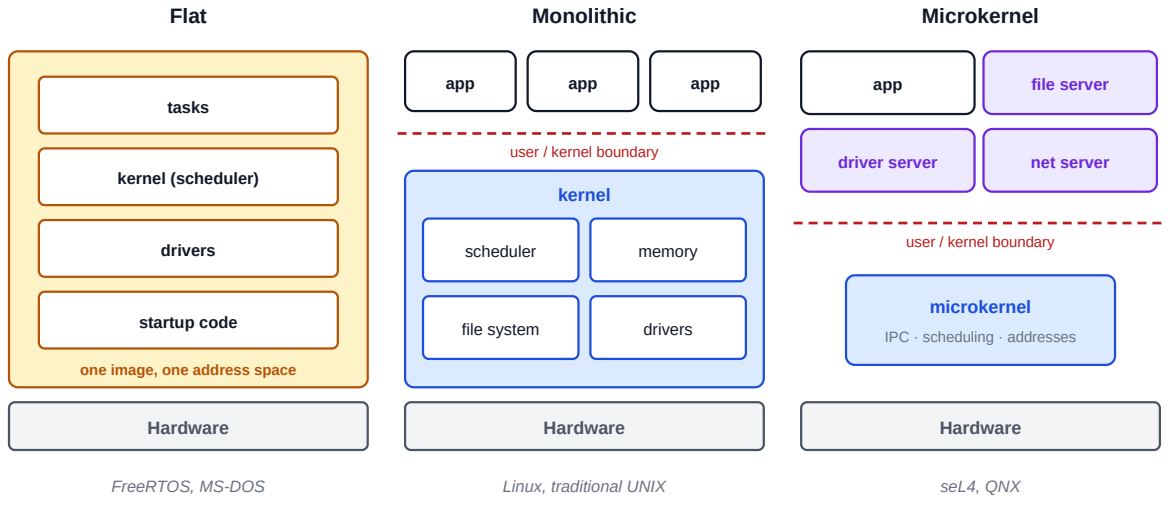


Figure 0.10: *Three kernel architectures. The red line is the hardware-enforced boundary between user and kernel mode; a flat system has none.*

1. In a ***flat* design**, startup code, kernel, drivers, and tasks are linked into one image and share one physical address space. OS services are ordinary functions that tasks call directly. No memory management unit and no privilege levels are needed, but any bug can corrupt any data.
2. A ***monolithic* kernel** separates user space from kernel space with hardware support (privilege levels and an **MMU** or **MPU**), so a **faulty application cannot corrupt the kernel**. Inside the kernel, however, scheduler, memory manager, file systems, and drivers share the same privileged space. Drivers are a large and error-prone part of the code, and a single faulty driver can crash the whole system.

3. A ***microkernel*** keeps only the **basic mechanisms in kernel mode**, typically scheduling, address-space management, and inter-process communication (IPC). File systems, drivers, and network stacks run as user-mode servers that exchange messages. A failed server can be restarted, and the small kernel is easier to verify. The **price is performance**: a system call in a monolithic kernel needs two mode switches (user to kernel and back), while the same request in a microkernel typically needs four (client to kernel, kernel to server, server to kernel, kernel to client).

Most real systems are *hybrids*. Linux is monolithic but loads drivers as modules at run time; macOS combines the Mach microkernel with BSD components.

> **CHOOSING A KERNEL ARCHITECTURE**
>
> Each architecture is a trade-off between isolation, fault containment, overhead, and complexity. The useful question is not "which one is best?" but "what must this system guarantee?".

6 Embedded and Real-Time Operating Systems

Embedded OSes differ from desktop OSes for three reasons:

1. They are ***resource-constrained***: a few KB of RAM leave no room for a large kernel.
2. They are ***real-time***: correctness depends on when things happen.
3. They usually have ***no untrusted users***: one vendor ships one firmware image with a known set of tasks. For these reasons an embedded OS is selected and configured for one product, not for an unknown future workload.

General-purpose mechanism	Embedded replacement
Dynamic allocation	Static number of tasks and fixed-size buffers
General fairness policy	Simple priority-based scheduler
Layered device stack	Direct register access
Maximum throughput	Predictable execution time

> **SMALL IS NOT INCOMPLETE**
>
> A small OS is complete when it matches the responsibilities of the system.

A ***real-time operating system*** **(RTOS)** such as FreeRTOS provides ready-made versions of the mechanisms an embedded application needs: tasks with priorities, delays and timer-driven scheduling, queues, semaphores and mutexes, and communication from interrupts to tasks. By default FreeRTOS has no memory protection between tasks: like flat firmware, all tasks share one address space. Knowing the mechanisms underneath an RTOS makes its API much easier to use correctly.

6.1 Single Image, Single Failure Domain

Many microcontroller products ship one firmware image that contains startup code, drivers, OS, and application. **Updating the image changes the whole system**. This makes deployment simple, but it requires discipline: a bad pointer can damage any writable data, a blocked high-priority task can stop an important service, and recovery and diagnostics must be designed into the firmware from the start. Scheduling, synchronisation, and memory protection are the mechanisms that reduce these risks.

6.2 From Bare Metal to a Minimal OS

A minimal flat OS is built as a **chain of mechanisms**, where each one depends on the previous one. This chain is also the path followed by the rest of these notes, on an ARM Cortex-M3 microcontroller emulated by QEMU.

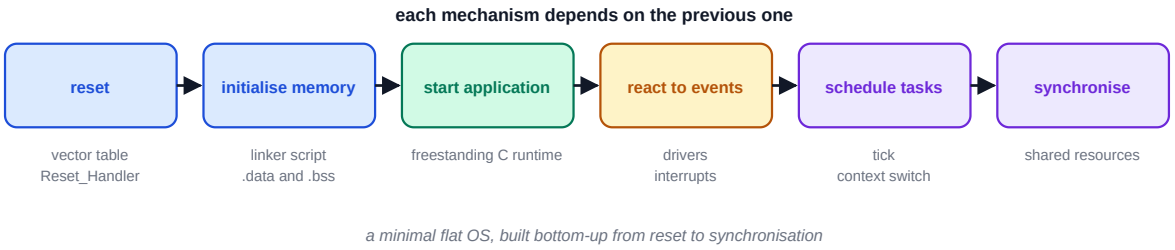


Figure 0.11: *The mechanisms of a minimal OS, from reset to synchronisation, with the main topics involved in each step.*

- **Reset**: the processor must start correctly after reset, which requires a vector table and a reset handler.
- **Initialise memory**: memory must be prepared as described by the linker script, copying `.data` and zeroing `.bss`, so that C code can run.
- **Start the application**: the freestanding C runtime calls `main()`, without any operating system underneath.
- **React to events**: the application reacts to the outside world through device drivers and interrupts.
- **Schedule tasks**: with a periodic timer tick and context switching, the kernel runs several tasks.
- **Synchronise**: the kernel coordinates the tasks' access to shared resources. Memory protection and system calls, built on the privilege levels of the Cortex-M3, add isolation on top of this chain.

1 ARM Cortex-M3 Core

The documentation of a Cortex-M3 system is split into **three layers**, each defined by a different party. When looking for information, the first step is to know which layer answers the question.

- **Architecture (ARMv7-M)**: a **specification** written by ARM, not a physical chip. It fixes what software sees: the registers and their roles (the *programmer's model*), the instruction set, the way interrupts and faults are handled (the *exception model*), and the meaning of each region of the address space.
- **Core (Cortex-M3)**: the processor that ARM designs following that specification. It adds implementation details, such as the 3-stage pipeline, and a fixed set of *core peripherals*: the NVIC, the SysTick timer, the System Control Block, and an optional MPU.
- **Chip or board**: the physical product that a silicon vendor builds around the licensed core. The vendor decides how much Flash and SRAM exist, which device peripherals (UARTs, timers) are connected, and at which addresses.

Layer	Defined by	Examples of what it fixes
Architecture (ARMv7-M)	ARM	Registers, Thumb-2 instructions, exception numbers, address regions
Core (Cortex-M3)	ARM	NVIC, SysTick, SCB, MPU, 3-stage pipeline
Chip / board	Vendor	Memory sizes, device peripherals, IRQ assignment, clock

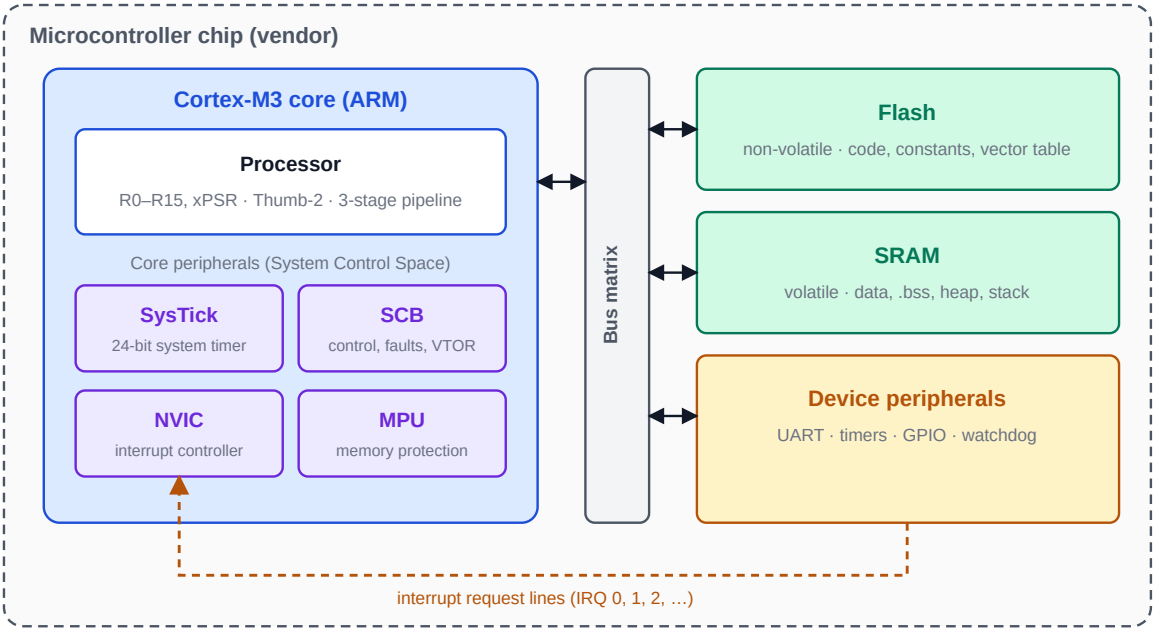


Figure 1.1: *Block diagram of a Cortex-M3 microcontroller: the ARM core with processor and core peripherals, the bus matrix, and the vendor's Flash, SRAM, and device peripherals.*

Everything in this chapter belongs to the first two layers and is therefore identical on every Cortex-M3. Board-specific values are taken from the `mps2-an385` platform (an FPGA image of a Cortex-M3 system, emulated by QEMU, clocked at 25 MHz) and are marked as such.

2 Programmer's model: registers and ISA

The programmer's model is the set of registers visible to software and to the debugger. At this level there are **no variables**, only registers and memory: a C statement such as `x = x + 1` becomes "load `x` into a register, add 1, store it back".

The registers of the Cortex-M3 fall into two groups:

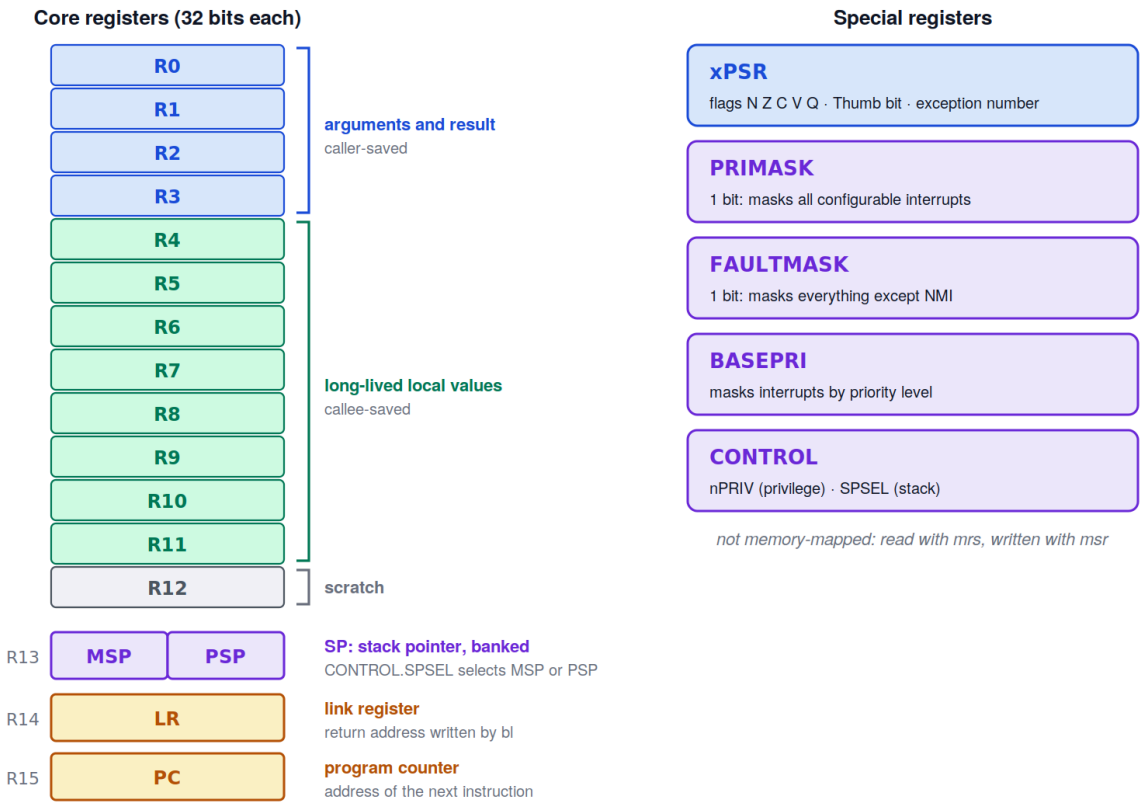
- **Core registers** (`R0` – `R15`): they are the operands of ordinary instructions, such as `add r0, r1, r2` or `ldr r3, [r0]` .
- **Special registers** (`xPSR` , `PRIMASK` , `FAULTMASK` , `BASEPRI` , `CONTROL`): they describe and control the state of the processor, and cannot be used as operands; they are read and written only with the dedicated instructions `mrs` and `msr` .

None of them has a memory address: they live inside the CPU. All of them are **32 bits** wide.

2.1 Core Registers

Register	Name	Role
`R0` – `R12`	General purpose	Operands and addresses; `R0` – `R3` carry function arguments
`R13`	`SP`	Stack pointer: address of the top of the stack
`R14`	`LR`	Link register: return address after a function call
`R15`	`PC`	Program counter: address of the next instruction
—	`xPSR`	Program status: flags, Thumb bit, current exception number

The stack is **full descending**: it grows towards lower addresses, and `SP` points to the last occupied word. A push first decrements `SP` by 4 and then writes at the new `SP` . `R13` is a banked name: two physical stack pointers exist (`MSP` and `PSP`), and `SP` always refers to the one currently selected (see the section on execution modes).



Cortex-M3 registers: *thirteen general-purpose registers with their AAPCS roles, the banked stack pointer, the link register, the program counter, and the special registers accessed with mrs and msr.*

2.2 Special Registers

Special registers hold the **state of the processor** rather than data. They cannot be used as operands of ordinary instructions such as `add` or `ldr`, and they have **no memory address**. They are moved to and from a core register with two dedicated instructions:

```
mrs r0, primask     @ move register from special: r0 = PRIMASK
msr control, r0     @ move special from register: CONTROL = r0
```

The names follow the direction of the copy: `mrs` copies a **special** register into a core **register**, `msr` copies a core **register** into a **special** register. To change a single bit, the usual read-modify-write sequence is needed: `mrs` to read, `orr` or `bic` to modify, `msr` to write back.

Special register	Content	Described in
`xPSR`	Program status: flags, Thumb bit, current exception number	The Program Status Register xPSR
`CONTROL`	Thread-mode privilege and stack selection	The CONTROL Register
`PRIMASK`	1 bit: masks every exception with configurable priority	The PRIMASK Register
`FAULTMASK`	1 bit: also masks HardFault; only NMI is taken	—

Special register	Content	Described in
`BASEPRI`	Masks exceptions whose priority value is greater than or equal to it	—

`FAULTMASK` and `BASEPRI` are listed for completeness: the firmware described in these notes does not use them.

> **IMPORTANT**
>
> Writing `CONTROL`, `PRIMASK`, `FAULTMASK`, and `BASEPRI` is reserved to privileged code. In unprivileged code the write is silently ignored: no fault is raised, the register simply keeps its value.

2.2.1 The Program Status Register xPSR

`xPSR` holds neither data nor addresses: it **describes** the current **state of the processor**. The name reflects that it is the union of three sub-registers occupying disjoint bits of the same 32-bit word.

Sub-register	Bits	Content
`APSR` (Application)	31–28	Flags `N` (negative), `Z` (zero), `C` (carry), `V` (overflow)
`EPSR` (Execution)	24	`T`: Thumb state, always 1 on Cortex-M
`IPSR` (Interrupt)	8–0	Number of the exception being handled, 0 if none



APSR: condition flags · EPSR: Thumb state (always 1) · IPSR: running exception (0 = Thread mode)

Figure 1.2: *Bit layout of xPSR: N, Z, C, V flags in APSR, Thumb bit in EPSR, exception number in IPSR.*

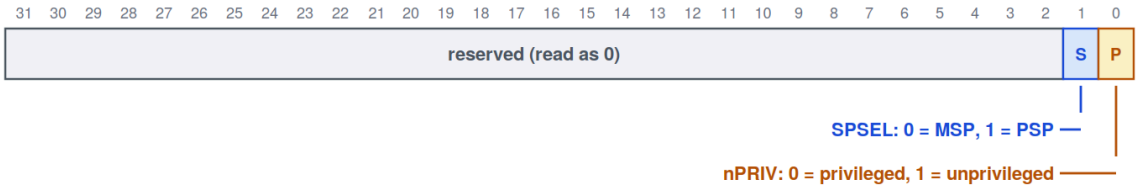
The flags are written by comparison and arithmetic instructions and read by conditional branches; this is how every `if` and loop condition is implemented at machine level.

> **EXAMPLE**
>
> Flags in action `cmp r1, r2` computes `r1 - r2`, discards the result, and updates only the flags: `Z = 1` if the operands are equal, `C = 1` if `r1 >= r2` as unsigned values. The following `bhs done` reads `C` and branches if it is 1.

`IPSR` identifies the execution mode: 0 means Thread mode (ordinary code), any other value is the number of the exception whose handler is running.

2.2.2 The CONTROL Register

`CONTROL` decides **how ordinary code runs**: with or without privileges, and on which of the two stacks. Only its **two lowest bits are used** on the Cortex-M3; the other bits are reserved and read as 0.



both bits act only in Thread mode; Handler mode is always privileged and always uses MSP

CONTROL register: *bit 1 (SPSEL) selects the Thread-mode stack, bit 0 (nPRIV) the Thread-mode privilege; bits 31–2 are reserved.*

Bit	Name	Meaning
0	`nPRIV`	0 = privileged, 1 = unprivileged (Thread mode only)
1	`SPSEL`	Thread-mode stack pointer: 0 = `MSP`, 1 = `PSP`

The two bits give four possible values:

`CONTROL`	Thread privilege	Thread stack	Typical use
`0`	Privileged	`MSP`	State after reset: boot code and kernel
`1`	Unprivileged	`MSP`	Rarely used
`2`	Privileged	`PSP`	Rarely used
`3`	Unprivileged	`PSP`	Application task with its own stack

Both bits act only in **Thread mode**, the mode of ordinary code. Exception handlers run in Handler mode, which is always privileged and always uses `MSP`, whatever `CONTROL` contains. The meaning of the two modes, of privilege, and of the two stacks is described in the section on execution modes, privilege, and stacks.

`CONTROL` is changed with a read-modify-write on a special register, followed by an `isb` instruction:

```
mrs r0, control     @ r0 = CONTROL
orr r0, r0, #1      @ set nPRIV
msr control, r0     @ CONTROL = r0: Thread mode is now unprivileged
isb                 @ apply the new setting to the following instructions
```

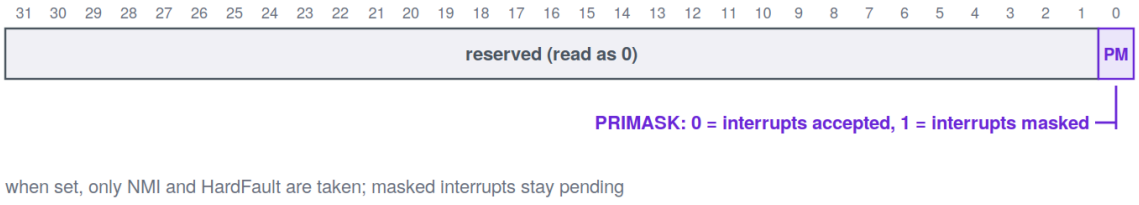
The `isb` (instruction synchronization barrier) is needed because the CPU has already fetched the next instructions under the old setting; `isb` discards them, so that every following instruction runs with the new value.

> **IMPORTANT**
>
> A one-way change Only privileged code can write `CONTROL`. Once Thread mode has become unprivileged, a `msr control` that tries to clear `nPRIV` is silently ignored: the only ways back to privileged execution are a reset or an exception, whose handler runs privileged in Handler mode.

2.2.3 The PRIMASK Register

`PRIMASK` is a one-bit register that **switches interrupts off and on**. Only bit 0 is used; the other bits are reserved

and read as 0.



PRIMASK register: *bit 0 masks every exception with configurable priority; bits 31–1 are reserved.*

PRIMASK	Effect
`0`	Interrupts are accepted normally (value after reset)
`1`	Every exception with configurable priority is held back: device interrupts, SysTick, PendSV, SVCall. Only NMI and HardFault can still be taken

Masked interrupts are **not lost**: they remain *pending*, and the CPU takes them as soon as `PRIMASK` returns to 0. Exceptions, priorities, and pending state are described in the section on the exception model.

> **WARNING**
>
> `svc` with interrupts masked SVCall is the exception that cannot wait: `svc` is a synchronous request and cannot stay pending. Executed while `PRIMASK` is 1, it escalates to HardFault.

`PRIMASK` is usually changed with two dedicated instructions, which do not need a core register:

Assembly	C (CMSIS)	Effect
`cpsid i`	`__disable_irq();`	`PRIMASK = 1`: interrupts masked
`cpsie i`	`__enable_irq();`	`PRIMASK = 0`: interrupts accepted

The main use of `PRIMASK` is a *critical section*: a short piece of code that must run without being interrupted, for example while updating a variable that an interrupt handler also uses. Boot code also masks interrupts at the start and unmasks them only once the vector table, handlers, and priorities are configured.

A critical section must restore the **previous** value of `PRIMASK`, not simply unmask interrupts. If a critical section runs inside another one, an unconditional `cpsie i` at its end would re-enable interrupts while the outer section is still running. The safe pattern saves `PRIMASK` with `mrs` and restores it with `msr`:

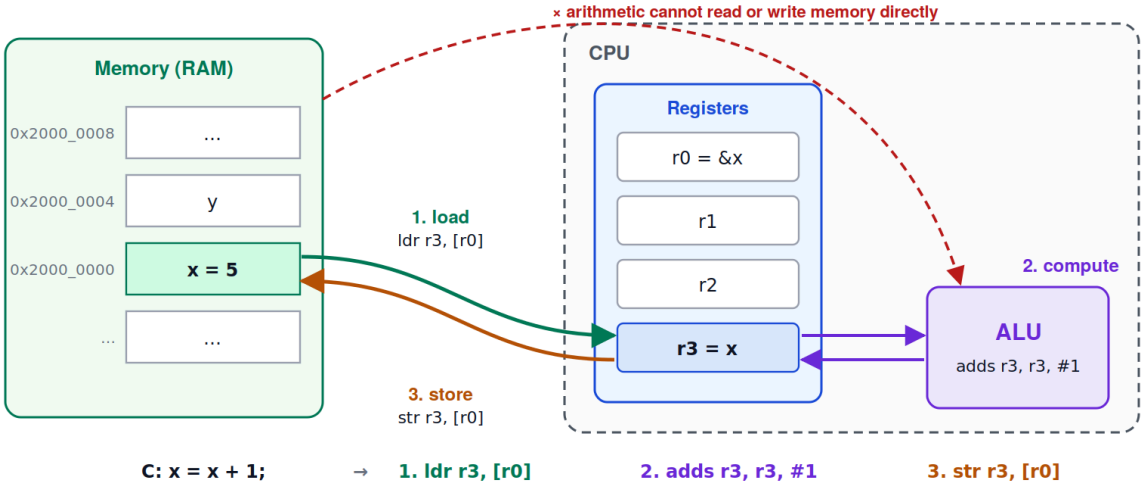
```
mrs   r0, primask   @ save the previous value (0 or 1)
cpsid i             @ mask interrupts
...                 @ critical section
msr   primask, r0   @ restore the previous value
```

> **IMPORTANT**
>
> Only privileged code can change `PRIMASK`. In unprivileged code `cpsid i`, `cpsie i`, and `msr primask` are silently ignored, so an application task cannot switch interrupts off.

2.3 Load/Store Architecture

The Cortex-M3 has a **load/store instruction set**: arithmetic and logic operate **only on registers**, and memory is accessed exclusively through load (`ldr` , memory to register) and store (`str` , register to memory) instructions. **Every operation** on a variable therefore **passes through a register**.

Two rules help to read every instruction below. The first operand is usually the destination (`ldr r3, …` writes `r3`), except in stores, where the first register is the value to save. Square brackets `[ ]` always mean "**the memory at this address**".



Load/store architecture: *memory is reached only through loads and stores; arithmetic works on registers. The C statement x = x + 1 needs three instructions.*

2.3.1 Load/Store Instructions

Load and store instructions are the only instructions that **access memory**. A *load* copies a value from memory into a register, a *store* copies a register into memory. The address is always taken from a register, written in square brackets, and can be adjusted by an offset (`[r0, #7]`) or updated after the access (`[r0], #4`). Suffixes select the size of the access: no suffix for a 32-bit word, `b` for a byte, `h` for a halfword (16 bits).

Assembly	C equivalent	Explanation
`ldr r3, [r0]`	`r3 = *r0;`	*Load register*: read the 32-bit word at address `r0` into `r3`
`str r3, [r1]`	`*r1 = r3;`	*Store register*: write `r3` to memory at address `r1`
`ldr r3, [r0], #4`	`r3 = *r0; r0 += 4;`	Load, then add 4 to `r0` so it points to the next word: used to walk arrays
`strb r1, [r0, #7]`	`((uint8_t *)r0)[7] = r1;`	*Store byte*: write only the low byte of `r1` at `r0 + 7` ; `r0` does not change
`ldr r0, =0x20400000`	`r0 = 0x20400000;`	Pseudo-instruction: load a 32-bit constant written as a number, through the literal pool

Assembly	C equivalent	Explanation
`ldr r0, =_estack`	`r0 = (uint32_t)&_estack;`	Same as above, with the value given by a symbol: the linker replaces `_estack` with `0x20400000`

2.3.1.1 LOADING A 32-BIT CONSTANT: THE LITERAL POOL

A Thumb instruction is 16 or 32 bits long. These bits must hold the operation and the registers, so there is no space left for a full 32-bit value such as the address `0x20400000`. The constant must therefore be stored in memory and loaded from there.

The pseudo-instruction `ldr r0, =0x20400000` does this automatically. The assembler:

1. writes the value `0x20400000` in memory, as data, just after the code; this small area of constants is called the *literal pool*;
2. replaces the pseudo-instruction with a normal load that reads that word. The load gives the position of the word as a distance from the current instruction, so a disassembler shows it as `ldr r0, [pc, #4]`, which means "read the word 4 bytes ahead of here".

```
0x48:  ldr r0, [pc, #4]     reads the word at 0x50 into r0
0x4A:  ...                  rest of the code
0x4E:  b   elsewhere        the CPU jumps away before reaching 0x50
0x50:  0x20400000           literal pool: a constant, not an instruction
```

The CPU must never execute the literal pool: if it did, it would read the bytes of `0x20400000` as an instruction. The directive `.ltorg` tells the assembler where to put the pool, and it is placed where execution never arrives, for example right after an unconditional branch.

2.3.2 Data-Processing Instructions

Data-processing instructions compute on registers: arithmetic (`add`, `sub`), logic (`and`, `orr`), bit shifts, copies between registers, and comparisons. They **never touch memory**. The destination is the first operand; the last operand can be a register or a small constant written with `#`. A trailing `s` (as in `movs`) also updates the flags of `xPSR`, and comparison instructions do nothing else.

Assembly	C equivalent	Explanation
`movs r3, #0`	`r3 = 0;`	*Move*: put a small constant in `r3` and update the flags
`mov sp, r0`	`sp = r0;`	*Move*: copy one register into another, here into the stack pointer
`orr r1, r1, #0x10`	`r1 |= 0x10;`	Bitwise *OR*: set bit 4 of `r1`, leave the other bits unchanged
`cmp r1, r2`	(compute `r1 - r2`)	*Compare*: subtract, update the flags with the result, then discard the result

2.3.3 Branch Instructions

Branch instructions **change the flow of execution** by **loading a new address into the program counter**. An unconditional branch always jumps; a conditional branch jumps only if the flags satisfy its condition, which is

written as a suffix (`eq` , `ne` , `hs` , `lt` …) and usually tests the result of a previous `cmp` . A *branch with link* also saves the return address in `lr` , which is how function calls work.

Assembly	C equivalent	Explanation
`b label`	`goto label;`	*Branch*: always jump to `label`
`bhs done`	`if (r1 >= r2) goto done;`	*Branch if higher or same*: jump only if the last `cmp` found `r1 >= r2` (unsigned)
`bl main`	`main();`	*Branch with link*: save the return address in `lr` , then jump; return is `bx lr`

2.3.3.1 WRITING A LOOP: COMPARE AND BRANCH

Assembly has no `while` or `for` . A loop is built from three pieces: a compare, a conditional branch that leaves the loop, and an unconditional branch that goes back to the start.

The C loop:

```
while ( r1 < r2 ) {
    /* body */
}
```

becomes:

```
loop:   cmp   r1, r2        @ 1. compare r1 with r2
        bhs   done          @ 2. if r1 >= r2, jump to done (exit)
        ...                 @ 3. body of the loop
        b     loop          @ 4. jump back to step 1
done:                       @ code after the loop
```

`loop:` and `done:` are *labels*: names given to code addresses, used as targets of branches.

- **Step 1**: `cmp` subtracts `r2` from `r1` , throws the result away, and only updates the flags, which record how the comparison went.
- **Step 2**: `bhs` (*branch if higher or same*) looks at the flags and jumps to `done` if `r1 >= r2` . It tests the opposite of the C condition, because it decides when to *leave* the loop.
- **Step 3**: if the branch is not taken, the CPU simply continues with the next instructions, which are the body of the loop.
- **Step 4**: `b` always jumps back to `loop` , so the comparison is repeated before every new iteration.

2.3.4 Special Registers Access Instructions

Special registers cannot be used as operands of ordinary instructions, so a separate group of instructions moves them to and from core registers. `cpsid` and `cpsie` are shortcuts that change only `PRIMASK` , without going through a core register. Writing a special register is allowed only in privileged code; in unprivileged code the write is silently ignored.

Assembly	C equivalent	Explanation
`mrs r0, control`	`r0 = CONTROL;`	*Move register from special*: copy a special register into a core register
`msr control, r0`	`CONTROL = r0;`	*Move special from register*: copy a core register into a special register
`mrs r0, ipsr`	`r0 = IPSR;`	Read one part of `xPSR`, here the number of the running exception
`cpsid i`	`__disable_irq();`	*Change processor state, interrupts disable*: set `PRIMASK` to 1, masking interrupts
`cpsie i`	`__enable_irq();`	*Change processor state, interrupts enable*: clear `PRIMASK` to 0

2.4 Thumb State and the Thumb Bit

The Cortex-M executes only the Thumb-2 instruction set, a mix of 16-bit and 32-bit instructions. This has a side effect on code addresses:

- **Bit 0 is free**: instructions always start at an even address, so bit 0 of a code address is always 0 and carries no information.
- **ARM reuses it as the Thumb bit**: every code pointer the CPU jumps to (a vector table entry, a function pointer, the target of `bx`) must have bit 0 set to 1.
- **The CPU clears it before jumping**: it copies the bit into `xPSR.T`, sets it to 0, and jumps to the even address.
- **Example**: a handler located at `0x40` is stored in the vector table as `0x41`.

> ⚠ **THUMB BIT EQUAL TO 0**
>
> Branching to an address with bit 0 clear requests ARM state, which the Cortex-M does not implement. The result is a UsageFault (`INVSTATE`), not an ordinary crash at the target address.

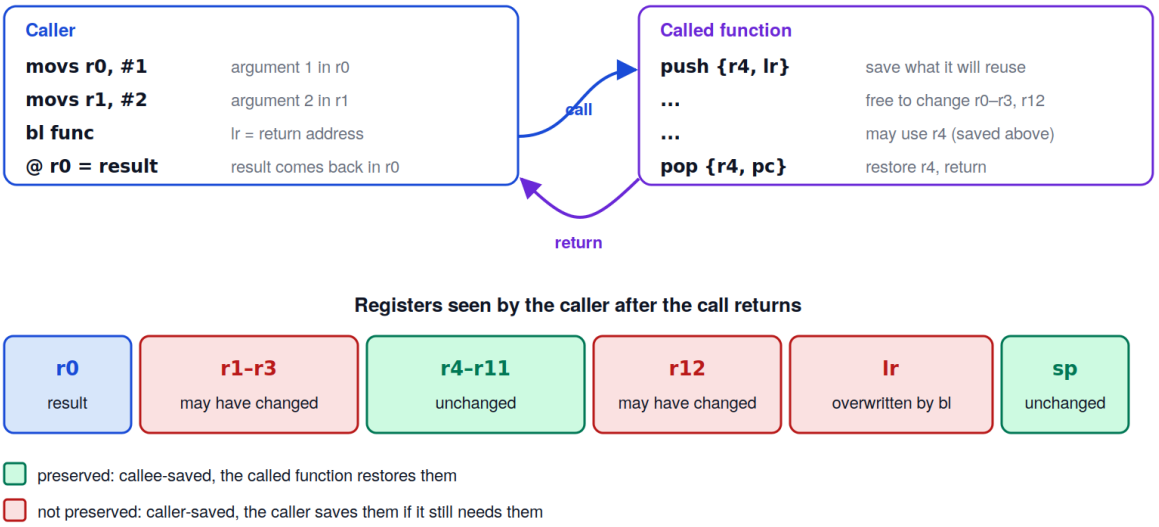
2.5 Calling Convention (AAPCS)

The **AAPCS (Procedure Call Standard for the Arm Architecture)** defines how functions **exchange arguments** and **which registers** they must preserve. Compiled C and hand-written assembly interoperate only if both follow it, and the exception model is designed around it.

Register	Role in a call	Preserved across the call
`r0` – `r3`	First four arguments; `r0` also returns the result	No (caller-saved)
`r4` – `r11`	Long-lived local values	Yes (callee-saved)
`r12`	Scratch	No (caller-saved)
`lr`	Return address written by `bl`	No; a non-leaf function saves it
`sp`	Multiple of 8 at every public call	Yes

Registers are shared: when a function calls another one, the called function may overwrite values the caller still needs. The AAPCS splits the responsibility for saving them:

- `r0` – `r3` **and** `r12` **are the caller's problem.** The called function may change them freely. If the caller still needs one of these values after the call, it must save it before calling.
- `r4` – `r11` **are the called function's problem.** The caller finds them unchanged after the call. A function that wants to use `r4` must save its old value at entry and restore it before returning, typically with `push {r4, lr}` at entry and `pop {r4, pc}` at exit; the same `pop` also returns, because it loads the saved `lr` into `pc`.
- **A leaf function may save nothing.** A *leaf function* calls no other function, so `lr` is never overwritten. If it also uses only `r0` – `r3` and `r12`, it has nothing to restore and does not touch the stack at all.



AAPCS function call: *arguments travel in r0–r3 and the result in r0; the called function restores r4–r11 and sp, while r0–r3, r12, and lr may be changed by the call.*

EXAMPLE

A leaf function: `add3` Consider a minimal C function that adds three numbers:

```
int add3( int a, int b, int c )
{
    return a + b + c;
}
```

Following the AAPCS, the arguments `a` , `b` , `c` arrive in `r0` , `r1` , `r2` , and the result must be returned in `r0` . The compiler translates the function into three instructions:

```
add3:
    adds r0, r0, r1     @ r0 = a + b
    adds r0, r0, r2     @ r0 = r0 + c
    bx   lr             @ return; the result is in r0
```

A call such as `r = add3( 1, 2, 3 );` places the arguments in registers and jumps to the function:

```
    movs r0, #1         @ a = 1
    movs r1, #2         @ b = 2
    movs r2, #3         @ c = 3
    bl   add3           @ lr = return address, jump to add3
                        @ back here: r0 = 6
```

`add3` is a *leaf function*: it calls no other function, so `lr` is never overwritten, and it uses only `r0` – `r2` , which are caller-saved. It therefore saves nothing: no `push` , no `pop` , no use of the stack, and it returns with a simple `bx lr` .

3 Address Space and Memory-Mapped Registers

The Cortex-M3 sees a single 4 GB physical address space (32-bit addresses). The architecture divides it into regions and fixes the purpose of each one; the **vendor decides how much memory or which devices actually populate them**.

There is **no address translation** (no MMU, hence no virtual memory): the address issued by an instruction is the address seen by memory and peripherals, and no run-time loader chooses where a program is placed. All placement decisions are made at link time and recorded in the image.

3.1 Address Space Overview

Address range	Region	Purpose	Populated on `mps2-an385`
`0x0000_0000` – `0x1FFF_FFFF`	Code	Program memory: code, constants, vector table at `0x0`	4 MB at `0x0` ("FLASH")
`0x2000_0000` – `0x3FFF_FFFF`	SRAM	Data memory: variables, heap, stack	4 MB at `0x2000_0000` ("RAM")
`0x4000_0000` – `0x5FFF_FFFF`	Peripherals	Registers of the on-chip devices added by the vendor	UARTs, timers, watchdog
`0x6000_0000` – `0xDFFF_FFFF`	External RAM / devices	Memory and devices outside the chip	Not used
`0xE000_0000` – `0xE00F_FFFF`	Private Peripheral Bus	Registers of the core peripherals (NVIC, SysTick, SCB, MPU) and debug units	System Control Space used by the firmware
`0xE010_0000` – `0xFFFF_FFFF`	Vendor-specific	System area reserved for the vendor	Not used in these notes

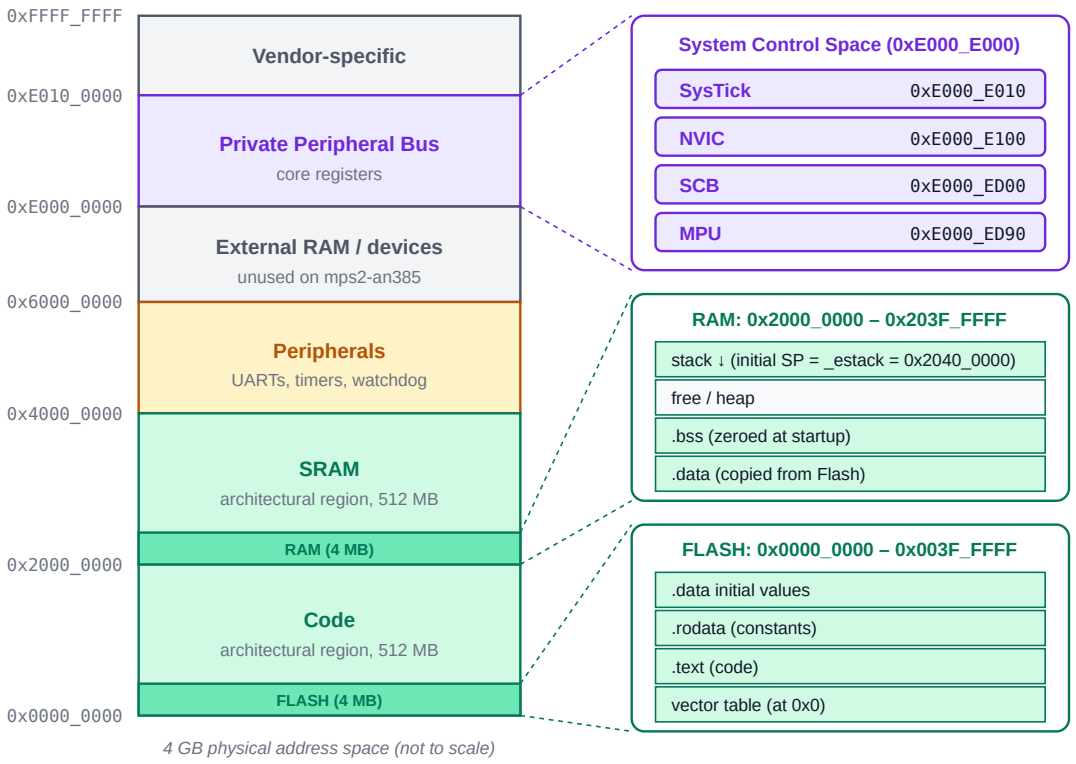


Figure 1.3: *Cortex-M3 address space with the regions populated on mps2-an385 and the System Control Space in detail.*

A region is only a range of addresses: memory or registers exist only where the chip connects something, and an access to an empty address raises a BusFault. On `mps2-an385` only a small part of the 4 GB is populated:



The External RAM / devices and Vendor-specific regions are not used on this board, so the following sections cover only the three populated regions, from the lowest address to the highest.

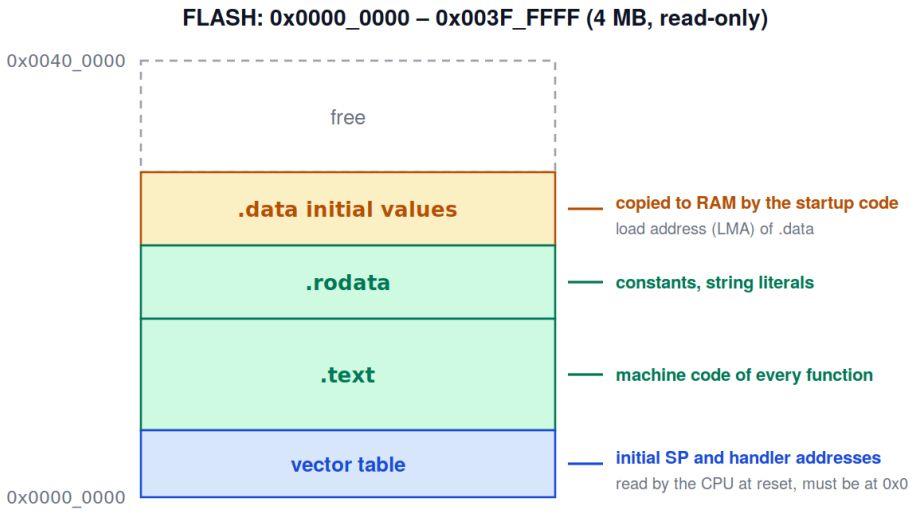
3.2 Code and SRAM: the Memories

The two lowest regions hold the **memories of the chip**. On `mps2-an385`, 4 MB of program memory ("**FLASH**") sit at the start of the Code region, and **4 MB** of data memory ("**RAM**") at the start of the **SRAM region**.

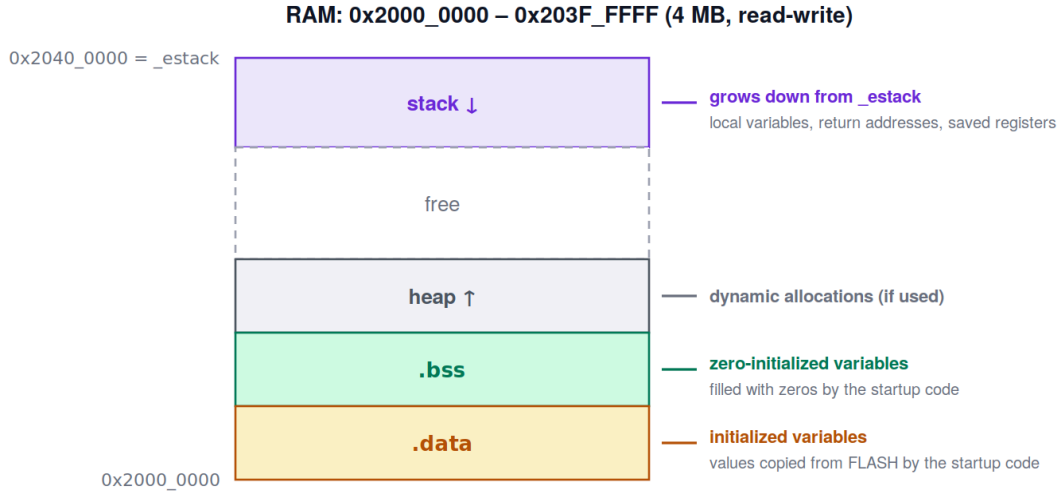
- **Flash** keeps its contents without power. It holds the vector table, the code, the constants, and the initial values of initialized variables.

- **SRAM** is fast writable memory, but its contents are undefined at power-on. It holds the variables (initialized and zeroed), the heap, and the stack.

As a consequence, before C code can run safely, the startup code must copy the initial values of variables from Flash to SRAM and fill the zeroed variables with zeros. Which part of the program goes into which memory is decided by the **linker script**, described in the chapter on boot and CPU configuration.



FLASH contents: *the vector table at address 0x0, the code, the constants, and the initial values of .data.*



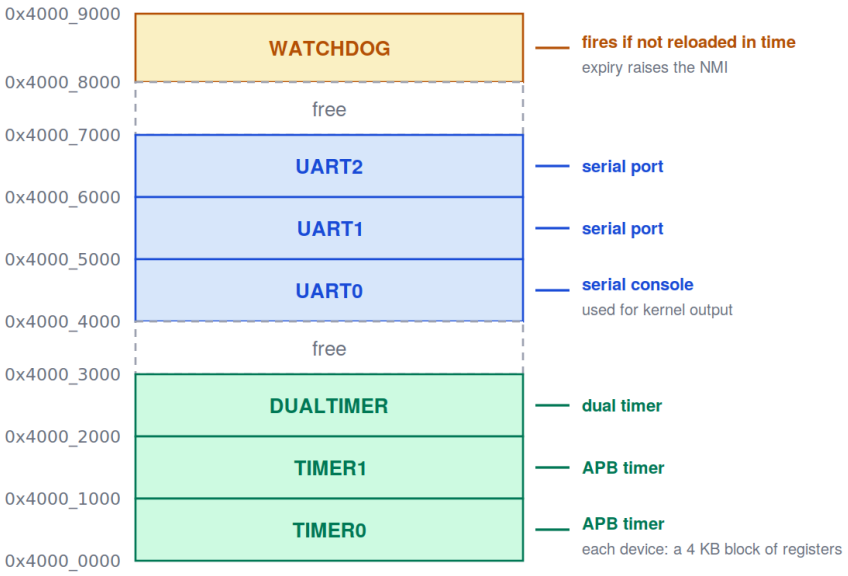
RAM contents: *the variables of .data and .bss at the bottom, the heap above them, and the stack growing down from the top.*

> **INFO**
>
> Emulated flash On `mps2-an385` , and in QEMU, the 4 MB at address `0x0` are fast RAM preloaded with the program image. They play the role of Flash and are treated as read-only, exactly like real Flash on a microcontroller.

3.3 Peripherals: Device Registers

The Peripherals region holds the registers of the devices that the vendor adds around the core: **timers, UARTs, GPIO,** and so on. Each device occupies a small block of addresses, often **4 KB**, and is controlled by reading and writing its registers. Which devices exist, and at which addresses, depends entirely on the chip: on `mps2-an385` they include three UARTs, two timers, a dual timer, and a watchdog, shown in the memory map above. These devices and their drivers are described in the chapter on devices, interrupts, and system calls.

Peripherals region on mps2-an385: devices used by the firmware



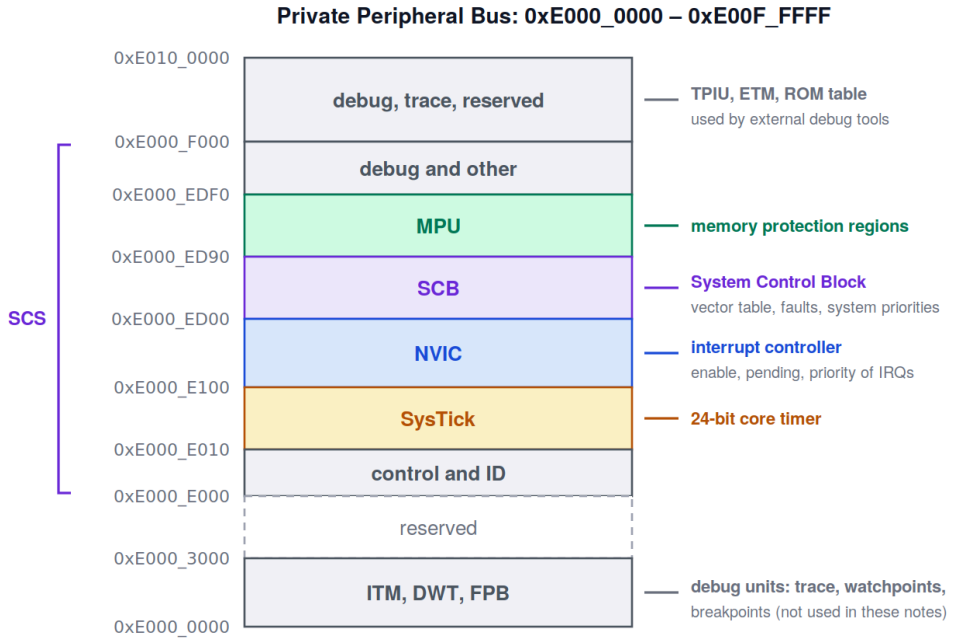
Peripherals contents: *on mps2-an385 each device occupies a 4 KB block of registers; only part of the 512 MB region is used.*

3.4 PPB: the Core Peripherals

The **Private Peripheral Bus** holds the registers of the core peripherals, together with debug and trace units that are not used in these notes. Inside it, the **System Control Space** (SCS, `0xE000_E000` – `0xE000_EFFF`) groups the four core peripherals, which are identical on every Cortex-M3 regardless of the vendor.

Core peripheral	Base address	Purpose
SysTick	`0xE000_E010`	24-bit periodic timer
NVIC	`0xE000_E100`	Enabling, pending state, and priority of device interrupts

Core peripheral	Base address	Purpose
SCB (System Control Block)	`0xE000_ED00`	Vector table location, fault configuration and status, system priorities
MPU	`0xE000_ED90`	Memory protection regions



PPB contents: *the System Control Space holds the registers of the four core peripherals; the rest of the bus belongs to debug and trace units.*

System Control Space (privileged access only)

0xE000_EFFF
Other 528 B
debug and other registers
0xE000_EDF0

0xE000_EDEF
MPU 96 B
memory protection unit
0xE000_EDA0 ◀ **RASR** region size and permissions
0xE000_ED9C ◀ **RBAR** region base address
0xE000_ED98 ◀ **RNR** region number
0xE000_ED94 ◀ **CTRL** enable, PRIVDEFENA
0xE000_ED90 ◀ **TYPE** number of regions
0xE000_ED90

0xE000_ED8F
SCB 144 B
System Control Block
0xE000_ED38 ◀ **BFAR** BusFault address
0xE000_ED34 ◀ **MMFAR** MemManage fault address
0xE000_ED2C ◀ **HFSR** cause of HardFault
0xE000_ED28 ◀ **CFSR** cause of configurable faults
0xE000_ED24 ◀ **SHCSR** enable fault handlers
0xE000_ED18 ◀ **SHPR1** system exception priorities (SHPR1–3)
0xE000_ED14 ◀ **CCR** fault traps, stack alignment
0xE000_ED0C ◀ **AIRCR** reset, priority grouping
0xE000_ED08 ◀ **VTOR** vector table address
0xE000_ED04 ◀ **ICSR** pending state, PendSV trigger
0xE000_ED00 ◀ **CPUID** core identification
0xE000_ED00

0xE000_ECFF
NVIC 3 KiB
interrupt controller
0xE000_E400 ◀ **IPR** priority, 1 byte per IRQ
0xE000_E300 ◀ **IABR** active (handler running)
0xE000_E280 ◀ **ICPR** clear-pending
0xE000_E200 ◀ **ISPR** set-pending
0xE000_E180 ◀ **ICER** clear-enable
0xE000_E100 ◀ **ISER** set-enable, 1 bit per IRQ
0xE000_E100

0xE000_E0FF
SysTick 240 B
24-bit core timer
0xE000_E01C ◀ **CALIB** calibration value
0xE000_E018 ◀ **VAL** current count
0xE000_E014 ◀ **LOAD** reload value
0xE000_E010 ◀ **CTRL** enable, interrupt, clock source
0xE000_E010

0xE000_E00F
Control 16 B
control and ID registers
0xE000_E000

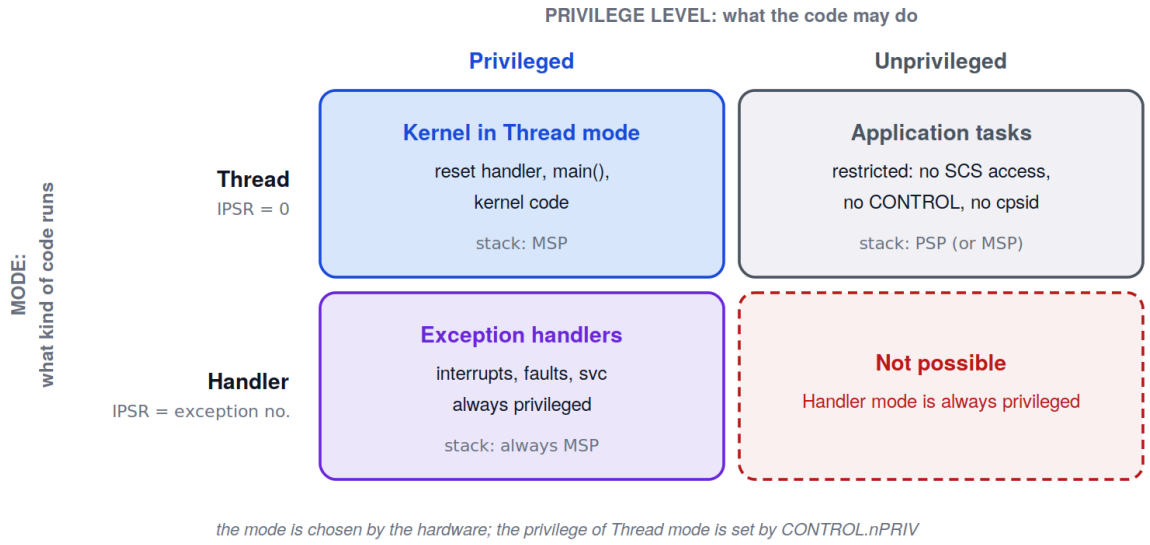
> **IMPORTANT**
>
> Only privileged code may access the SCS. An unprivileged access raises a BusFault, which is what prevents application code from reconfiguring the machine.

4 Execution Modes, Privilege, and Stacks

The Cortex-M3 implements in hardware the **separation** between **kernel mode** and **user mode** found in operating systems. Two independent properties describe the current state:

- the **mode** (what kind of code runs)
- the **privilege level** (what that code may do).

4.1 Thread Mode and Handler Mode



Mode and privilege: *two independent properties of the current state. Three of the four combinations exist; Handler mode is always privileged.*

1. **Thread mode** runs ordinary code: `Reset_Handler`, `main()`, and the functions they call; `IPSR` is 0.
2. **Handler mode** runs exception handlers; `IPSR` holds the exception number. The CPU enters Handler mode automatically when it takes an exception and returns to the previous mode when the handler finishes.

Mode	Privilege	Typical code
Thread	Privileged	`Reset_Handler`, kernel code after reset
Thread	Unprivileged	Application tasks after the kernel drops privilege
Handler	Always privileged	Every exception handler

ONLY THREE COMBINATIONS.

Privilege can be lowered only in Thread mode. Handler mode is always privileged, so an exception always runs privileged and returns to Thread mode with the privilege level it had before.

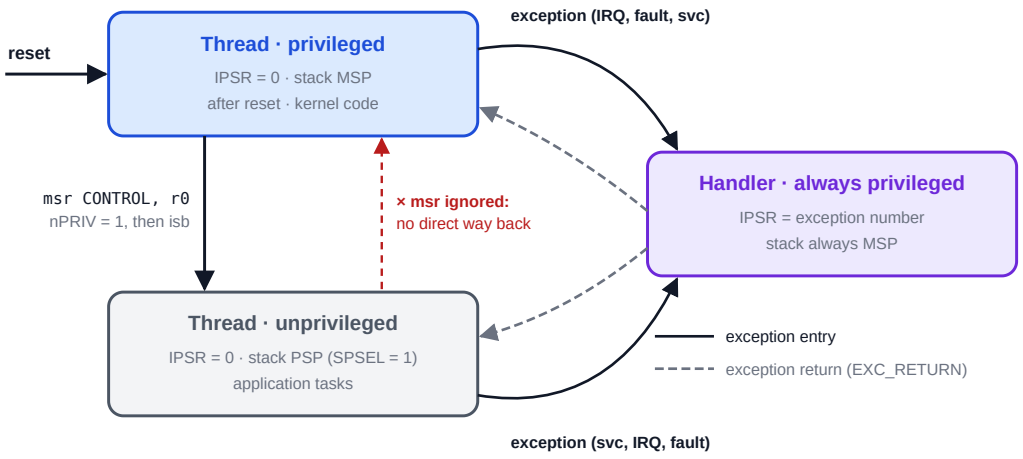


Figure 1.4: *Transitions between privileged Thread mode, unprivileged Thread mode, and Handler mode.*

After reset the CPU is in privileged Thread mode, which is why the boot code can configure the vector table, fault handling, and priorities.

4.2 Dropping Privilege

Privilege is dropped by setting `CONTROL.nPRIV` with the read-modify-write sequence shown in the section on the CONTROL register, followed by `isb`. The change is one-way: an unprivileged `msr control, r0` that tries to clear `nPRIV` is silently ignored, and the only ways back to privileged execution are a reset or an exception, whose handler runs in Handler mode.

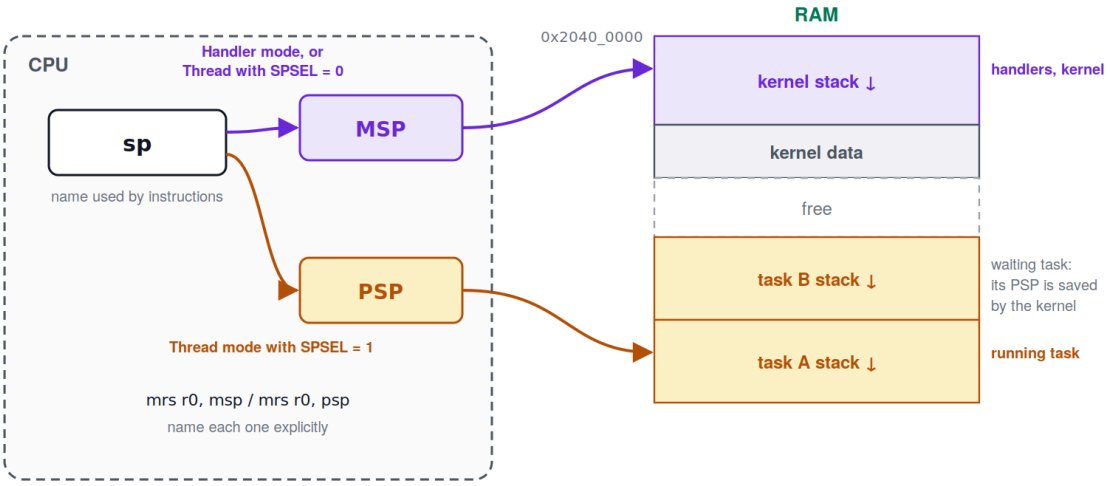
Unprivileged code keeps access to ordinary RAM, Flash, and its own code, but loses the ability to reconfigure or silence the machine:

Attempt by unprivileged code	Result
Access to the SCS (SysTick, NVIC, SCB, MPU)	BusFault
`msr control, r0` to regain privilege	Ignored
`cpsid i` / `cpsie i`	Ignored
Access to a region denied by the MPU	MemManage fault
Semihosting `bkpt 0xab`	Refused by QEMU by default: the fault escalates to HardFault

4.3 Main and Process Stack Pointers

The Cortex-M3 has two physical stack pointers:

1. The **main stack pointer** (`MSP`) is used after reset and always in **Handler mode**; it is the kernel's stack.
2. The **process stack pointer** (`PSP`) is used in **Thread mode** when `CONTROL.SPSEL` is 1; an operating system gives each task its own `PSP` stack. A task therefore typically runs with `CONTROL = 3` : unprivileged and on `PSP` .



an exception always switches to MSP: handlers and system calls never run on a task's stack, so a task that overflows its own stack cannot damage the kernel stack

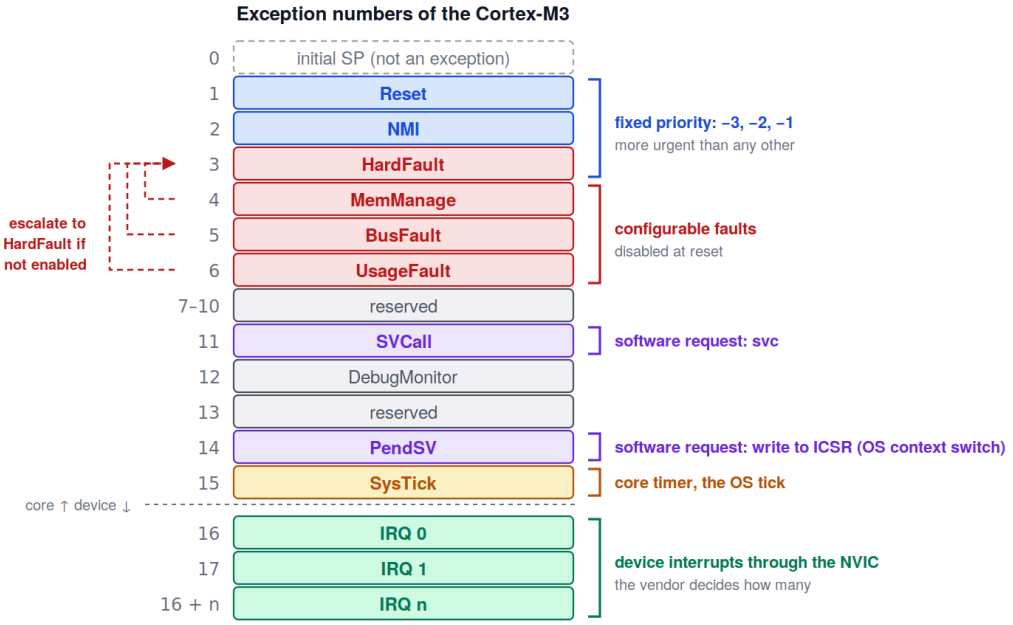
Main and process stack pointers: *sp refers to MSP or PSP depending on the mode and on CONTROL.SPSEL. MSP points to the kernel stack, PSP to the stack of the running task.*

The register name `sp` always refers to the stack pointer currently selected, while `mrs r0, msp` and `mrs r0, psp` name each one explicitly. Because Handler mode always switches to `MSP`, interrupt handlers and system calls never execute on a task's stack: a task that corrupts or overflows its own stack cannot corrupt the stack the kernel runs on.

5 Exception Model

An **exception** is any event that makes the **CPU suspend** the current code and **run a specific handler** in **Handler mode**. An **interrupt** is an **exception** raised by a **hardware device**. Faults, system calls, the system timer, and device interrupts all go through the same mechanism.

5.1 Exception Types and Numbers



Exception numbers: *numbers 1–15 belong to the core and are the same on every Cortex-M3; device interrupts start at 16. Disabled configurable faults escalate to HardFault.*

Number	Exception	Raised by
1	Reset	Power-on, reset pin
2	NMI	Non-maskable interrupt (e.g. watchdog expiry)
3	HardFault	Any fault with nowhere else to go
4	MemManage	Violation of MPU rules
5	BusFault	Access to an address that does not answer or refuses the access
6	UsageFault	Undefined opcode, Thumb bit 0, division by zero (if trapped)
7–10, 13	Reserved	—
11	SVCall	The `SVC` instruction
12	DebugMonitor	Debug events in monitor mode
14	PendSV	A software request (`ICSR.PENDSVSET`)
15	SysTick	The core timer

Number	Exception	Raised by
16 + n	IRQ n	Device interrupt line n, through the NVIC

Numbers 0–15 belong to the core and are identical on every Cortex-M3; device interrupts follow. Number 0 is not an exception: its vector table slot holds the initial stack pointer.

At reset only HardFault is active among the faults: MemManage, BusFault, and UsageFault are disabled, so every fault *escalates* to HardFault until the boot code enables the dedicated handlers.

PendSV deserves a note because of its role in operating systems. It is raised only by software, by writing a bit in a system register, and it is normally given the lowest priority. An OS uses it to request work, typically a context switch, that must run only after every more urgent handler has finished.

5.2 Priorities, Preemption, and Lockup

When several exceptions compete, the more urgent one runs first. On Cortex-M a **lower number means more urgent**. A pending exception preempts the running code only if its priority is *strictly* more urgent; otherwise it stays pending until the current handler returns. Preemption of one handler by another is called *nesting*, which gives the **NVIC (Nested Vectored Interrupt Controller)** its name.

Implementations do not use all eight bits of a priority byte. Real chips implement only the upper 3 or 4 bits, and the lower bits read as 0, so distinct priorities must differ in their top bits (`0x00` , `0x20` , ..., `0xE0` for 3 bits).

> **WARNING**
>
> Lockup If a fault occurs while the CPU already runs at HardFault priority, or the fault cannot be reported because exception entry itself fails (for example, the stack pointer points outside RAM), there is nowhere to escalate. The CPU enters *lockup* and stops executing. QEMU reports it as `Lockup: can't escalate 3 to HardFault (current priority -1)` .

5.3 Vector Table and Reset Sequence

The vector table is an array of 32-bit words where slot N holds the address of the handler of exception N . When exception N occurs, the CPU reads the slot and branches there. Slot 0 is special: it holds the initial value of `MSP` , not a code address. The table is located through the `VTOR` register, which resets to `0x0000_0000` ; at reset it must therefore be at the start of the Code region, which is why the linker places it first in Flash.

When the Cortex-M3 leaves reset, before any software runs, the hardware performs exactly two reads:

1. the word at address `0x0` is loaded into `MSP` ;
2. the word at address `0x4` is loaded into `PC` , and execution starts there in privileged Thread mode.

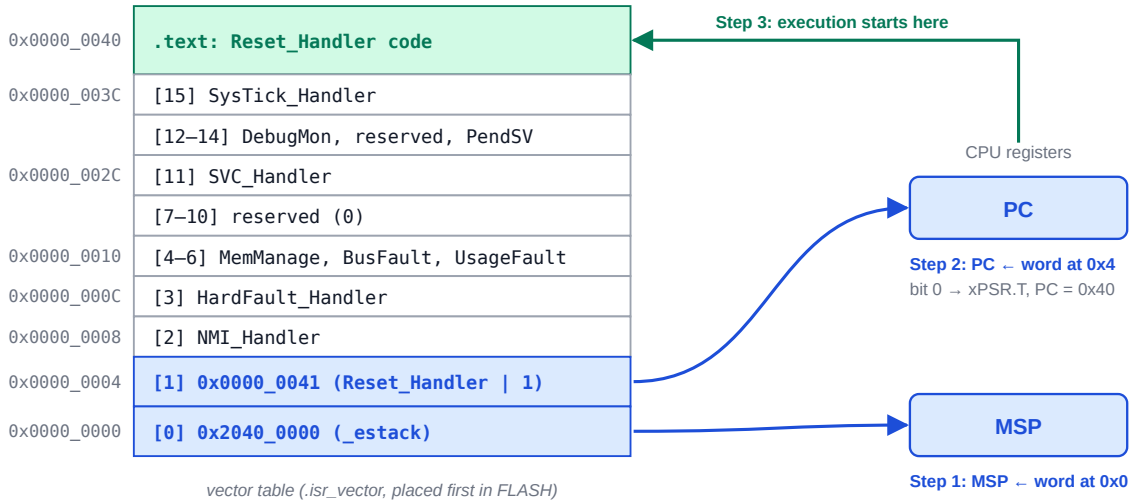


Figure 1.5: *Reset sequence: the CPU loads MSP from address 0x0 and PC from address 0x4, then executes Reset_Handler.*

There is no BIOS, no loader, and no C runtime. Initializing memory, configuring the core, and calling `main()` are the job of the code whose address is stored in slot 1, `Reset_Handler`.

Offset	Slot	Content
`0x00`	0	Initial `MSP` (e.g. `0x2040_0000`, one byte past the end of RAM)
`0x04`	1	`Reset_Handler` address, with bit 0 set
`0x08`	2	`NMI_Handler`
`0x0C` – `0x18`	3–6	HardFault, MemManage, BusFault, UsageFault handlers
`0x2C`	11	`SVC_Handler`
`0x38`, `0x3C`	14, 15	`PendSV_Handler`, `SysTick_Handler`
`0x40` + 4n	16 + n	Handler of IRQ n

Every handler address is stored with the Thumb bit set. Reserved slots hold 0, and every other slot must point to valid code even if the exception is never expected; startup code usually fills unused slots with a default handler that loops forever, so that an unexpected exception stops at a recognizable address.

> **TIP**
>
> From IRQ number to vector slot IRQ n is exception 16 + n, and its handler address is at offset 4 × (16 + n). IRQ 1 is exception 17, at offset `0x44`.

5.4 Exception Entry and the Stacked Frame

Before the first instruction of a handler, the hardware:

1. pushes 8 words, the *stacked frame*, onto the stack in use when the exception occurred (`MSP` or `PSP`);

2. writes a special `EXC_RETURN` value into `lr`;
3. reads the handler address from the vector table and branches to it in Handler mode, on `MSP`.

Offset from new `sp`	Stacked register
`+0x00`	`r0`
`+0x04`	`r1`
`+0x08`	`r2`
`+0x0C`	`r3`
`+0x10`	`r12`
`+0x14`	`lr` of the interrupted code
`+0x18`	`PC`: where the interrupted code resumes
`+0x1C`	`xPSR`

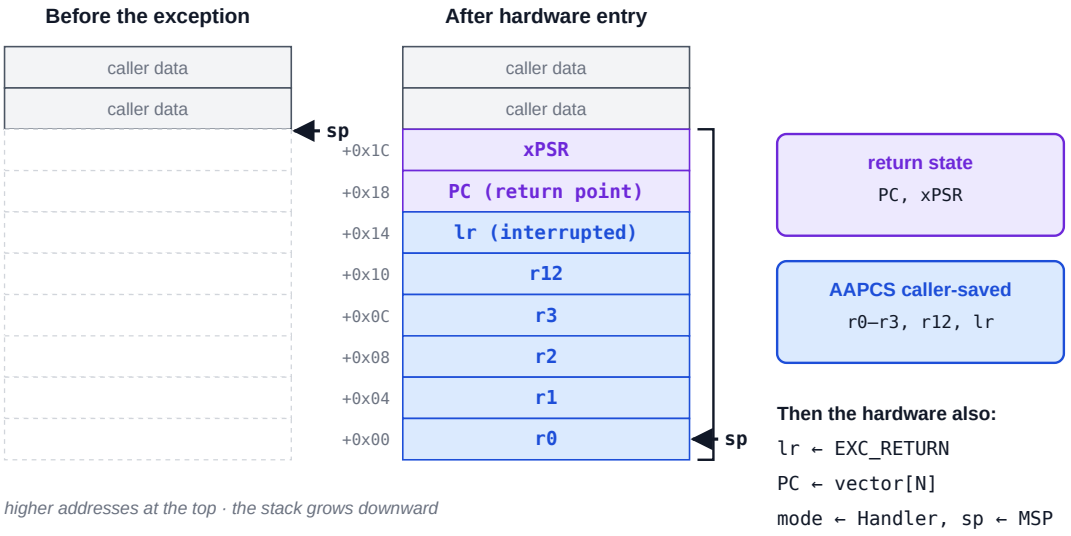


Figure 1.6: *Stack before and after exception entry, with the 8-word stacked frame pushed by hardware.*

The hardware saves exactly the AAPCS caller-saved registers plus `PC` and `xPSR`, and any compiled C function already preserves the callee-saved `r4`–`r11`. Together, these two facts mean that an exception handler can be an ordinary C function, with no special prologue or epilogue. When `CCR.STKALIGN` is set (its reset value on the Cortex-M3), the hardware also aligns `sp` to 8 bytes on entry, as the AAPCS requires, and records the adjustment in bit 9 of the stacked `xPSR`. Entry takes about 12 cycles.

5.5 Exception Return through EXC_RETURN

A handler ends with an ordinary return instruction (`bx lr` or `pop {..., pc}`), yet the CPU must restore the saved state rather than jump to a return address. While a handler runs, `lr` holds a value of the form `0xFFFFFFxx`, where no code can exist. Loading such a value into `PC` is recognized as an *exception return*: the

CPU pops the 8-word frame from the indicated stack, restores the mode, and resumes at the stacked `PC`.

`EXC_RETURN`	Return to	Frame popped from
`0xFFFFFFF1`	Handler mode (a preempted handler)	`MSP`
`0xFFFFFFF9`	Thread mode	`MSP`
`0xFFFFFFFD`	Thread mode	`PSP`

Bits 31–4 are all ones and mark the value as `EXC_RETURN`. Bit 3 selects the return mode (1 = Thread, 0 = Handler) and bit 2 the stack (1 = `PSP`, 0 = `MSP`). On entry, bit 2 therefore tells the handler which stack holds the frame just pushed, which a system-call handler needs in order to read the caller's arguments.

5.6 Tail-Chaining and Late Arrival

When exceptions occur close together, the core avoids saving and restoring the same state repeatedly.

Optimization	Situation	Behaviour
Tail-chaining	A handler ends while another exception is pending	No pop and re-push of the frame: direct jump to the next handler (about 6 cycles instead of about 24)
Late arrival	A more urgent exception arrives while the frame of a less urgent one is being pushed	The frame is reused; the more urgent handler runs first, then tail-chains to the other

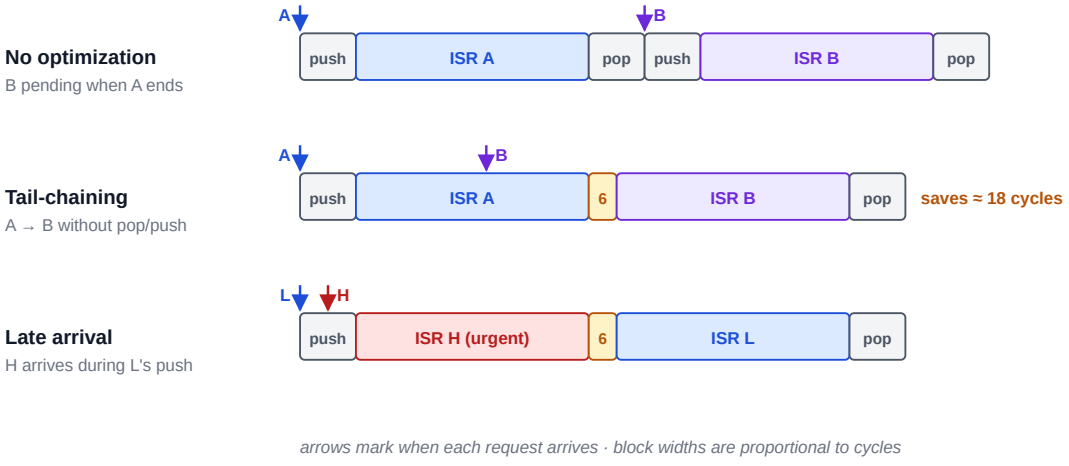


Figure 1.7: *Timelines of two exceptions without optimization, with tail-chaining, and with late arrival.*

5.7 SVC: A Controlled Entry into Privileged Code

```
void SVC_Handler( void )
{
    /* privileged service code */
}
```

```
/* in unprivileged Thread mode */
__asm__ volatile ( "svc #0" : : : "memory" );
```

`svc #n` (supervisor call) is one of the few things unprivileged code can always do. It raises SVCall (exception 11) synchronously, so the handler in slot 11 runs in Handler mode and therefore privileged. When the handler returns, execution resumes at the instruction after `svc`, in Thread mode and still unprivileged. Inside the handler `CONTROL.nPRIV` still reads 1: the handler is privileged because of the mode, not because the bit changed.

Privilege is borrowed only for the duration of the handler, and only code placed in the handler runs with it: the caller can request a service but cannot choose what executes. The immediate `#n` is encoded in the instruction and, together with arguments in `r0`–`r3`, can select the requested service. This is the hardware foundation of every system call: restricted code cannot touch the machine directly and must go through a single, controlled entry point.

> ⚠ **MISSING HANDLER**
>
> If `SVC_Handler` is not defined, `svc` reaches the default handler and the program hangs silently. Unlike a fault handler, `SVC_Handler` returns normally, because `svc` is a deliberate request, not an error.

6 Configuring the Core: SCB, NVIC, and SysTick

The behaviour of the exception model is configured through memory-mapped registers in the System Control Space. All of them are accessible only from privileged code.

6.1 System Control Block Registers

Register	Address	Role
`CPUID`	`0xE000_ED00`	Core identification (`0x410FC231` : ARM, Cortex-M3)
`ICSR`	`0xE000_ED04`	Pending state of system exceptions; `PENDSVSET` (bit 28) raises PendSV
`VTOR`	`0xE000_ED08`	Address of the vector table
`CCR`	`0xE000_ED14`	Which conditions are treated as faults
`SHPR1` – `SHPR3`	`0xE000_ED18` – `0xE000_ED20`	Priorities of system exceptions
`SHCSR`	`0xE000_ED24`	Enable and status of system handlers
`CFSR`	`0xE000_ED28`	Cause of MemManage, BusFault, UsageFault
`HFSR`	`0xE000_ED2C`	Cause of HardFault
`MMFAR`	`0xE000_ED34`	Address that caused a MemManage fault
`BFAR`	`0xE000_ED38`	Address that caused a BusFault

`VTOR`. On an exception the CPU reads the handler address from `VTOR + 4 × N`. Bits 31–7 hold the table address, which must be aligned to at least 128 bytes (more when many IRQs exist). Writing `VTOR` is redundant when the table is already at 0, but it makes the boot code correct wherever the image is placed: a bootloader at address 0 can start an application stored higher in Flash, which then points `VTOR` at its own table, or a program can copy its table into RAM to replace handlers at run time.

`SHCSR`. Bits 16 (`MEMFAULTENA`), 17 (`BUSFAULTENA`), and 18 (`USGFAULTENA`) enable the dedicated fault handlers. At reset they are 0, so every fault escalates to HardFault and a single handler cannot tell what went wrong. The other bits report whether handlers are active or pending, so the register is updated with read-modify-write (`|= 0x00070000`), never overwritten.

`CCR`. The configuration bits relevant here are listed below.

Bit	Name	Effect when set
3	`UNALIGN_TRP`	Unaligned word and halfword accesses raise a UsageFault
4	`DIV_0_TRP`	Integer division by zero raises a UsageFault
9	`STKALIGN`	8-byte stack alignment on exception entry (1 at reset)

Without `DIV_0_TRP` , `sdiv` and `udiv` silently return 0 when the divisor is 0 and the program continues with a wrong value; setting the bit turns this silent bug into a fault. Unaligned accesses are legal on the Cortex-M3, only

slower, and trapping them can break correct compiler-generated code, so `UNALIGN_TRP` is usually left off. Since `STKALIGN` is already 1, enabling the division trap requires read-modify-write, giving `0x210`.

`SHPR1` – `SHPR3`. Each system exception has one priority byte, so the three registers can be viewed as a 12-byte array starting at `0xE000_ED18`, indexed by exception number minus 4. Each byte is written with a byte store and needs no read-modify-write.

Register	Byte 0	Byte 1	Byte 2	Byte 3
`SHPR1`	MemManage	BusFault	UsageFault	—
`SHPR2`	—	—	—	SVCall
`SHPR3`	—	—	PendSV	SysTick

A typical policy makes faults most urgent (`0x00`), system calls intermediate (`0x80`), and PendSV and SysTick least urgent (`0xE0`). Reading the words as little-endian, this gives `SHPR2 = 0x80000000` and `SHPR3 = 0xE0E00000`.

6.2 Fault Status Registers

```
void UsageFault_Handler( void )
{
    kprintf( "UsageFault: CFSR=0x%x\n", SCB_CFSR );
    for ( ;; ) { }
}

int main( void )
{
    volatile uint32_t numerator = 42;
    volatile uint32_t zero = 0;
    volatile uint32_t result = numerator / zero;
    (void)result;
    for ( ;; ) { }
}
```

`kprintf` is the kernel's formatted print function, built on the UART driver described in the chapter on devices, interrupts, and system calls; before that driver exists, the same experiment can be followed in gdb by stopping in `UsageFault_Handler` and reading `CFSR`.

The divisor is `volatile`, so the compiler cannot evaluate the division at build time and must emit a real `udiv` instruction. With `CCR.DIV_0_TRP` and `SHCSR.USGFAULTENA` set, the instruction raises a UsageFault, the dedicated handler runs, and it prints `CFSR=0x2000000`: bit 25, `DIVBYZERO`. The handler then loops forever, because a fault is a bug with no sensible way to continue. Removing each configuration step isolates its role:

Configuration	Observed behaviour
`SHCSR` bits and `DIV_0_TRP` set	`UsageFault_Handler` runs and reports the cause

Configuration	Observed behaviour
`SHCSR` bits not set	The fault escalates to HardFault; its kind is lost
`DIV_0_TRP` not set	No fault: the division returns 0 and `main` continues

`CFSR` is the concatenation of three status registers, one per configurable fault. Its most useful bits are:

Bits	Sub-register	Main flags
7–0	MMFSR (MemManage)	`IACCVIOL` (0), `DACCVIOL` (1), `MUNSTKERR` (3), `MSTKERR` (4), `MMARVALID` (7)
15–8	BFSR (BusFault)	`PRECISERR` (9), `IMPRECISERR` (10), `BFARVALID` (15)
31–16	UFSR (UsageFault)	`UNDEFINSTR` (16), `INVSTATE` (17), `UNALIGNED` (24), `DIVBYZERO` (25)

`HFSR` bit 30 (`FORCED`) indicates that the HardFault is an escalated configurable fault; the original cause is then found in `CFSR` . The bits are sticky and are cleared by writing 1 to them.

6.3 NVIC: Device Interrupts

```
NVIC->IPR[UART0TX_IRQn] = 0xC0;                              /* priority of IRQ 1 */
NVIC->ISER[UART0TX_IRQn / 32] = 1UL << ( UART0TX_IRQn % 32 ); /* enable IRQ 1 */
```

Device interrupt requests do not reach the CPU directly: they pass through the NVIC, which numbers them, enables them, and assigns their priority. The first line writes the priority byte of IRQ 1 with a byte store. The second sets bit `n % 32` of word `n / 32` in the enable register; since set and clear registers ignore 0 bits, a plain assignment enables one line without disturbing the others.

Register	Base address	Bit or byte for IRQ n	Write semantics
`ISER`	`0xE000_E100`	Bit `n % 32` of word `n / 32`	1 = enable
`ICER`	`0xE000_E180`	Same	1 = disable
`ISPR`	`0xE000_E200`	Same	1 = set pending
`ICPR`	`0xE000_E280`	Same	1 = clear pending
`IPR`	`0xE000_E400`	Byte n	Priority; lower = more urgent

A device interrupt runs its handler only if all of the following hold:

1. the device is configured to raise it (an interrupt-enable bit in the device's own control register);
2. the NVIC line is enabled in `ISER` ;
3. its priority is more urgent than that of the running code, and `PRIMASK` is 0.

NVIC lines driven by device flags are level-sensitive: as long as the device keeps its interrupt flag set, the request stays active. A handler must therefore *acknowledge* the interrupt at the device, typically by writing 1 to clear the flag, otherwise the handler is re-entered forever as soon as it returns.

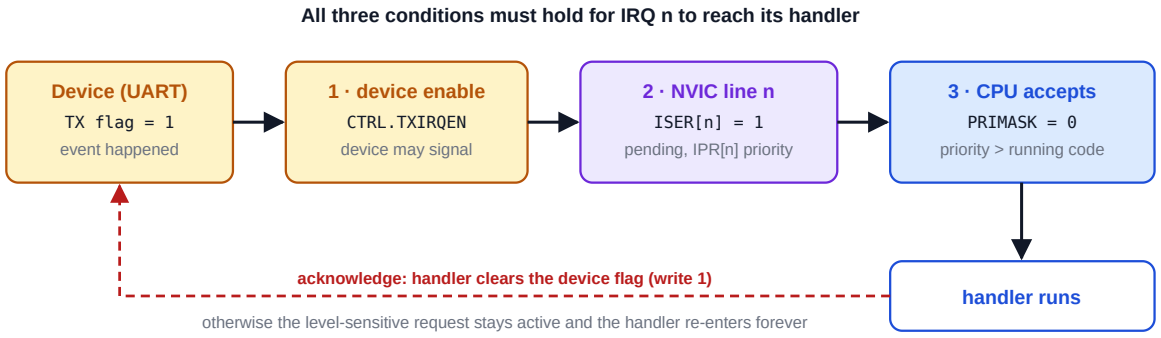


Figure 1.8: *Path of a device interrupt through the device enable, the NVIC, and the CPU masking and priority check, with acknowledgement at the device.*

6.4 SysTick: The Core Timer

SysTick is a 24-bit down-counter present in every Cortex-M core. It counts from the reload value to 0, reloads, and can raise exception 15 each time it reaches 0. An operating system uses this periodic exception, the *tick*, as its time base.

Register	Address	Content
`CTRL`	`0xE000_E010`	`ENABLE` (0), `TICKINT` (1), `CLKSOURCE` (2), `COUNTFLAG` (16, cleared on read)
`LOAD`	`0xE000_E014`	24-bit reload value (max `0xFFFFFF`)
`VAL`	`0xE000_E018`	Current count; any write clears it
`CALIB`	`0xE000_E01C`	Read-only; `TENMS` = reload value for 10 ms on the reference clock

`CLKSOURCE` selects between the processor clock (1) and an implementation-defined reference clock (0). On `mps2-an385` the reference clock runs at 1 MHz (`CALIB.TENMS = 9999`) and the processor clock at 25 MHz. `CALIB` serves portable code that does not know the processor frequency; when the frequency is known, the reload value is computed directly. Writing `CTRL = 7` starts the counter on the processor clock with the exception enabled.

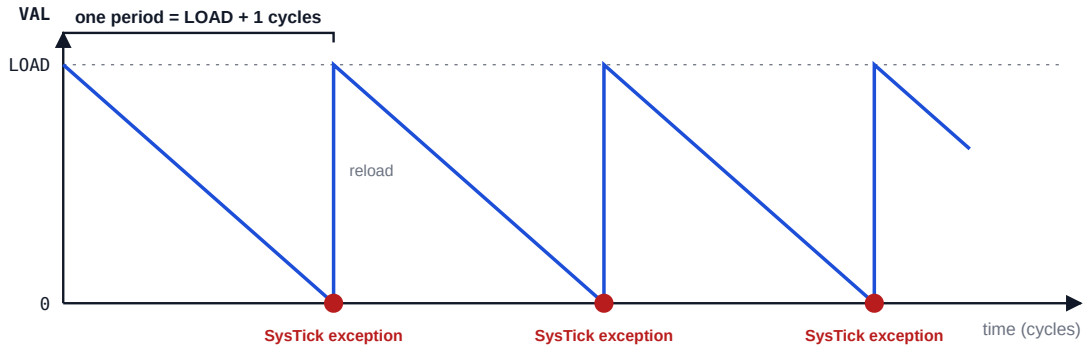


Figure 1.9: *SysTick counter value over time: a sawtooth from LOAD to 0 with an exception at every zero.*

Since one period lasts `LOAD + 1` cycles, the reload value for a tick rate r at clock frequency f is

$$\mathrm{LOAD} = \frac{f}{r} - 1$$

At 25 MHz and 100 ticks per second, `LOAD = 249 999` (`0x3D08F`), which fits in 24 bits; the longest possible period is 2^{24} cycles, about 0.67 s. A faster tick gives finer time resolution at the cost of more exceptions per second.

SysTick is a core exception, not a device interrupt, and this changes how it is configured and handled:

Aspect	SysTick	Device interrupt
Priority set in	`SHPR3`	NVIC `IPR`
Enabled by	`CTRL.TICKINT`	Device enable bit and NVIC `ISER`
Acknowledge in the handler	Not needed: pending state clears on entry	Required, otherwise the handler is re-entered

7 Memory Protection Unit

The Cortex-M3 has no MMU. What it offers instead is an optional **Memory Protection Unit** (MPU), present on `mps2-an385` and disabled after reset.

7.1 MPU Versus MMU

An MMU *translates* every virtual address into a physical one through page tables, and checks permissions as part of the translation. An MPU *only checks*: on every load, store, and instruction fetch, it compares the address with a short list of regions. If the access is allowed, it proceeds at the same physical address; if not, it does not happen at all (a store writes nothing, a load reads nothing) and the CPU raises a MemManage fault. Every task sees real physical addresses, so the linker must place each task at its own address.

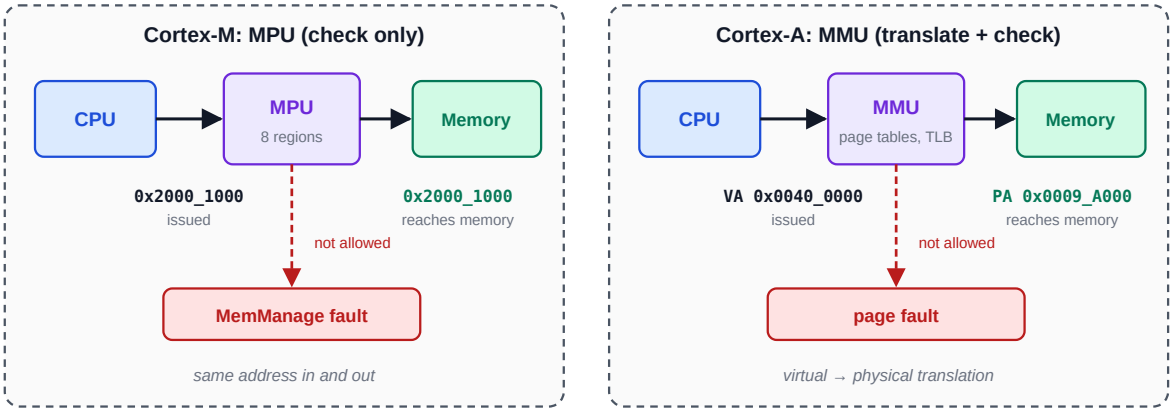


Figure 1.10: *Comparison of an MPU, which only checks addresses, and an MMU, which translates virtual to physical addresses.*

Microcontrollers adopt the MPU for reasons that follow directly from embedded constraints. An MPU is a handful of region registers, while an MMU needs page tables in RAM, a hardware table walker, and a TLB, which cost silicon area and energy. An MPU check takes the same time on every access, whereas a TLB miss makes some accesses slower by an amount that depends on the TLB contents, which harms timing predictability. Finally, with no disk and a small fixed set of tasks, virtual memory would bring no benefit.

7.2 Regions and Permissions

The Cortex-M3 MPU has 8 regions, numbered 0 to 7 (the count is read from `MPU_TYPE` bits 15–8). Each region has a base address, a size, permissions, and an enable bit, subject to two rules:

- the size is a power of two, from 32 bytes to 4 GB;
- the base address is a multiple of the size, i.e. the region is aligned to its own size.

A 4 KB region can start at `0x2000_1000` but not at `0x2000_0400`. A task that needs 5 KB receives an 8 KB region, the next power of two.

The **AP** (access permission) field encodes two rights at once, one for privileged and one for unprivileged code:

AP	Privileged	Unprivileged
`000`	No access	No access
`001`	Read/write	No access
`010`	Read/write	Read-only
`011`	Read/write	Read/write
`110`	Read-only	Read-only

The **XN** (execute never) bit forbids instruction fetches from a region while still allowing reads and writes if AP permits them. Marking data and stack regions XN prevents code injection: a task cannot execute machine code it has written into a buffer.

The **PRIVDEFENA** bit of `MPU_CTRL` decides what happens to privileged accesses that hit no region. When it is 1, they use the default memory map as if the MPU were off, so the kernel keeps access to all memory and peripherals and only unprivileged code is restricted. When it is 0, even privileged code may access only what the regions allow. An unprivileged access that hits no enabled region is always refused.

7.3 MPU Registers and Region Encoding

Register	Address	Fields
`MPU_TYPE`	`0xE000_ED90`	Bits 15–8: number of regions
`MPU_CTRL`	`0xE000_ED94`	`ENABLE` (0), `PRIVDEFENA` (2)
`MPU_RNR`	`0xE000_ED98`	Region selected by `RBAR` and `RASR`
`MPU_RBAR`	`0xE000_ED9C`	Base address (31–5), `VALID` (4), `REGION` (3–0)
`MPU_RASR`	`0xE000_EDA0`	`XN` (28), `AP` (26–24), `S` / `C` / `B` (18–16), `SIZE` (5–1), `ENABLE` (0)

A single `RBAR` / `RASR` pair serves all eight regions; `RNR` selects which one it refers to. When `RBAR` is written with its `VALID` bit set, the region number is taken from its own `REGION` field, so one write replaces the pair of writes to `RNR` and `RBAR`. The low 5 bits of a base address are always 0, because the base is aligned to at least 32 bytes, which leaves room for `VALID` and `REGION` in the same word.

The `SIZE` field holds $\log_2(\text{size}) - 1$, so a region spans $2^{\text{SIZE}+1}$ bytes; the offset of 1 lets 5 bits reach 4 GB with `SIZE = 31`. The `S`, `C`, and `B` bits describe the memory type for caches and multi-core systems and have no functional effect on a core without cache.

```
#define MPU_CTRL (*(volatile uint32_t *)0xE000ED94)
#define MPU_RBAR (*(volatile uint32_t *)0xE000ED9C)
#define MPU_RASR (*(volatile uint32_t *)0xE000EDA0)

void mpu_setup( void )
{
    MPU_RBAR = 0x00000000u | 0x10u | 0u;  /* region 0: Flash, VALID */
    MPU_RASR = 0x0602002Bu;               /* AP=110, C, 4 MB, enabled */
```

```
    MPU_RBAR = 0x20001000u | 0x10u | 1u;  /* region 1: task partition, VALID */
    MPU_RASR = 0x13060017u;               /* XN, AP=011, S, C, 4 KB, enabled */
    MPU_CTRL = ( 1u << 2 ) | 1u;          /* PRIVDEFENA | ENABLE */
    __asm volatile ( "dsb\n\tisb" : : : "memory" );
}
```

The function configures a typical two-region layout for running one unprivileged task. Region 0 covers the 4 MB of Flash as read-only and executable for everyone, so the task can run its code and read constants. Region 1 covers the 4 KB partition of the running task as read/write and non-executable. Everything else (other tasks' memory, kernel data, kernel stack, peripherals) has no region and is therefore forbidden to unprivileged code, while `PRIVDEFENA` leaves the kernel unrestricted. A kernel that switches tasks only needs to rewrite region 1. The final `dsb` waits for all register writes to complete and `isb` ensures that the following instructions are checked against the new regions.

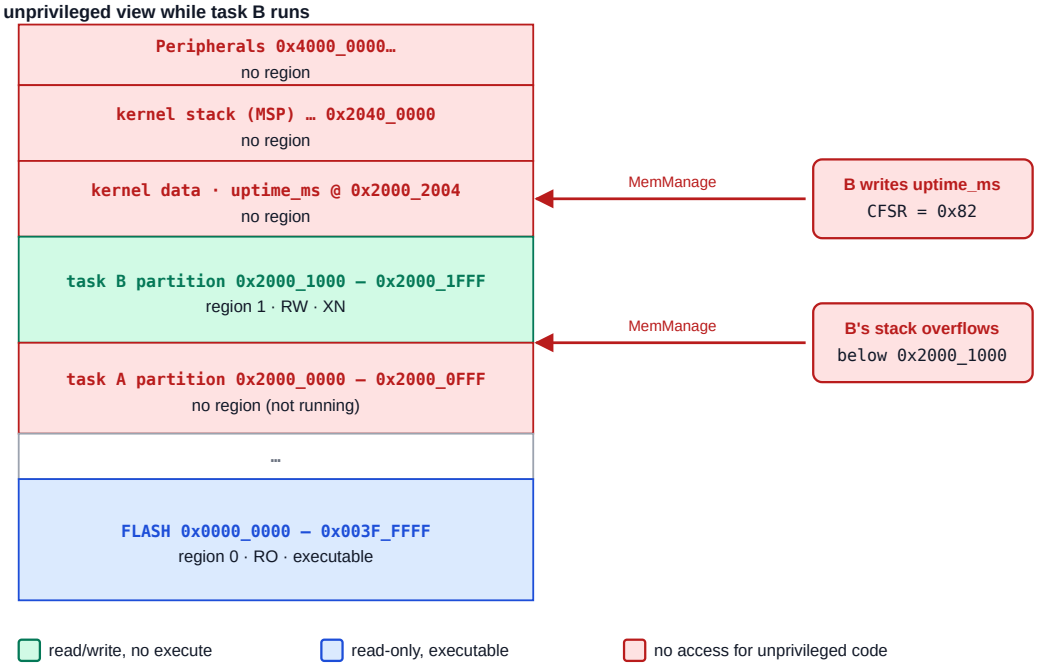
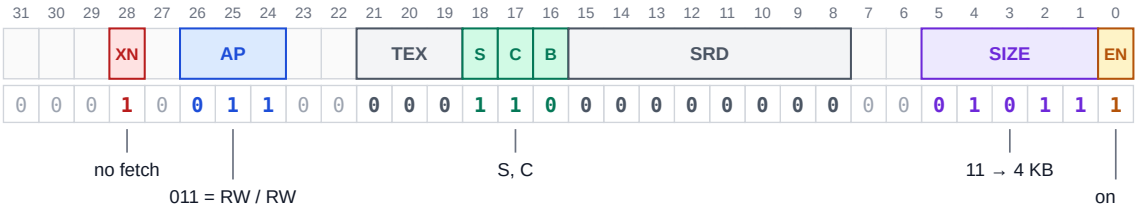


Figure 1.11: *Unprivileged view of memory with the MPU enabled while task B runs: Flash region 0, task B partition region 1, everything else forbidden.*

The two `RASR` values decompose as follows:

Field	Region 0 (Flash)	Region 1 (partition)
`XN` (bit 28)	0	1 → `0x10000000`
`AP` (bits 26–24)	`110` → `0x06000000`	`011` → `0x03000000`
`S`, `C` (bits 18, 17)	`C` → `0x00020000`	`S`, `C` → `0x00060000`
`SIZE` (bits 5–1)	21 (4 MB = 2^{22}) → `0x2A`	11 (4 KB = 2^{12}) → `0x16`
`ENABLE` (bit 0)	`0x01`	`0x01`
Total	`0x0602002B`	`0x13060017`



RASR = 0x1306_0017: region 1 (task partition) · TEX and SRD left at 0

Figure 1.12: *Field layout of MPU_RASR with the value 0x13060017 decoded bit by bit.*

7.4 MemManage Faults

A refused access raises MemManage (exception 4), which reaches its own handler only if `SHCSR.MEMFAULTENA` is set; otherwise it escalates to HardFault. The handler runs privileged and on `MSP`, and it finds the cause in the low byte of `CFSR`:

Bit	Name	Set when
0	`IACCVIOL`	An instruction fetch was refused (e.g. from an XN region)
1	`DACCVIOL`	A data load or store was refused
3	`MUNSTKERR`	Popping the frame on exception return was refused
4	`MSTKERR`	Pushing the frame on exception entry was refused
7	`MMARVALID`	`MMFAR` holds the refused address

Several bits can be set at once. A value of `0x82`, for example, means `MMARVALID` plus `DACCVIOL`: a data access was refused and its address is in `MMFAR`. Looking that address up in the symbol table of the image (with `nm`) names the variable the faulty code tried to touch. A task that overflows its stack below the start of its partition produces the same kind of fault, which makes the MPU an effective stack-overflow detector.

> **WARNING**
>
> Read `MMFAR` only when `MMARVALID` is set For instruction-fetch and stacking faults the CPU does not update `MMFAR`, which may then hold a stale, meaningless value.

1 Cross-Compilation Toolchain

1.1 Host, Target, and Emulator

Role	Meaning
Host	Computer running the editor, `make`, and the toolchain (x86-64 or Apple silicon)
Target	ARM Cortex-M3 machine for which the ELF is built
Emulator	QEMU model that executes target instructions on the host

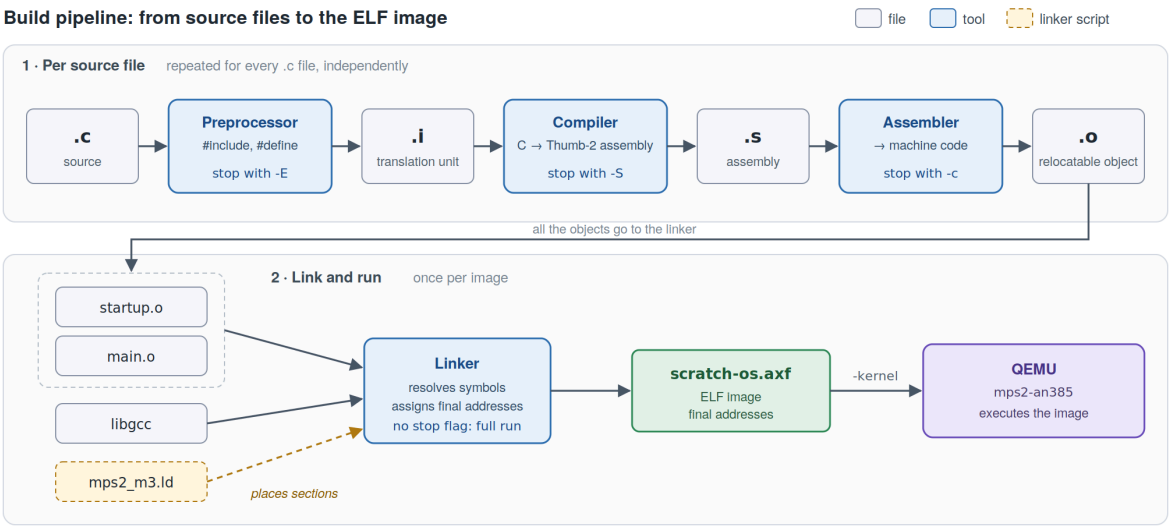
A native `gcc` emits code for the machine it runs on; it cannot produce Cortex-M3 firmware just because the source language is C. A **cross toolchain** (`arm-none-eabi-*`) runs on the host but emits code for the target instruction set architecture (ISA).

```
gcc -dumpmachine              # host triple, e.g. x86_64-linux-gnu
arm-none-eabi-gcc -dumpmachine  # target triple: arm-none-eabi
```

The triple `arm-none-eabi` means: ARM architecture, **no** operating system on the target, standard ARM **E**mbedded **ABI**.

- Consequences of "no OS":
  - No `_start` from a C library unless the application provides one.
  - No dynamic loader, and no process to return to when `main()` finishes.
  - No guaranteed `printf` console, no `malloc` backed by an OS.
- QEMU is not the target CPU, but it provides a reproducible board model before physical hardware is used.

1.2 Build Pipeline Stages



Stage	What it does	stop flag	ext
Preprocessing	Expands `#include`, macros, conditional compilation; pure text substitution	`-E`	`.i`

Stage	What it does	stop flag	ext
Compilation	Checks syntax and types, **optimises**, emits **ARM assembly** for the selected CPU and ABI	`-S`	`.s`
Assembly	Encodes assembly into a **relocatable object file**	`-c`	`.o`
Linking	Combines objects, resolves symbols, assigns final addresses	N/A	`.axf` / `.elf`

`arm-none-eabi-gcc` is a *driver*: a single invocation runs all four stages in sequence and shows **only the final result**, deleting the intermediate files. A stop flag interrupts the pipeline after a given stage and leaves its output on disk, so that the intermediate result can be inspected:

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -ffreestanding <stop flag> src/main.c -o build/main.<ext>
```

1.3 Target Compiler and Linker Flags

```
CFLAGS := -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding -std=gnu17
CFLAGS += -Wall -Wextra -O0 -g3

LDFLAGS := -T scripts/mps2_m3.ld
LDFLAGS += -Xlinker -Map=build/output.map
LDFLAGS += -nostdlib -lgcc
```

Flag	Meaning
`-mthumb -mcpu=cortex-m3`	Selects the Thumb instruction subset implemented by the Cortex-M3
`-nostartfiles`	Excludes the toolchain's default C startup objects, so reset handling is done by the project's own startup code
`-ffreestanding`	Compiles for a freestanding (bare-metal) C environment, without hosted-library assumptions
`-std=gnu17`	C17 with GNU extensions
`-Wall -Wextra`	Warning policy
`-O0 -g3`	No optimisation and full debug information: a debuggable, source-corresponding image
`-T scripts/mps2_m3.ld`	Uses the given linker script to describe memory and section placement
`-Xlinker -Map=build/output.map`	Makes the linker emit a map file of every placement and symbol address
`-nostdlib -lgcc`	Links no standard library, only `libgcc` for compiler support routines

DIAGNOSING A BUILD FAILURE

The full command line is the exact contract that produced the failure: copy it before changing anything.

1.4 Separate Compilation and Linking

The compiler translates one source file at a time and never looks at the others. When `startup.c` calls `main()`, the compiler does not know where `main` is, or whether it exists: it emits the call and leaves its target address blank, to be filled in later. Connecting the files is the job of the linker.

1.4.1 Object Files and Symbols

The result of compiling is a **relocatable object file** (`.o`): machine code whose addresses are not yet final, plus the *symbols* (names of functions and global variables) that the file defines or only references. `arm-none-eabi-nm` lists the symbols of an object, one per line: address, a letter for the kind of symbol, and name. The meaning of each letter is listed in the chapter Inspecting the Image.

`STARTUP.O` REFERENCES `MAIN`, `MAIN.O` DEFINES IT

The two source files are compiled separately with `-c` .

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -ffreestanding -c boot/startup.c -o
build/boot/startup.o
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -ffreestanding -c src/main.c -o
build/src/main.o
```

Then their symbols are listed:

```
arm-none-eabi-nm build/boot/startup.o build/src/main.o
```

Output:

```
build/boot/startup.o:
00000008 T Reset_Handler
         U _estack
         U main
...
build/src/main.o:
00000018 T main
00000000 t semihost_write0
```

`startup.o` defines `Reset_Handler` but only references `main` and `_estack` (`U`); `main.o` defines `main` (`T`). The addresses are offsets inside each object, not final addresses: neither object is a runnable program on its own.

1.4.2 The Linker: Resolving Symbols

The **linker** combines all the objects into one image in three steps:

1. it concatenates the sections with the same name from every object, which gives each function and variable its final address;
2. it resolves every undefined symbol against a definition, here `main` against `main.o` and `_estack` against the linker script, and writes the final address into every instruction that referenced it;
3. it checks that every referenced symbol has exactly one definition.

> **`MAIN` AFTER THE LINK**
>
> The two objects are linked into the final image:
>
> ```
> arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostdlib \
>     -T scripts/mps2_m3.ld -Xlinker -Map=build/output.map \
>     build/boot/startup.o build/src/main.o -lgcc -o build/scratch-os.axf
> ```
>
> Then the symbols of the image are listed, sorted by address:
>
> ```
> arm-none-eabi-nm -n build/scratch-os.axf
> ```
>
> Output:
>
> ```
> 00000048 T Reset_Handler
> 000000a4 t semihost_write0
> 000000bc T main
> ...
> 20400000 A _estack
> ```
>
> The `U` lines have disappeared: `main` is now defined at its final address in Flash, and `_estack` is defined by the linker script (`A` , an absolute value that belongs to no section). Every address is the real one: `0x48` , `0xA4` , and `0xBC` match the Flash layout shown by the map file, which also records which object supplied each symbol.

Where the sections are placed is not decided by the linker on its own: the **linker script** (`.ld`) gives the addresses of `FLASH` and `RAM` and the order of the sections. On a desktop a default script is used without the developer seeing it; on **bare metal it must be supplied**, because only the developer knows the chip's memory map.

1.4.2.1 COMMON LINK ERRORS

The check of step 3 produces the **two most common link errors**:

Error	Cause
`undefined reference to 'main'`	A `U` that no object defines: a missing `.o` on the command line or a misspelled name

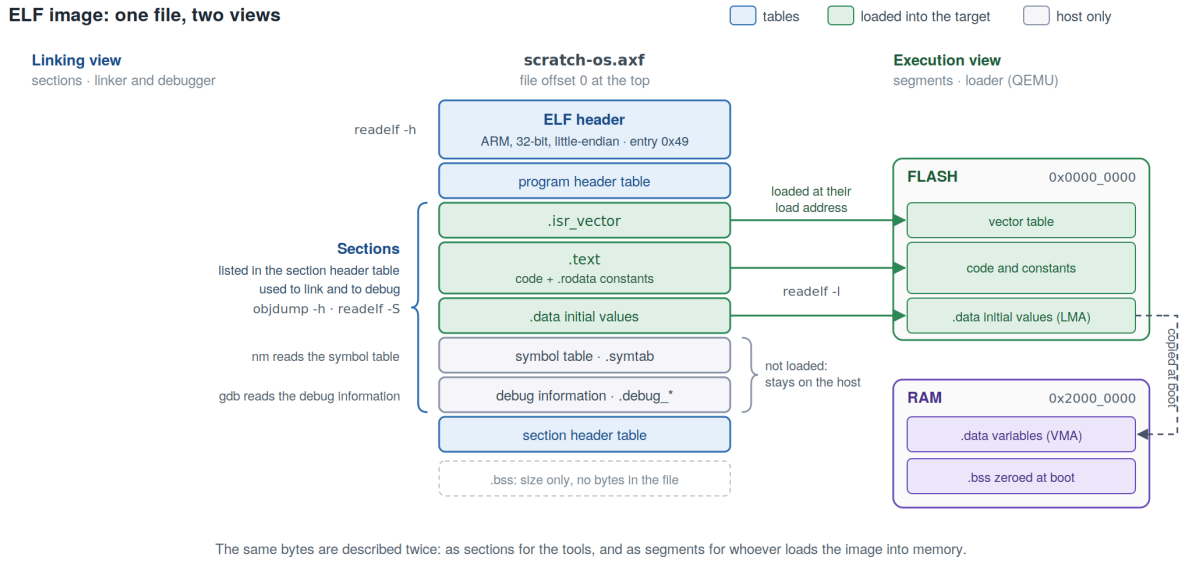
Error	Cause
`multiple definition of 'main'`	Two objects define the same symbol

2 The ELF Format

2.1 Structure of an ELF File

ELF (Executable and Linkable Format) is the format of the final image `build/scratch-os.axf`; the `.axf` suffix conventionally denotes an ARM executable with debug information. The image is a 32-bit, little-endian ARM ELF file that offers several views of the same bytes:

Part	Content	Used by
ELF header	At offset 0: file type, architecture, entry point, location of the other tables	Every tool that opens the file
Sections	`.text`, `.rodata`, `.data`, `.bss`, symbol table, debug information	Linker and debugger
Program headers (segments)	Which chunks of the file go to which memory addresses at run time	Loader (here QEMU)
Symbols	Names of functions and variables with their final addresses	Debugger, developer



The same bytes are described twice: as sections for the tools, and as segments for whoever loads the image into memory.

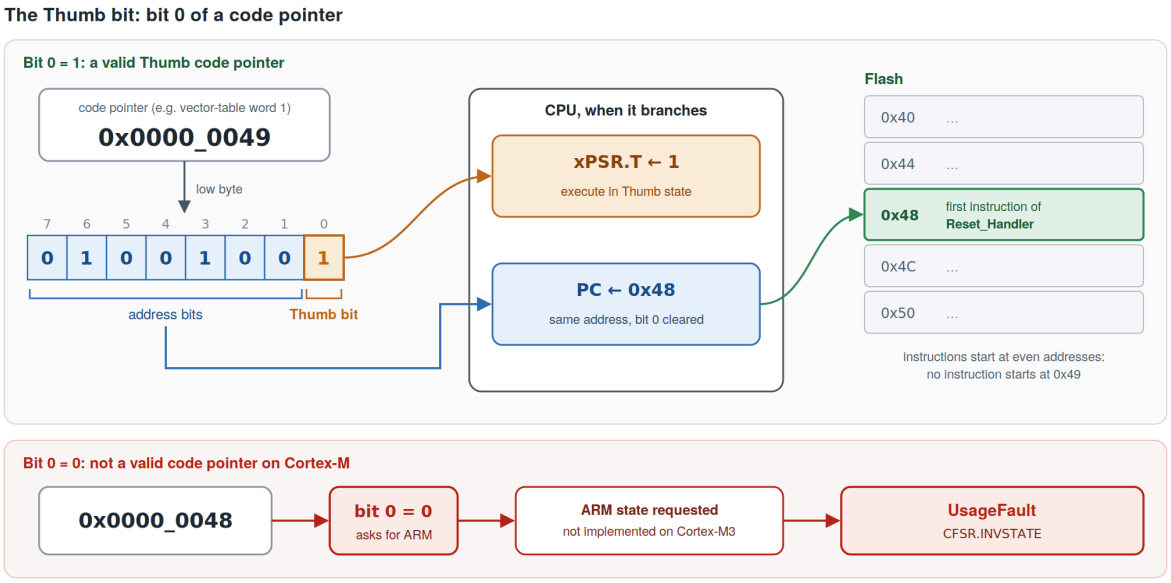
2.2 Entry Point and the Thumb Bit

ARM processors define two instruction sets:

- **ARM**, with 32-bit instructions,
- **Thumb**, with 16-bit instructions, extended in Thumb-2 to a mix of 16- and 32-bit ones. A core that supports both must know, at every branch, **which set the code at the target uses**.

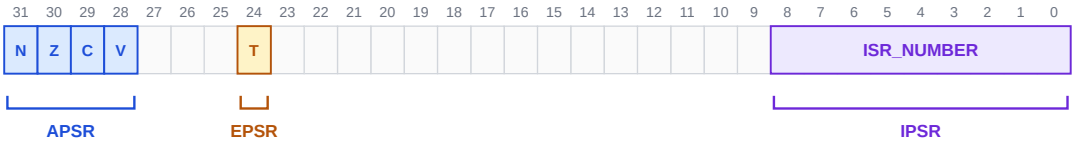
> **THE CORTEX-M3 EXECUTES ONLY THUMB, BUT KEEPS THE SAME CONVENTION.**

Thumb instructions are 2 or 4 bytes long and always start at an **even address**, so bit 0 of an instruction address is always 0 and carries **no information**.



ARM reuses that bit as a flag, the *Thumb bit*:

- **Bit 0 = 1**: the target is **Thumb code**.
  - The CPU copies the bit into the `T` bit of `xPSR` (the special register that describe the current state of the processor), clears it, and jumps to the even address. The `T` bit of `xPSR` represents the Thumb State.



APSR: condition flags · EPSR: Thumb state (always 1) · IPSR: running exception (0 = Thread mode)

Figure 1.2: *Bit layout of xPSR: N, Z, C, V flags in APSR, Thumb bit in EPSR, exception number in IPSR.*

- **Bit 0 = 0**: the target would be ARM code, which the Cortex-M3 **does not implement**.
  - The branch raises a UsageFault (`INVSTATE`) instead of executing anything at the target address.

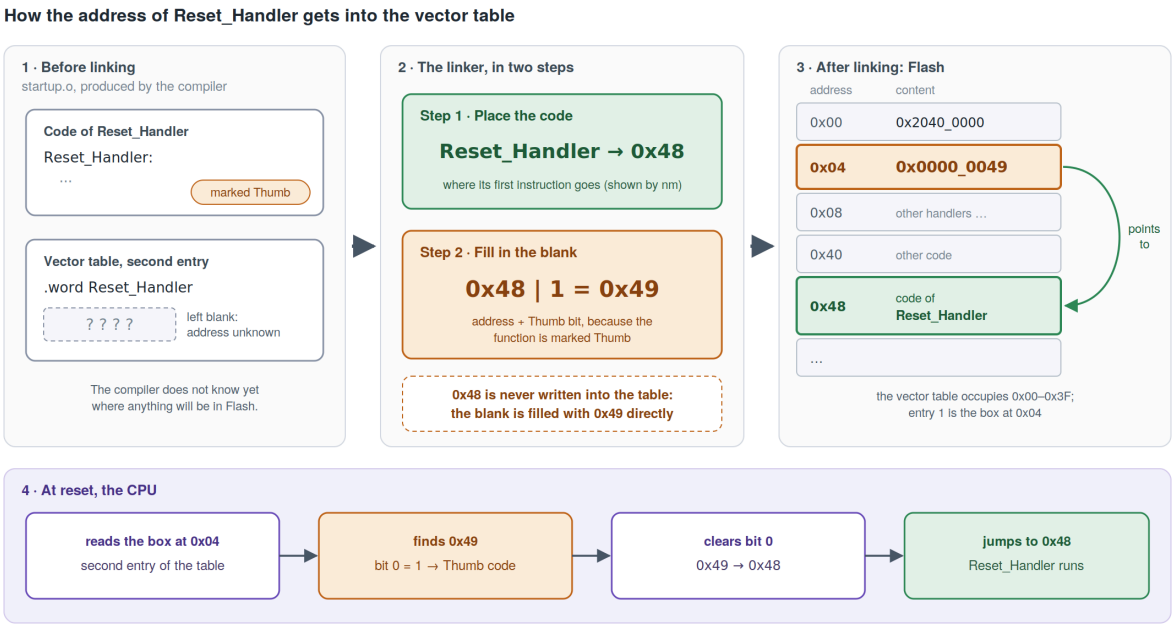
> **ONLY CODE POINTERS CARRY THE THUMB BIT THE THUMB BIT APPLIES ONLY TO ADDRESSES THE CPU JUMPS TO: FUNCTION ADDRESSES IN THE VECTOR TABLE, FUNCTION POINTERS, THE ELF ENTRY POINT. A DATA POINTER, SUCH AS `&COUNTER`, IS THE ADDRESS OF A VARIABLE THAT THE CPU READS OR WRITES, AND IS NEVER MODIFIED: IF `COUNTER` IS AT `0X20000004`, `&COUNTER` IS `0X20000004`.**
>
> The reason is alignment. Instructions always start at even addresses, so bit 0 of a code address is free to carry the Thumb bit. Data can also sit at odd addresses (a `char` at `0x20000005`), so for data bit 0 is a real part of the address.

2.2.1 How the linker sets the Thumb bit

The address of `Reset_Handler` reaches the vector table in four steps, from compilation to reset:

1. **Before linking**: `startup.o` contains the code of `Reset_Handler`, marked as Thumb. The second entry of the vector table only names `Reset_Handler` and is left **blank**, because its address is not known yet.

2. **Linking**: the linker first assigns the code of `Reset_Handler` the address `0x48`, which becomes the value of the symbol. It then resolves the reference in the vector table, filling the blank entry with `0x48 | 1 = 0x49`. The value `0x48` is never written into the table.
3. **After linking**: the entry at address `0x04` contains `0x49` and points to the code at `0x48`.
4. **At reset**: the CPU reads the entry at `0x04`, finds `0x49`, **clears bit 0**, and jumps to `0x48`.



2.2.1.1 WHICH TOOLS REPORT `0X48` AND WHICH `0X49`

Value	Meaning	Reported by
`0x48`	Location of the first instruction	Map file, `nm`, `objdump -d`
`0x49`	Code pointer, with the Thumb bit	`readelf -h` (entry point), `objdump -s -j .isr_vector` (vector table)

No instruction starts at `0x49`: the byte at `0x49` belongs to the instruction that starts at `0x48`, with **Thumb Bit** enabled.

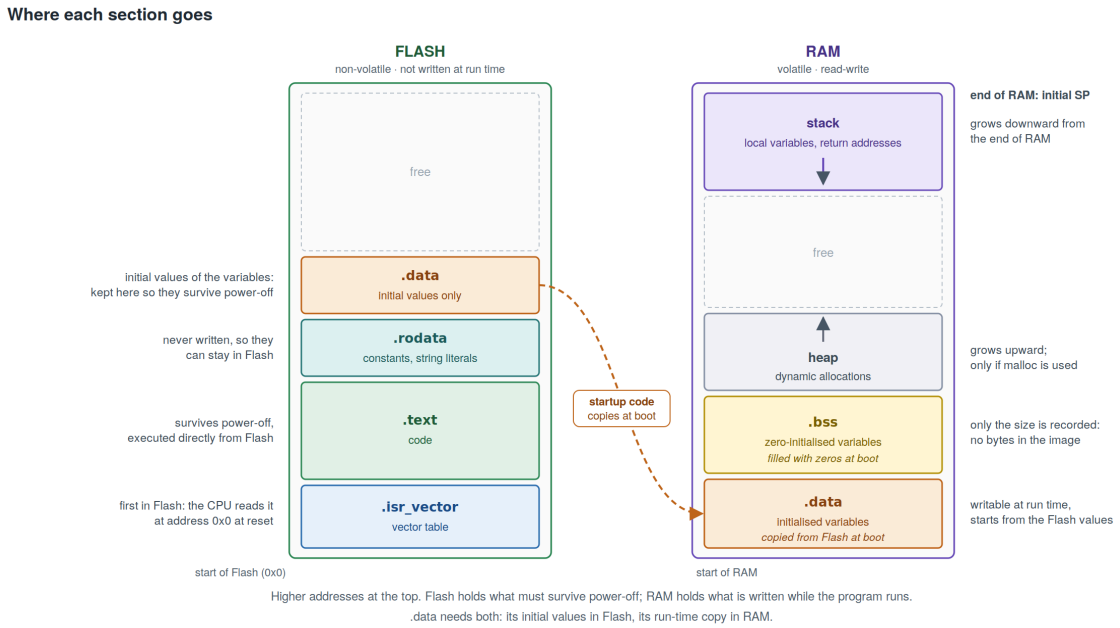
2.3 Memory Layout of the Image

The linker distributes the sections of the program between the memory regions declared in the linker script, Flash and RAM. Where each section goes depends on two questions: must its content survive power-off, and must it be writable at run time?

2.3.1 Where Each Section Goes

Section	Content	Placed in	Why
`.isr_vector`	Vector table	Start of Flash	The CPU reads it at address `0x0` at reset
`.text`	Code	Flash	Must survive power-off; executed directly from Flash
`.rodata`	Constants, string literals	Flash	Never written, so they can stay in Flash

Section	Content	Placed in	Why
`.data`	Initialised variables	RAM at run time, initial values stored in Flash	Must be writable, but their initial values must survive power-off: the startup code copies them
`.bss`	Zero-initialised variables	RAM, no bytes in the image	Only their size is recorded: the startup code fills them with zeros



RAM also holds two areas that contain nothing from the program's object files:

1. **heap**, space reserved by the linker script for dynamic allocation
2. the **stack**, which is not a section at all but an address range that grows downward from the end of RAM.

Inside each region the linker follows the **order of the linker script**, and inside each section the order of the object files on the command line. The order matters for the vector table: `.isr_vector` is written first in the script so that it starts at `0x0`, where the CPU looks for it.

The same layout gives the memory budgets:

```
Flash usage = .isr_vector + .text + .rodata + .data initial values
RAM usage   = .data + .bss + heap + stack
```

`.data` is counted twice, once in each budget. `arm-none-eabi-size` reports the totals as `text`, `data`, and `bss`; the size of the ELF file is not a measure of memory use, because it includes debug information and excludes `.bss`.

3 Inspecting the Image

The binutils tools **read the ELF file** on the host, without running anything on the target: they show what the toolchain produced, before the image is executed or debugged.

3.1 Inspection Commands

Command	Shows
`arm-none-eabi-size`	Section sizes in three totals (`text` , `data` , `bss`): the Flash and RAM budgets
`arm-none-eabi-nm`	Symbols with address and type letter (see Symbol Letters of nm)
`arm-none-eabi-readelf -h`	The ELF header, including the entry point
`arm-none-eabi-readelf -l`	The program headers: the segments loaded by QEMU
`arm-none-eabi-objdump -h`	One line per section: size, VMA, and LMA
`arm-none-eabi-objdump -d`	Disassembly: every instruction with the address where it is stored
`arm-none-eabi-objdump -s -j <section>`	The raw bytes of one section

RUN ADDRESS (VMA) AND LOAD ADDRESS (LMA)

In the `-h` listing each section has two addresses: the **VMA**, where the section is used at run time, and the **LMA**, where its bytes are stored in the image. They differ only for `.data` , whose initial values are stored in Flash and copied to RAM by the startup code.

The name "virtual memory address" comes from systems with an MMU. On a core without one, such as the Cortex-M3, both addresses are physical: the VMA is simply the run address.

READING AN `-s` DUMP

Each line shows the address of its first byte, 16 bytes in four groups of 4 in memory order, and the same bytes as text on the right (`.` for non-printable bytes).

3.2 Symbol Letters of nm

Letter	Meaning
`U`	Undefined: used here, defined in another file
`T` / `t`	Function (`.text`); `t` if `static`
`R` / `r`	Constant (`.rodata`); `r` if `static`
`D` / `d`	Initialised variable (`.data`); `d` if `static`
`B` / `b`	Zero-initialised variable (`.bss`); `b` if `static`

Letter	Meaning
`W`	Weak symbol, replaced by a normal definition with the same name
`A`	Absolute symbol, belonging to no section

3.3 Reading the Map File

The map file (`-Xlinker -Map=…`) is a text report written by the linker during the link. It records where each section and each symbol ended up, how big it is, and which object file it came from. It has two main parts: *Memory Configuration*, with the regions declared in the linker script, and *Linker script and memory map*, with the placement of every section.

Line type	Example	Meaning
Memory region	`FLASH 0x00000000 0x00400000`	Name, start, and length of a region
Output section (not indented)	`.text 0x00000040 0xa0`	A section of the final image: address and total size
Input section (indented)	`.text 0x00000040 0x64 build/boot/startup.o`	The piece of that section coming from one object file
Symbol	`0x00000048 Reset_Handler`	The final address of a function or variable

The map file answers questions such as: at which address did a function or variable end up, which object file contributed a piece of code, which section grew and made a region overflow. Static (file-local) symbols do not appear in it.

3.4 Which Tool Answers Which Question

Question	Tool
How much Flash and RAM does the program use?	`size`
At which address did a function or a variable end up?	`nm` , map file
Which object file supplied a piece of code?	map file
Why did a region overflow?	map file
What is the entry point?	`readelf -h`
Is the Thumb bit set in the vector table?	`objdump -s -j .isr_vector`
Is `.data` stored in Flash and run in RAM?	`objdump -h`

3.5 Working Example

This example follows a small program from source to memory: first it predicts in which section each declaration ends up, then it checks the prediction with `size` , `objdump` , the map file, and `nm` . The program is made of two source files, linked with a linker script `mps2_m3.ld` .

`src/main.c` declares one global of every kind:

```
#include <stdint.h>

const uint32_t limit = 100;
uint32_t counter = 5;
uint32_t total;
char buffer[64];
const char *greeting = "hello";

int main( void )
{
    total = counter + limit;
    buffer[0] = greeting[0];
    for ( ;; ) { }
}
```

`boot/startup.c` provides the vector table and a `Reset_Handler` that copies `.data` from Flash to RAM, fills `.bss` with zeros, and calls `main()`. The symbols `_estack`, `_sidata`, `_sdata`, `_edata`, `_sbss`, and `_ebss` are not variables: the linker script defines them as **addresses**, so the code uses only their address (`&_sdata`).

```
#include <stdint.h>

extern uint32_t _estack, _sidata, _sdata, _edata, _sbss, _ebss;
int main( void );

void Default_Handler( void )
{
    for ( ;; ) { }
}

void Reset_Handler( void )
{
    uint32_t *src = &_sidata;
    uint32_t *dst = &_sdata;
    while ( dst < &_edata )          /* copy .data from Flash to RAM */
        *dst++ = *src++;
    for ( dst = &_sbss; dst < &_ebss; )
        *dst++ = 0;                  /* zero .bss */
    main();
    for ( ;; ) { }
}

__attribute__(( section( ".isr_vector" ), used ))
```

```
const uint32_t vector_table[16] = {
    (uint32_t) &_estack,             /* 0: initial stack pointer */
    (uint32_t) Reset_Handler,        /* 1: reset                 */
    (uint32_t) Default_Handler,      /* 2: NMI                   */
    (uint32_t) Default_Handler,      /* 3: HardFault             */
};                                   /* 4-15: left at 0          */
```

`scripts/mps2_m3.ld` describes the two memory regions and the order of the output sections:

```
MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}

_estack = ORIGIN(RAM) + LENGTH(RAM);

ENTRY(Reset_Handler)

SECTIONS
{
    .isr_vector :
    {
        . = ALIGN(4);
        KEEP(*(.isr_vector))
    } > FLASH

    .text :
    {
        *(.text*)
        *(.rodata*)
        . = ALIGN(4);
        _etext = .;
    } > FLASH

    _sidata = LOADADDR(.data);
    .data :
    {
        . = ALIGN(4);
        _sdata = .;
        *(.data*)
        . = ALIGN(4);
        _edata = .;
```

```
    } > RAM AT > FLASH

    .bss (NOLOAD) :
    {
        . = ALIGN(4);
        _sbss = .;
        *(.bss*)
        *(COMMON)
        . = ALIGN(4);
        _ebss = .;
    } > RAM
}
```

The image is built with these commands:

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -ffreestanding -std=gnu17 -O0 -g3 -Wall \
    -c boot/startup.c -o build/startup.o
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -ffreestanding -std=gnu17 -O0 -g3 -Wall \
    -c src/main.c -o build/main.o
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostdlib \
    -T scripts/mps2_m3.ld -Xlinker -Map=build/output.map \
    build/startup.o build/main.o -lgcc -o build/example.axf
```

3.5.1 Step 1: Predict

Declaration	Section	Region
`main`, `Reset_Handler`	`.text`	Flash
`const uint32_t limit = 100;`	`.rodata`	Flash
`uint32_t counter = 5;`	`.data`	RAM, initial value stored in Flash
`uint32_t total;`, `char buffer[64];`	`.bss`	RAM, zeroed at boot
`const char *greeting = "hello";`	The pointer in `.data`, the characters in `.rodata`	Pointer in RAM, string in Flash

The last row is the subtle one: `greeting` is a writable variable that holds an address, while the string it points to is a constant.

3.5.2 Step 2: Measure the budgets with `size`

```
$ arm-none-eabi-size build/example.axf
   text    data     bss     dec     hex filename
```

```
    224       8      68     300     12c build/example.axf
```

- `text` = 224 bytes: the 64-byte vector table plus 160 bytes of code and constants.
- `data` = 8 bytes: `counter` and `greeting`, one 4-byte word each.
- `bss` = 68 bytes: `total` (4) and `buffer` (64).

Flash holds `text + data` = 232 bytes; RAM holds `data + bss` = 76 bytes, plus the stack. The ELF file itself is about 29 KB, almost all debug information.

3.5.3 Step 3: Check the sections with `objdump -h`

```
$ arm-none-eabi-objdump -h build/example.axf
Idx Name          Size      VMA       LMA
  0 .isr_vector   00000040  00000000  00000000
  1 .text         000000a0  00000040  00000040
  2 .data         00000008  20000000  000000e0
  3 .bss          00000044  20000008  000000e8
```

The vector table starts at `0x0` and `.text` follows it. `.data` runs at `0x20000000`, the start of RAM, but its bytes are stored at `0xE0`, right after the code: this is where the startup code copies them from. `.bss` follows `.data` in RAM; it has no bytes in the image (`objdump` marks it `ALLOC` only), so its LMA has no meaning. The debug sections listed after these are omitted.

3.5.4 Step 4: Read the map file

```
Memory Configuration

Name             Origin             Length             Attributes
FLASH            0x00000000         0x00400000         xr
RAM              0x20000000         0x00400000         xrw

Linker script and memory map

.isr_vector     0x00000000       0x40
 .isr_vector    0x00000000       0x40 build/startup.o
                0x00000000                vector_table

.text           0x00000040       0xa0
 .text          0x00000040       0x64 build/startup.o
                0x00000040                Default_Handler
                0x00000048                Reset_Handler
 .text          0x000000a4       0x30 build/main.o
                0x000000a4                main
 .rodata        0x000000d4        0xa build/main.o
```

```
                0x000000d4                limit
 *fill*         0x000000de        0x2
                0x000000e0                        _sidata = LOADADDR (.data)

.data           0x20000000        0x8 load address 0x000000e0
 .data          0x20000000        0x8 build/main.o
                0x20000000                counter
                0x20000004                greeting

.bss            0x20000008       0x44
 .bss           0x20000008       0x44 build/main.o
                0x20000008                total
                0x2000000c                buffer
```

The excerpt keeps only the lines discussed here. It confirms the prediction piece by piece: the code of each object file inside `.text`, the constants of `main.o` (`limit` and the 6 bytes of `"hello"`, 10 bytes in total) in a `.rodata` piece inside `.text`, two bytes of `*fill*` to end `.text` on a multiple of 4, and the `load address` of `.data`, which is also the value of `_sidata`, the symbol the startup code uses as the source of the copy.

3.5.5 Step 5: Find every symbol with `nm`

```
$ arm-none-eabi-nm -n build/example.axf
00000000 R vector_table
00000040 T Default_Handler
00000048 T Reset_Handler
000000a4 T main
000000d4 T limit
000000e0 T _etext
000000e0 A _sidata
20000000 D _sdata
20000000 D counter
20000004 D greeting
20000008 D _edata
20000008 B _sbss
20000008 B total
2000000c B buffer
2000004c B _ebss
20400000 R _estack
```

`counter` and `greeting` are `D`, `total` and `buffer` are `B`, as predicted. `limit` is the surprise: in `main.o` it is `R`, but in the final image it is `T`, because the linker script folds `.rodata` into the `.text` output section and `nm` classifies a symbol by the output section it ended up in. The string `"hello"` has no name, so it does not appear.

3.5.6 Step 6: Read the bytes with `objdump -s`

```
$ arm-none-eabi-objdump -s -j .isr_vector build/example.axf
 0000 00004020 49000000 41000000 41000000  ..@ I...A...A...

$ arm-none-eabi-objdump -s -j .text build/example.axf
 00d0 0c000020 64000000 68656c6c 6f000000  ... d...hello...

$ arm-none-eabi-objdump -s -j .data build/example.axf
 20000000 05000000 d8000000                    ........
```

The Cortex-M3 is little-endian: a 32-bit word is stored lowest byte first, so `00004020` is `0x20400000`.

- **Vector table**: word 0 is `0x20400000`, the initial stack pointer (`_estack`); word 1 is `0x49`, the address of `Reset_Handler` with the Thumb bit; words 2 and 3 are `0x41`, the address of `Default_Handler` with the Thumb bit.
- **Constants**: at `0xD4`, `64000000` is `limit` = 100; at `0xD8` the characters `hello` followed by the terminating zero.
- **Initial values of `.data`**: `05000000` is `counter` = 5, and `d8000000` is `greeting` = `0xD8`, exactly the Flash address of `"hello"`. A pointer is nothing more than a variable whose value is an address.

3.5.7 Result

Flash and RAM of the example



4 Running and Debugging the Image

4.1 Running the Image in QEMU

QEMU **emulates** the target board on the host, so the image can run before any physical hardware is used:

```
qemu-system-arm -machine mps2-an385 -kernel build/scratch-os.axf -nographic -semihosting
```

Option	Meaning
`-machine mps2-an385`	The board to emulate (here a Cortex-M3): its memory map must match the linker script
`-kernel`	The ELF image to boot
`-nographic`	Routes the target console to the host terminal
`-semihosting`	Lets the program print through the host (see below)

> ⚠ **QEMU DOES NOT EXIT**
>
> A bare-metal program has no operating system to return to: when `main()` returns, the startup code enters an infinite loop, so QEMU keeps running and must be stopped by hand.

4.2 Semihosting: Output Through the Host

A bare-metal program has **no console**: `printf` needs a C library and an operating system, and a UART needs a driver. **Semihosting** solves this during **development**: the program asks the **host**, the computer running QEMU or a debugger, to perform the output on its behalf.

The request follows a convention defined by ARM:

Element	Role	For printing a string
`r0`	Which service is requested	`0x04` (`SYS_WRITE0`)
`r1`	The argument of the service	Address of the string
`bkpt 0xab`	Signals that the request is ready	—

When the CPU executes `bkpt 0xab`, QEMU stops it, reads `r0` and `r1`, prints the string on the host terminal, and resumes the program at the next instruction. This is a **trap**: control passes to a more powerful entity, which performs the service and returns. A system call has the same shape, with the kernel in the role of the host.

```
static void semihost_write0( const char *message )
{
    register uint32_t    r0 __asm__( "r0" ) = 0x04;    /* SYS_WRITE0 */
    register const char *r1 __asm__( "r1" ) = message; /* string address */
    __asm__ volatile ( "bkpt 0xab" : : "r"( r0 ), "r"( r1 ) : "memory" );
}
```

C cannot choose registers nor emit `bkpt`, so the function uses two GCC extensions: `register … __asm__( "r0" )` forces a variable into a register, and inline assembly inserts the instruction (`volatile` stops the compiler from removing it).

> ⚠ **DEVELOPMENT TOOL ONLY**
>
> Semihosting works only when QEMU or a debugger is listening. On a real board with nothing attached, `bkpt` simply halts the CPU: production firmware prints through a peripheral such as a UART.

4.3 Debugging with `gdb`

A **debugger** stops the program, executes it one instruction or one line at a time, and shows registers and memory. On a microcontroller, where a crash often prints nothing, it is often the only way to see what the CPU does.

Debugging is **remote**: the program runs in QEMU, started with two extra options, and **gdb connects to it over TCP**:

- `-S`: freeze the CPU before its first instruction;
- `-gdb tcp::1234`: wait for gdb on port 1234.

gdb (`arm-none-eabi-gdb`, or `gdb-multiarch` on Linux) is started on the same ELF, from which it reads only symbols and debug information.

Command	Effect
`target remote :1234`	Connect to QEMU
`info registers sp pc`	Show registers
`x/2wx 0`	Examine 2 words in hex at address 0
`p/x $r1`	Print a register or an expression in hex
`stepi` (`si`)	Execute one machine instruction
`break main` (`b`)	Stop when execution reaches `main`
`continue` (`c`)	Run to the next breakpoint

4.4 Locating a Failure

Each kind of failure points to a **different stage** of the pipeline:

Symptom	Where to look
The compiler rejects a command	Toolchain installation and Makefile flags
Unresolved symbol	Object files on the link command and matching declarations
Section overflow	Memory regions in the linker script, map file

Symptom	Where to look
QEMU runs but prints nothing	Reset path, output call, `-semihosting` option; attach gdb to see how far execution gets

LOCATE THE FAILING STAGE

Read the **first** diagnostic and identify which stage failed before changing anything: changing compiler flags at random is not a diagnosis.

5 Build Automation with Make

`make` is not a compiler: it reads a `Makefile`, which describes how each file of the build is produced, and runs only the commands needed to bring the requested files up to date.

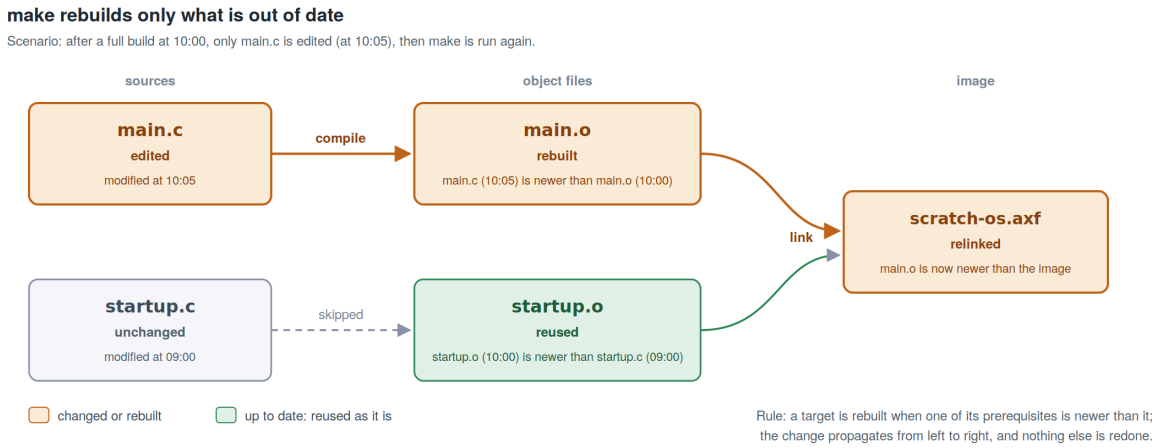
5.1 Rules and Rebuilds

A `Makefile` is a list of **rules**:

```
target: prerequisite1 prerequisite2
        recipe line 1
        recipe line 2
```

Part	Meaning
`target`	The file to produce, such as an object file or the ELF image
`prerequisites`	The files the target is made from; `make` brings them up to date first
`recipe`	The shell commands that produce the target; every line **starts with a Tab**, not spaces

A target is rebuilt only if it does not exist or if one of its prerequisites is **newer** than it (by modification time). Since prerequisites can be targets of other rules, the rules form a dependency graph, and a change propagates along it:



The rebuild is correct only if the graph lists every real dependency. A header or a linker script that is not listed as a prerequisite does not trigger a rebuild when it changes: `make clean && make` forces a full one.

5.2 The Tab Rule

A recipe line indented with spaces stops `make` with an error that names the line but not the cause:

```
$ make
Makefile:5: *** missing separator.  Stop.
```

Spaces and a Tab look the same in an editor; `cat -et` shows the difference by printing each Tab as `^I` and each line end as `$`:

```
$ cat -et Makefile
$(BUILD_DIR)/%.o: %.c$
    @mkdir -p $(dir $@)$
^I$(CC) $(CFLAGS) -c $< -o $@$
```

The first recipe line starts with four spaces, the second with a Tab (`^I`). Many editors insert spaces when the Tab key is pressed: configure them to keep real Tabs in Makefiles.

5.3 Variables

```
CC      := arm-none-eabi-gcc
CFLAGS  := -mthumb -mcpu=cortex-m3
CFLAGS  += -Wall -Wextra
```

Operator	Behaviour
`:=`	Evaluates the right-hand side once, immediately: the usual choice
`=`	Re-evaluates the right-hand side at every use
`+=`	Appends to the current value
`?=`	Assigns only if the variable is not already set, so it can be overridden from the command line (`make CC=clang`)

A variable is used with `$(NAME)` . By convention, `CC` , `CFLAGS` , `LDFLAGS` , and `LDLIBS` hold the compiler and its options; other names, such as `SOURCES` or `BUILD_DIR` , are the Makefile's own.

5.4 Pattern Rules and Automatic Variables

One **pattern rule** compiles every source file, instead of one rule per file:

```
$(BUILD_DIR)/%.o: %.c
        @mkdir -p $(dir $@)
        $(CC) $(CFLAGS) -c $< -o $@
```

`%` matches the same text on both sides: for `build/src/main.o` it matches `src/main` , so the prerequisite is `src/main.c` . Inside the recipe, **automatic variables** name the files of the rule being run:

Variable	Meaning
`$@`	The target
`$<`	The first prerequisite
`$^`	All the prerequisites
`$(dir $@)`	The directory of the target, created with `mkdir -p` before the compiler writes into it

The list of objects is derived from the list of sources with a **substitution**: `$(SOURCES:%.c=$(BUILD_DIR)/%.o)`

turns `src/main.c` into `build/src/main.o`. The source list can be written by hand or collected with `$(wildcard src/*.c)`: the explicit list is easier to check, the wildcard picks up new files automatically.

5.5 Phony Targets

```
.PHONY: all run clean
```

`run` and `clean` are actions, not files. Without `.PHONY`, a file called `clean` in the directory would make `make` think the target is up to date and skip it. `all`, the first target of the file, is the **default goal**, built when `make` is run with no arguments.

5.6 Debugging a Makefile

A Makefile can be wrong without failing, for example when a substitution builds the wrong list. What matters is what the variables expand to:

Tool	Effect
`$(info OBJECTS = $(OBJECTS))`	Prints a value while the Makefile is read
`$(warning …)`	Same, with the `file:line:` of the call
`make -n`	Prints the commands it would run, without running them

5.7 A Complete Makefile

```
CC       := arm-none-eabi-gcc
CFLAGS   := -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding -std=gnu17
CFLAGS   += -Wall -Wextra -O0 -g3
LDFLAGS  := -T scripts/mps2_m3.ld
LDFLAGS  += -Xlinker -Map=build/output.map
LDFLAGS  += -nostdlib -lgcc

BUILD_DIR := build
BIN       := $(BUILD_DIR)/scratch-os.axf
SOURCES   := boot/startup.c src/main.c
OBJECTS   := $(SOURCES:%.c=$(BUILD_DIR)/%.o)

.PHONY: all run clean
all: $(BIN)

$(BUILD_DIR)/%.o: %.c
        @mkdir -p $(dir $@)
        $(CC) $(CFLAGS) -c $< -o $@
```

```
$(BIN): $(OBJECTS)
        $(CC) $(CFLAGS) $(LDFLAGS) $(OBJECTS) -o $(BIN)

run: $(BIN)
        qemu-system-arm -machine mps2-an385 -kernel $(BIN) -nographic -semihosting

clean:
        rm -rf $(BUILD_DIR)
```

`make` compiles the stale objects and links them; run again with no changes, it does nothing. `make run` also boots the image in QEMU, and `make clean` deletes the `build/` directory, which holds every generated file.

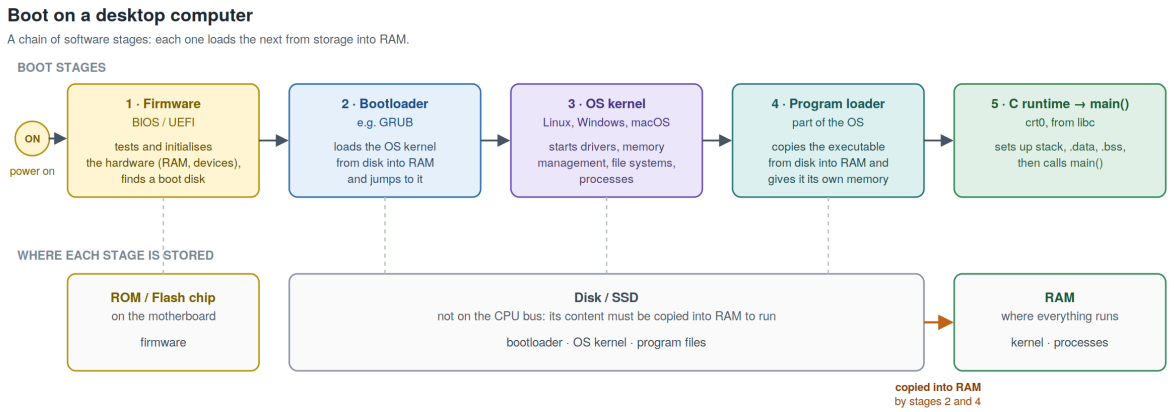
> ⚠ **LIBRARY ORDER**
>
> The linker takes from a library only the functions requested by the objects read **before** it. With `-lgcc` in `LDFLAGS`, which comes before the objects on the command line, code that needs a libgcc helper (such as a 64-bit division, `__aeabi_uldivmod`) fails to link. Moving `-lgcc` after the objects fixes it.

Part II

ScratchOS

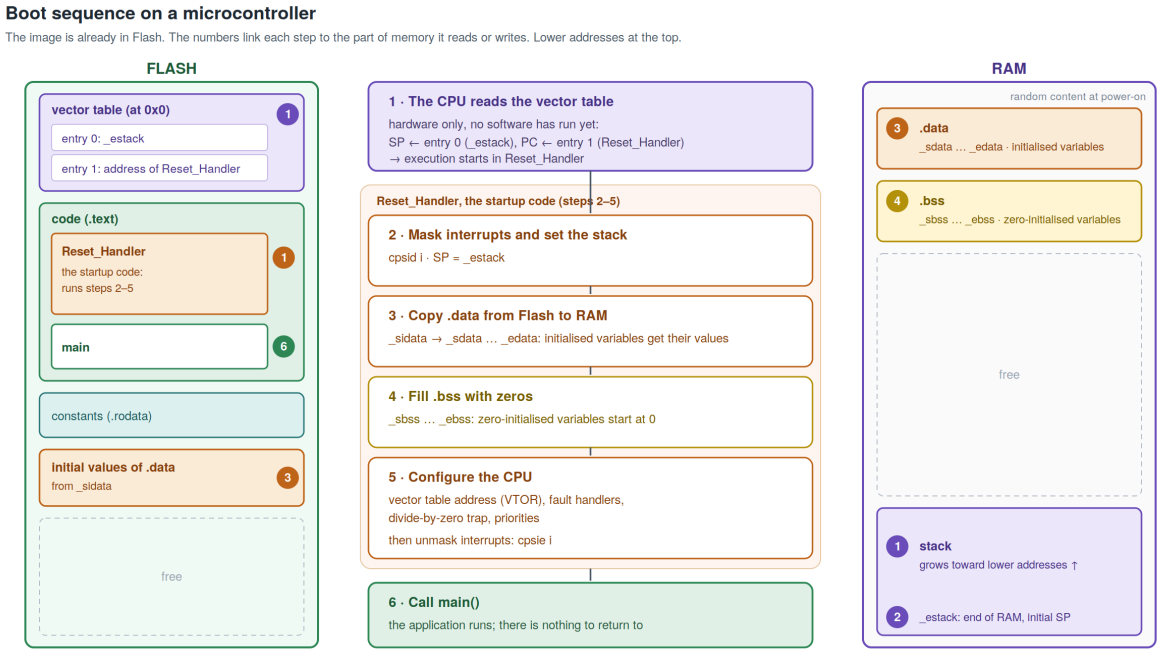
1 The Boot Problem

On a desktop, the operating system prepares everything before a program runs: its loader copies the executable from disk into RAM, sets up memory, and calls `main()`.



By the time main() runs, firmware, bootloader, kernel and loader have already prepared the hardware and the memory.

A microcontroller has no operating system and no loader. At reset the CPU only reads two words at address `0x0`, the initial stack pointer and the address of the first instruction, and starts executing. The code runs directly from Flash, but RAM holds random values at power-on: before `main()`, the program itself must set up the stack, copy the initial values of the variables from Flash to RAM, and clear the zero-initialised ones.



Order matters: the CPU must know where to start (step 1) before any instruction, including the copy of .data, can run.

The job is split between the **linker script**, used at build time, and the **startup code**, run right after reset.

2 Linker Script

A **linker script** is a text file, passed to the linker with `-T`, that describes the memories of the chip and how the program must be placed in them. The linker alone does not know which memories exist or where they are: the script tells it, so that it can assign every address, place each section in Flash or RAM, define the symbols the startup code needs, and fail the link if something does not fit.

```
MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}

ENTRY(Reset_Handler)

_estack = ORIGIN(RAM) + LENGTH(RAM);
_Min_Stack_Size = 0x1000;
_Min_Heap_Size = 0x1000;

SECTIONS
{
    .isr_vector :
    {
        . = ALIGN(4);
        KEEP(*(.isr_vector))
        . = ALIGN(4);
    } > FLASH

    .text :
    {
        . = ALIGN(4);
        *(.text*)
        *(.rodata*)
        . = ALIGN(4);
        _etext = .;
    } > FLASH

    _sidata = LOADADDR(.data);

    .data :
    {
        . = ALIGN(4);
        _sdata = .;
```

```
        *(.data*)
        . = ALIGN(4);
        _edata = .;
    } > RAM AT > FLASH

    .bss :
    {
        . = ALIGN(4);
        _sbss = .;
        *(.bss*)
        *(COMMON)
        . = ALIGN(4);
        _ebss = .;
    } > RAM

    .heap :
    {
        . = ALIGN(8);
        PROVIDE( end = . );
        PROVIDE( _end = . );
        _heap_bottom = .;
        . = . + _Min_Heap_Size;
        _heap_top = .;
    } > RAM

    __StackLimit = _estack - _Min_Stack_Size;
    ASSERT(__StackLimit >= _heap_top, "region RAM overflowed: heap collides with the stack")
}
```

Part	Role
`MEMORY { ... }`	declares the available memory regions: name, start address (`ORIGIN`), size (`LENGTH`), attributes (`r` , `w` , `x`)
`SECTIONS { ... }`	builds the output sections from the input sections of the object files and assigns each one to a memory region
`ENTRY(...)`	names the symbol where execution starts; used by tools (debugger, QEMU), not by the CPU
Symbol assignments (`_estack = ...;`)	create named addresses that the code can use, such as the top of the stack or the bounds of `.data` and `.bss`

2.1 MEMORY: Regions and Size Checking

Each line of `MEMORY` defines one **memory region**; the two regions correspond to the two populated memories of

the Cortex-M3 memory map.

Field	Meaning	Example
Name	freely chosen identifier, not a keyword	`FLASH`, `RAM`
Attributes	allowed access: `r` read, `w` write, `x` execute	`(rx)`, `(rwx)`
`ORIGIN`	start address of the region	`0x00000000`, `0x20000000`
`LENGTH`	size of the region	`4M`

The linker uses `LENGTH` to verify that the program fits: if the content assigned to a region does not fit, the link fails (e.g. `region 'FLASH' overflowed by 28 bytes`) instead of producing an image that cannot work.

The script also defines the symbol `_estack`, the **initial value** of the **stack pointer**: `ORIGIN(RAM) + LENGTH(RAM)` = `0x20400000`, just past the end of RAM. The stack grows **downwards**, so it starts at the top of RAM to have the most room. This address is never used itself: each push first decrements the stack pointer and then writes, so the first push writes at `0x203FFFFC`, the last word of RAM.

2.2 SECTIONS: From Input Sections to Output Sections

Each object file (`.o`) contains its own sections, called **input sections**:

- `.text` (code),
- `.rodata` (constants),
- `.data` (initialised variables),
- `.bss` (zero-initialised variables).

The linker merges them into the sections of the final ELF file, called **output sections**. The `SECTIONS` block contains one definition per output section, always with the same format:

```
<output_section> :
{
    <rule>            /* which input sections to take, e.g. *(.text*) */
    <rule>            /* rules are applied in order                   */
    <symbol> = .;     /* optional: give a name to the current address */
} > <REGION>          /* memory region where the section is placed    */
```

An **output section** is created with a name and placed in a memory region:

- `.text : { … }`: creates an output section called `.text` in the final ELF.
- `> FLASH`: places that output section in the FLASH region.

The rules inside the braces select the **input sections** that go into it:

- `*(.text*)`: from every object file (`*`), takes every input section whose name starts with `.text`.
- `*(.rodata*)`: same, for the read-only data.
- `KEEP(…)`: always keeps the selected input sections, even when the linker removes unused ones (garbage collection).

Addresses inside the section are controlled through the location counter:

- `.`: the **location counter**, the address where the linker is currently placing content.
- `. = ALIGN(4)` : rounds the location counter up to a multiple of 4.
- `_etext = .;` : defines a symbol equal to the current address, here the end of the code.

In this linker script the **mapping** is:

Input section (in each `.o`)	Created by	Rule	Output section	Region
`.isr_vector`	the startup code, on purpose	`KEEP(* (.isr_vector))`	`.isr_vector`	FLASH, at `0x0`
`.text`	the compiler: code	`*(.text*)`	`.text`	FLASH
`.rodata`	the compiler: constants	`*(.rodata*)`	`.text`	FLASH
`.data`	the compiler: initialised variables	`*(.data*)`	`.data`	RAM, stored in FLASH
`.bss`	the compiler: zero-initialised variables	`*(.bss*)` , `* (COMMON)`	`.bss`	RAM

The compiler chooses the names `.text` , `.rodata` , `.data` , and `.bss` by itself. `.isr_vector` is a name chosen by the startup code, which puts the vector table in a section of its own so that the linker script can place it first. `COMMON` holds globals declared without a value (`int counter;`) that some compilers keep in a separate pool; they belong in `.bss` too.

> **THE ORDER IN THE SCRIPT IS THE ORDER IN MEMORY**
>
> The linker places output sections in the order they are written in the script, and inside each output section it places input sections in the order of the rules. Because `.isr_vector` is the first output section in FLASH, the vector table starts at `0x0` , where the CPU reads it at reset. For the same reason, inside `.text` all the code comes first and all the constants after it.

2.2.1 Run Address (VMA) and Load Address (LMA)

```
_sidata = LOADADDR(.data);
.data :
{
    _sdata = .;       /* start of .data in RAM */
    *(.data*)
    _edata = .;       /* end of .data in RAM   */
} > RAM AT > FLASH
```

Every output section has two addresses:

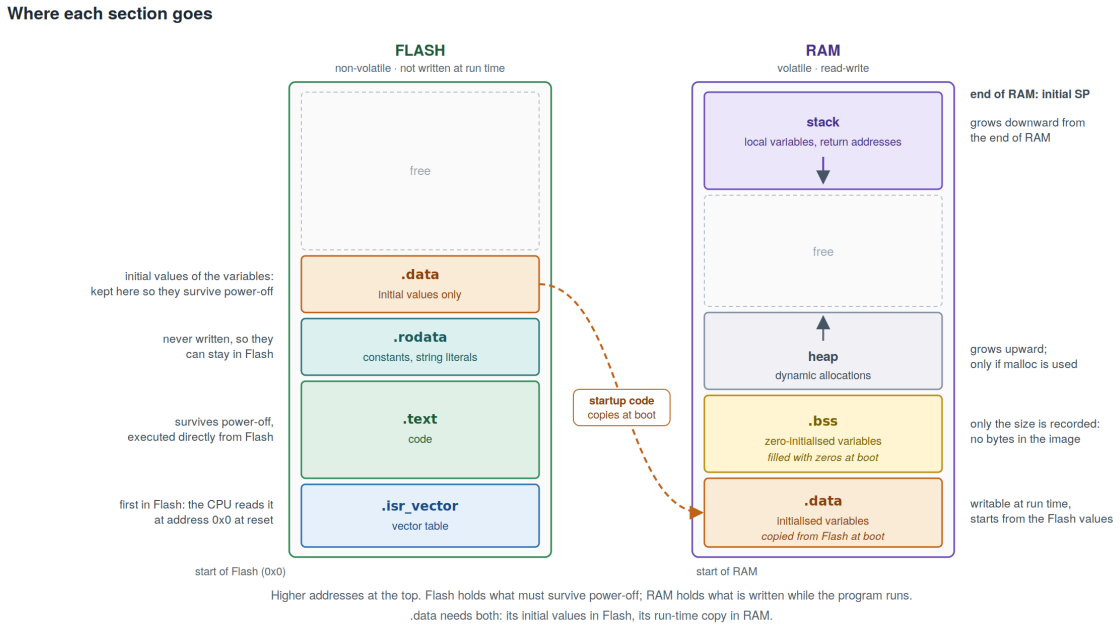
Address	Meaning	In the script
VMA (virtual memory address, *run address*)	where the program uses the section at run time	`> REGION`

Address	Meaning	In the script
LMA (load memory address)	where the section's bytes are stored in the image	`AT > REGION` (if omitted, LMA = VMA)

For most sections the two coincide. `.data` is the exception: `> RAM AT > FLASH` gives it a VMA in RAM, where the variables are modified, and an LMA in FLASH, where their initial values survive power-off. The script also records three addresses for the boot code:

Symbol	Value	Role in the copy
`_sidata`	`LOADADDR(.data)` : start of the initial values in FLASH (LMA)	source
`_sdata`	start of `.data` in RAM (VMA)	destination start
`_edata`	end of `.data` in RAM (VMA)	destination end

> **NOTHING COPIES LMA TO VMA AUTOMATICALLY**
>
> The linker only records the two addresses, and the CPU copies nothing at reset. The startup code must copy `.data` from `_sidata` to `_sdata` … `_edata` .



2.2.2 Heap and Stack Reservation

The linker script also reserves space in RAM for the heap and the stack, and checks at link time that they do not overlap:

```
.heap :
{
    . = ALIGN(8);
    _heap_bottom = .;
```

```
    . = . + _Min_Heap_Size;
    _heap_top = .;
} > RAM

__StackLimit = _estack - _Min_Stack_Size;
ASSERT(__StackLimit >= _heap_top, "region RAM overflowed: heap collides with the stack")
```

- `. = . + _Min_Heap_Size` : advances the location counter by 4 KB, so nothing else can be placed in the heap.
- `__StackLimit` : `_estack - _Min_Stack_Size`, the lowest address the stack may reach; the stack is not a section, only this address range.
- `ASSERT(cond, msg)` : makes the link fail with `msg` if `cond` is false.

HEAP–STACK COLLISION CAUGHT AT LINK TIME

The `ASSERT` makes the **link fail** if the heap would run into the stack limit. A memory corruption that would appear at run time, and be very hard to debug, becomes a clear error at build time.

A SMALLER RAM

With RAM `LENGTH = 8K`, `_estack` moves to `0x20002000` and `__StackLimit` to `0x20001000`. The sections `.data` + `.bss` + `.heap` still fit inside 8 KB, so there is no region overflow. However, the 4 KB heap now crosses `__StackLimit`, and the link fails because of the `ASSERT`.

2.2.3 Linker-Defined Symbols

Symbols such as `_sdata`, `_edata`, `_sidata`, `_sbss`, `_ebss` and `_estack` have **no storage**: no memory cell holds their value. The linker only assigns each one an **address**, and the address itself is the information.

```
extern uint32_t _sdata, _edata;   /* "there is something at this address" */
uint32_t *start = &_sdata;        /* correct: the address (0x20000000)    */
uint32_t  oops  = _sdata;         /* wrong: reads RAM at that address     */
```

They are declared as `extern` objects only so that C allows taking their address with `&`. Reading `_sdata` itself would read whatever happens to be in RAM at `0x20000000`. In assembly, `ldr r0, =_sdata` means `r0 = &_sdata;`. The boot code uses these addresses as loop bounds.

2.2.4 Orphan Sections

GCC lets the programmer put a variable or function in a section with any name:

```
int tagged __attribute__((section(".mysection"))) = 42;
```

If no rule in `SECTIONS` matches that name, the section is an **orphan**. The linker gives no error and places it by itself: an initialised writable section goes into RAM, right after `.data`, with its initial values in Flash; a read-only one goes into Flash.

> ⚠ **AN ORPHAN IS NOT INITIALISED BY THE STARTUP CODE**
>
> The startup code copies only the range `_sdata` … `_edata`. The orphan lies after `_edata`, outside that range, so its initial value is never copied: `tagged` reads 0 instead of 42. The link succeeds without warnings; the map file shows where the orphan ended up. Every section the program uses needs a rule in the linker script.

2.3 Section Placement of C Declarations

The section of each kind of declaration follows the mapping above. A few cases are easy to get wrong:

Case	Where it goes	Why
`int z = 0;`	`.bss`	"starts at zero" is exactly what `.bss` guarantees: no reason to store the zero in flash
`const char *p = "hi";`	`"hi"` → `.rodata` (FLASH), `p` → `.data` (RAM)	two distinct objects: the characters are constant, the pointer is an initialized writable variable
local variable (`int local = 7;` inside a function)	**no section**: the stack	it exists only while the function runs and has no fixed address
`static` local (`static int s = 3;` inside a function)	`.data` (or `.bss`)	it keeps one fixed address for the whole run, like a global; only its name is private

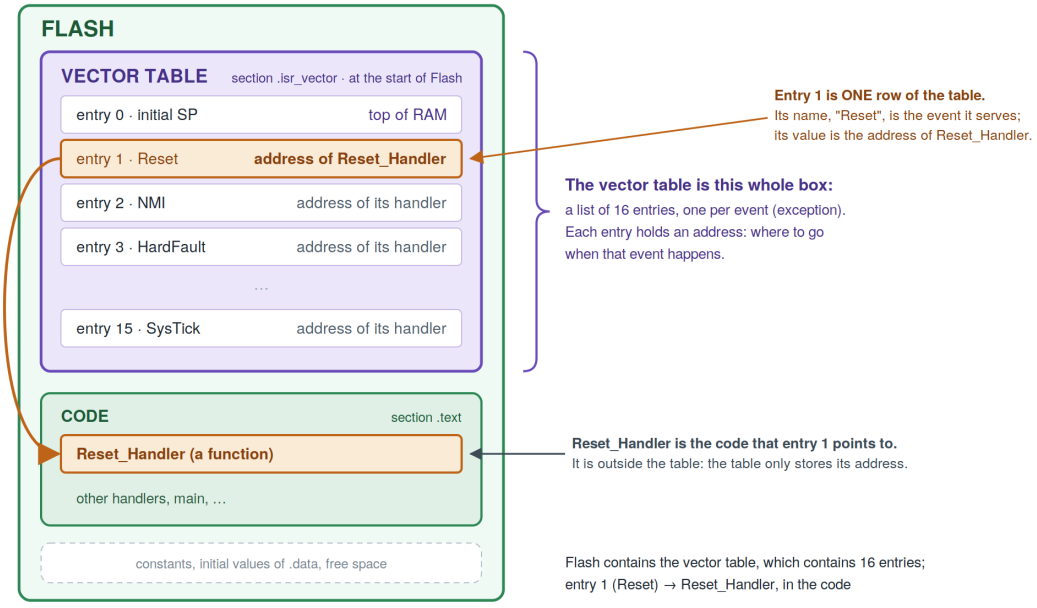
> ⚠ **`const` POINTER VS POINTER TO `const`**
>
> In `const char *greeting = "hi";` the `const` applies to the **characters**, not to the pointer. The pointer can still be changed (`greeting = "other";`), so it is an initialized writable variable and goes to `.data` (`D`), while `"hi"` goes to `.rodata`. Only `const char * const greeting = "hi";` makes the pointer itself constant, placing it in `.rodata` (`R`) as well. Reading rule: start from the name and read leftwards: "greeting is a pointer to constant chars".

3 The Vector Table

The **vector table** is a **table of addresses** at the **start of Flash**, one 32-bit word per entry. The CPU reads it by itself, without running any software: at reset, and every time an exception (an interrupt or a fault) occurs, to find the code it must run.

Entry	Address	Content
0	`0x00`	Initial value of the stack pointer (`_estack`)
1	`0x04`	Address of `Reset_Handler`, the first code to run
2–15	`0x08` – `0x3C`	Addresses of the handlers of the other exceptions (NMI, HardFault, SysTick, …)

Conceptual view: where the vector table is



At reset the CPU loads entry 0 into SP and entry 1 into PC, and starts executing `Reset_Handler`. Every address of code in the table is stored with the Thumb bit set: a handler at `0x40` is stored as `0x41`. The full list of exceptions is in the chapter on the ARM Cortex-M3 architecture.

The CPU looks for the table at address `0x0`, because `VTOR`, the register that holds its address, resets to `0x0000_0000`: this is why the linker script places `.isr_vector` first in FLASH. The program can later move the table, for example to a copy in RAM, by writing `VTOR`.

3.1 KEEP and Section Garbage Collection

With the option `-Wl,--gc-sections`, the linker performs **garbage collection of sections**:

1. it starts from the `ENTRY` symbol (`Reset_Handler`);
2. it follows every reference from one section to another (calls, pointers, addresses);
3. it **drops every input section that nothing references**.

This is useful to trim unused code, but it is dangerous for data that **only the hardware reads**. No code references

the vector table: the CPU reads it by itself at reset. For the linker it is therefore unused, and it would be discarded.

`KEEP(*(.isr_vector))` marks those input sections as **always kept**, whatever the garbage collector decides.

`--gc-sections`	`KEEP`	Result
off	present or absent	nothing is collected: `KEEP` changes nothing, it only acts as insurance
on	present	`.isr_vector` is kept at `0x0`: the image works
on	absent	`.isr_vector` is discarded: the image does not boot

> ⚠ **WHAT HAPPENS WITHOUT `KEEP`**
>
> 1. `.isr_vector` shrinks to size 0; the map file lists it under *Discarded input sections*.
> 2. `.text` becomes the first section in FLASH, so `Reset_Handler` lands at `0x0`.
> 3. At reset the CPU reads the first two words at `0x0`, which are now **instructions**, and loads them as SP and PC.
> 4. With a meaningless SP and PC, execution faults immediately and the CPU ends in **lockup**.

4 The Startup Code: `boot/startup.s`

The **startup code** (`boot/startup.s`) is the **first code the CPU executes** after reset, before `main()`. It contains the **vector table** and `Reset_Handler`, the function that prepares the machine to run C code.

```
// THE COMPLETE STARTUP CODE

/* Assembler settings */
    .syntax unified             /* modern ARM syntax */
    .cpu cortex-m3              /* only M3 instructions */
    .thumb                      /* Thumb code only */

/* 1. Vector table */
    .section .isr_vector,"a",%progbits
    .global isr_vector
    .type isr_vector, %object
isr_vector:
    .word _estack               /* 0: initial SP */
    .word Reset_Handler         /* 1: reset */
    .word NMI_Handler           /* 2: NMI */
    .word HardFault_Handler     /* 3: HardFault */
    .word MemManage_Handler     /* 4: MemManage */
    .word BusFault_Handler      /* 5: BusFault */
    .word UsageFault_Handler    /* 6: UsageFault */
    .word 0                     /* 7: reserved */
    .word 0                     /* 8: reserved */
    .word 0                     /* 9: reserved */
    .word 0                     /* 10: reserved */
    .word SVC_Handler           /* 11: SVCall */
    .word DebugMon_Handler      /* 12: debug monitor */
    .word 0                     /* 13: reserved */
    .word PendSV_Handler        /* 14: PendSV */
    .word SysTick_Handler       /* 15: SysTick */
    .size isr_vector, . - isr_vector

/* Default handlers: weak aliases of Default_Handler */
    .weak      NMI_Handler
    .thumb_set NMI_Handler, Default_Handler
    .weak      HardFault_Handler
    .thumb_set HardFault_Handler, Default_Handler
    .weak      MemManage_Handler
    .thumb_set MemManage_Handler, Default_Handler
    .weak      BusFault_Handler
```

```
    .thumb_set BusFault_Handler, Default_Handler
    .weak      UsageFault_Handler
    .thumb_set UsageFault_Handler, Default_Handler
    .weak      SVC_Handler
    .thumb_set SVC_Handler, Default_Handler
    .weak      DebugMon_Handler
    .thumb_set DebugMon_Handler, Default_Handler
    .weak      PendSV_Handler
    .thumb_set PendSV_Handler, Default_Handler
    .weak      SysTick_Handler
    .thumb_set SysTick_Handler, Default_Handler

/* 2. Reset_Handler */
    .section .text,"ax",%progbits
    .global Reset_Handler
    .type Reset_Handler, %function
    .thumb_func
Reset_Handler:

    /* Step 1: mask interrupts */
    cpsid i                     /* PRIMASK = 1 */

    /* Step 2: set SP */
    ldr r0, =_estack
    mov sp, r0

    /* Step 3: copy .data FLASH -> RAM */
    ldr r0, =_sidata            /* src */
    ldr r1, =_sdata             /* dst */
    ldr r2, =_edata             /* end */
copy_data:
    cmp r1, r2                  /* while (dst < end) */
    bhs copy_data_done
    ldr r3, [r0], #4            /* r3 = *src++ */
    str r3, [r1], #4            /* *dst++ = r3 */
    b copy_data
copy_data_done:

    /* Step 4: zero .bss */
    ldr r1, =_sbss              /* dst */
    ldr r2, =_ebss              /* end */
    movs r3, #0
zero_bss:
```

```
    cmp r1, r2                  /* while (dst < end) */
    bhs zero_bss_done
    str r3, [r1], #4            /* *dst++ = 0 */
    b zero_bss
zero_bss_done:

    /* Step 5: VTOR = &isr_vector */
    ldr r0, =0xE000ED08
    ldr r1, =isr_vector
    str r1, [r0]

    /* Step 6: SHCSR |= bits 16-18 */
    ldr r0, =0xE000ED24
    ldr r1, [r0]
    orr r1, r1, #0x00070000
    str r1, [r0]

    /* Step 7: CCR |= DIV_0_TRP */
    ldr r0, =0xE000ED14
    ldr r1, [r0]
    orr r1, r1, #(1 << 4)
    str r1, [r0]

    /* Step 8: priorities (SHPR1-3) */
    ldr r0, =0xE000ED18
    movs r1, #0x00
    strb r1, [r0, #0]           /* MemManage */
    strb r1, [r0, #1]           /* BusFault */
    strb r1, [r0, #2]           /* UsageFault */
    movs r1, #0x80
    strb r1, [r0, #7]           /* SVCall */
    movs r1, #0xE0
    strb r1, [r0, #10]          /* PendSV */
    strb r1, [r0, #11]          /* SysTick */

    /* Step 9: unmask interrupts */
    cpsie i                     /* PRIMASK = 0 */

    /* Step 10: call main, then hang */
    bl main
hang:
    b hang
```

```
    .ltorg                      /* literal pool */
    .size Reset_Handler, . - Reset_Handler

/* 3. Default_Handler */
    .global Default_Handler
    .type Default_Handler, %function
    .thumb_func
Default_Handler:
    b Default_Handler           /* for (;;) {} */
    .size Default_Handler, . - Default_Handler
```

The order of the steps is dictated by **dependencies**: nothing may interrupt a half-built system, C needs a stack, globals must hold their values before any C code reads them, and **the configuration must be complete before interrupts are accepted**.

Step	Action	Reason
1	mask interrupts (`cpsid i`)	nothing may interrupt a half-built system
2	set SP	C functions need a stack
3	copy `.data` FLASH → RAM	initialized globals get their values
4	zero `.bss`	C guarantees uninitialized globals start at 0
5–8	configure the CPU	vector table address, fault handlers, divide-by-zero trap, priorities
9	unmask interrupts (`cpsie i`)	setup is complete
10	`bl main` , then loop forever	hand over to C; on bare metal there is nothing to return to

WHY ASSEMBLY

When the startup code begins, the stack has not been set up yet and `.data` and `.bss` are not initialized. C code takes all of these for granted, so it cannot be trusted until the startup code has finished. On a desktop this job is done by **crt0**, the C library's startup code; on bare metal it is excluded (`-nostartfiles`) and written by hand.

4.1 Assembler Directives

Lines starting with a **dot produce no instructions**; they **steer the assembler**.

Directive	Effect
`.section name,"flags"`	selects the section; `a` = occupies space in the image, `x` = executable
`.global name`	exports the name to other files and the linker (otherwise it behaves like a `static` C name)
`.thumb_func`	marks the next label as Thumb code, so its address is stored with bit 0 set

Directive	Effect
`.word X`	stores the 32-bit value X
`.type`, `.size`	tell `nm` and gdb whether a name is code or data, and its size
`.weak name`	declares a weak symbol, overridable by a normal definition
`.thumb_set a, b`	makes `a` an alias of `b`, marked as Thumb code
`.ltorg`	emits the literal pool here

5 Reset Handler: From Reset to main()

`Reset_Handler` takes the place of the operating system and of the C library: it brings the memory and the CPU into the state that C code assumes, and only then calls `main()`.

> **A STARTUP THAT ONLY CALLS `MAIN`**
>
> Consider a minimal `Reset_Handler` that executes only `bl main`, and a global `char data_message[] = "hello";`:
>
> - the linker places the bytes `"hello"` in **FLASH** (LMA) and reserves space for `data_message` in **RAM** (VMA);
> - nobody copies the bytes from FLASH to RAM;
> - `main` reads `data_message` from **RAM**, where the copy was never written.
>
> The result depends on what RAM contains at power-on. QEMU fills RAM with zeros, so the first byte is `'\0'` and the string reads as **empty**. Real hardware leaves random content, so the string reads as **garbage**. The program compiles and runs, but the data is wrong: this is the kind of silent error the copy loop prevents.

5.1 Weak Default Handlers

Every entry of the vector table must point to **valid code**, even for exceptions the program does not handle yet; reserved entries simply hold `0`. Instead of writing nine real handlers, the startup code uses two pieces:

- **`Default_Handler`**: a single catch-all function, an infinite loop (`for (;;) {}`);
- **weak names**: every handler name in the table is declared `.weak` and, by default, is an alias of `Default_Handler` (same address).

A **weak** symbol is a default that gives way: if another file defines a normal (strong) function with the same name, the linker uses that one and drops the weak alias. A handler is installed simply by writing a C function with the right name, without touching the assembly. The C equivalent of each weak name is:

```
void HardFault_Handler(void) __attribute__((weak, alias("Default_Handler")));
```

With only `UsageFault_Handler` defined in `main.c`:

Entry	Name	Defined in C?	CPU jumps to	`nm`
3	`HardFault_Handler`	no	`Default_Handler`	`W`, same address as `Default_Handler`
6	`UsageFault_Handler`	yes	the C function	`T`, its own address
15	`SysTick_Handler`	no	`Default_Handler`	`W`, same address as `Default_Handler`

> **WHY AN INFINITE LOOP**
>
> An unexpected exception is a bug. Freezing at a known place lets a debugger show immediately that PC is in `Default_Handler`, instead of letting the CPU run off into random memory.

5.2 Interrupt Masking and Stack Setup

`cpsid i` masks interrupts (PRIMASK = 1) until the vector table, handlers and priorities are in place, and `cpsie i` accepts them again; PRIMASK is described in the chapter on the ARM Cortex-M3 architecture.

SP is set explicitly even though the hardware already loaded it from entry 0 of the vector table, because `Reset_Handler` is not always reached through a real reset: a bootloader or debugger may jump to it with SP holding any value. SP is a special register, so the address is first loaded into `r0` with `ldr` and then moved with `mov sp, r0`.

> ⚠ **FAILURE MODES OF A WRONG SP**
>
> With SP = `0x30000000` (outside RAM), the first push in `main` writes to a non-existent address and faults; reporting the fault requires stacking registers, which faults again, and the CPU enters lockup. With SP = `0x20000001`, the CPU silently clears the two low bits (SP is always word-aligned), so the stack starts at the bottom of RAM and grows *below* it, into an unmapped range where QEMU ignores writes: every call fails silently and nothing is printed.

5.3 Initializing .data and .bss

The `.data` copy translates `while (dst < &_edata) *dst++ = *src++;` with `r0` = `src` (`_sidata`), `r1` = `dst` (`_sdata`) and `r2` = end (`_edata`). It copies whole words, which is safe because the linker script aligns the start and end of `.data` to 4 bytes. Stepping through the loop in a debugger shows `r1` climbing `0x20000000`, `0x20000004`, … until it equals `&_edata`; only then does the string appear in RAM.

The `.bss` loop has the same shape without a source pointer: `while (dst < &_ebss) *dst++ = 0;`. When `.bss` is empty, `_sbss` equals `_ebss` and the loop exits immediately. Relying on the emulator zeroing RAM is incorrect: real SRAM is random at power-on. After these two loops, `Reset_Handler` provides exactly what a C runtime startup (crt0) provides.

5.4 Configuring the System Control Block

Steps 5–8 of `Reset_Handler` configure the System Control Block. The meaning of each register and field is described in the chapter on the ARM Cortex-M3 architecture; this section shows only what the boot code writes and why. All four registers are memory-mapped, so the startup code uses ordinary loads and stores: a plain store when the whole register is written, read-modify-write when other bits must keep their value, and byte stores for the priority bytes.

Step	Register	Operation	Effect
5	`VTOR` (`0xE000ED08`)	`= isr_vector`	exceptions use this table, wherever the image is placed
6	`SHCSR` (`0xE000ED24`)	`|= 0x00070000`	MemManage, BusFault, and UsageFault get their own handlers instead of escalating to HardFault

Step	Register	Operation	Effect
7	`CCR` (`0xE000ED14`)	`|= 1 << 4`	integer division by zero raises a UsageFault; `STKALIGN` stays 1, so `CCR` becomes `0x210`
8	`SHPR1` – `SHPR3` (`0xE000ED18` …)	byte stores	faults `0x00` , SVCall `0x80` , PendSV and SysTick `0xE0`

`VTOR` is written as a whole word, because all its bits are the table address. `SHCSR` and `CCR` contain status bits or bits that are already set (`STKALIGN`), so they are updated with read-modify-write and never overwritten. Each priority is a separate byte: viewing `SHPR1` – `SHPR3` as a 12-byte array at `0xE000ED18` , the boot code writes indices 0–2 (faults), 7 (SVCall), and 10–11 (PendSV, SysTick).

The effect of steps 6 and 7 is easy to observe with a division by a `volatile` zero. With both settings in place, a C function named `UsageFault_Handler` (which replaces the weak alias) runs and can report `CFSR = 0x2000000` , the `DIVBYZERO` bit. Without the `SHCSR` bits the fault escalates to HardFault and its kind is lost; without `DIV_0_TRP` the division silently returns 0 and `main` continues. The worked example and the fault status bits are in the fault status section of the Cortex-M3 chapter.

5.5 Additional Startup Work on Real Chips

QEMU's CPU runs at full speed out of reset. On a real microcontroller the startup code usually does more, often in a vendor function `SystemInit()` called from `Reset_Handler` :

- **Clock and PLL**: the chip starts on a slow internal oscillator, and a PLL multiplies the frequency up to full speed.
- **Flash wait states**: flash is slower than a fast CPU, so the flash controller must be told to insert wait cycles *before* the clock is raised, or the CPU reads garbage.
- **Watchdog**: a timer that resets the chip unless software periodically feeds it; on some chips it runs from reset, so boot code must feed or disable it.
- **FPU**: on a Cortex-M4F the floating-point unit is off at reset and must be enabled in **CPACR** (`0xE000ED88`), otherwise the first float instruction raises a UsageFault.
- **C++**: static constructors listed in `.init_array` must be called before `main` .

5.6 Verifying Boot State with gdb

Attached before the first instruction (see the toolchain chapter), gdb already shows SP = `0x20400000` and PC = `Reset_Handler` , while `x/2wx 0` shows `0x20400000 0x00000041` : no instruction has executed, so the hardware itself loaded both registers from the vector table, clearing the Thumb bit for PC. At `break main` , the SCB registers hold the values written in steps 5–8 (`VTOR` = `0x00000000` , `CCR` = `0x00000210` , `SHCSR` = `0x00070000` , `SHPR2` = `0x80000000` , `SHPR3` = `0xE0E00000`) and `info registers primask` reads 0 (it reads 0 at reset and 1 after `cpsid i`).

6 Freestanding C and Semihosting

6.1 Hosted and Freestanding Environments

The C standard defines two environments:

Aspect	Hosted	Freestanding
Example	desktop program	bare-metal Cortex-M3 image
Below the program	a C library (libc) and an OS	nothing
`printf`, `malloc`, `fopen`, `exit`	available	not available
Who starts `main()`	libc startup code, invoked by the OS	the application's own startup code

A freestanding environment still provides the headers that need **no run-time code**, only types, constants and macros supplied by the compiler:

Header	Provides
`stdint.h`	`uint32_t`, `int8_t`, …
`stddef.h`	`size_t`, `NULL`, `offsetof`
`stdbool.h`	`bool`, `true`, `false`
`stdarg.h`	`va_list` for variadic functions
`limits.h`, `float.h`	`INT_MAX`, `FLT_EPSILON`, …

There is no `stdio.h`, `stdlib.h` or `string.h`: no printing, no `malloc`, no `exit`, not even `memcpy`. This is why the boot code copies `.data` with its own loop.

6.2 The Layers Under printf

On a desktop, `printf` looks like a **single function** that writes to the screen. In reality it is the top of a stack of layers, and **most of the work happens in layers that a bare-metal system does not have**:

```
printf("x=%d\n", x)        program   formats the text into a buffer
  └─ write(1, buf, len)    libc      system call: the CPU enters the kernel
       └─ device driver    kernel    sends the bytes to the device
            └─ screen, UART, disk   hardware
```

`printf` itself only **formats**: it turns `"x=%d"` and `42` into the characters `x=42` in a buffer. **Getting those characters out of the machine** is the **OS's** job, reached through the `write()` system call. Many other libc functions have the same structure: a portable part on top, and a dependency on something underneath.

libc feature	Requires underneath
`printf`, `puts`	`write()`: a way to send bytes to a device
`malloc`, `free`	a heap region, grown with `sbrk()`

libc feature	Requires underneath
`exit`	something to return to
`main()` being called	crt0: sets up stack, `.data`, `.bss`, then calls `main`

Embedded C libraries such as **newlib** keep the portable part and leave the bottom functions (`write` , `sbrk` , `exit` , …) as **hooks** to implement for each board. The firmware described in these notes links **no libc at all**: there is nothing yet to put under the hooks, `Reset_Handler` already does the work of crt0, and the image contains no hidden code.

> **WHAT REPLACES `PRINTF`**
>
> Without libc, output is produced by the program itself: in this chapter through **semihosting**, and later through a UART driver and the kernel's own `kprintf` / `printf` (see the chapter on devices, interrupts, and system calls).

6.3 Why libgcc Is Kept

```
CFLAGS := -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding -std=gnu17
LDFLAGS += -nostdlib -lgcc
```

The flags are described in the toolchain chapter. `libgcc` is kept because the compiler sometimes turns a plain C operator into a **call** to a helper when the CPU has no single instruction for it. These helpers need neither OS nor libc, and dropping them would make harmless code fail to link.

C code	CPU	GCC emits
`a / b` (32-bit)	Cortex-M0 (no divide instruction)	`bl __aeabi_idiv`
`a / b` (32-bit)	Cortex-M3	`sdiv` (hardware)
`x / y` (64-bit)	Cortex-M3	`bl __aeabi_uldivmod`

Calling `printf` in such a build always fails: with a toolchain that ships no C library, compilation stops with `stdio.h: No such file or directory` ; with a toolchain that ships newlib, the header is found but the link fails with `undefined reference to 'printf'` because of `-nostdlib` .

6.4 Semihosting Output via bkpt 0xab

Without libc and without a UART driver, the program has no way of its own to print anything. All the output of the boot code is produced with **semihosting**: the program places `0x04` (`SYS_WRITE0`) in `r0` and the address of a NUL-terminated string in `r1` , then executes `bkpt 0xab` , and the host (QEMU or a debugger) prints the string on its terminal. The convention, the C wrapper `semihost_write0` , and its limits are described in the toolchain chapter.

MULTIPLE INSTRUCTIONS IN ONE `__ASM__`

C concatenates adjacent string literals, so `"ldr r2, [r0]" "ldr r3, [r1]"` reaches the assembler as the single line `ldr r2, [r0]ldr r3, [r1]` and fails with `garbage following instruction`. Each instruction must end with `\n` (commonly `\n\t`) inside the string.

1 Peripherals and Buses of the MPS2-AN385 Platform

Before writing a driver, it is necessary to know the hardware it talks to: which board is emulated, which peripherals it contains, where they sit in the address space, and how the CPU reaches them.

1.1 CMSDK Hardware and CMSIS Software Conventions

Two Arm standards cover the two sides of a peripheral:

- **CMSDK** defines the hardware blocks,
- **CMSIS** defines how software names and accesses them.

Aspect	CMSDK	CMSIS
Full name	Cortex-M System Design Kit	Cortex Microcontroller Software Interface Standard
Nature	**hardware**: ready-made peripheral designs placed next to a Cortex-M core	**software**: a standard for the C headers and helper functions that describe a chip
Examples	APB UART, APB timer and dual timer, watchdog, AHB GPIO	`core_cm3.h`, `NVIC_EnableIRQ()`, `UART0->DATA`, `UART0TX_IRQn`
Key property	registers documented once by Arm, identical on every chip that integrates the block	shipped by chip vendors with their parts
In this context	the peripherals of the MPS2-AN385 board	a register header written by hand **in CMSIS style**, without vendor libraries, so names match real vendor code

1.2 Board, Clock, and Peripheral Address Map

The code runs on an emulated Arm development board. This section identifies the board, the single clock that times everything, and the addresses where its peripherals are located.

Element	Description
MPS2	Arm development board whose main chip is an **FPGA**: a reprogrammable chip whose circuit is loaded at power-on, so the same board can imitate different microcontrollers
AN385	the circuit (FPGA image, described in Arm Application Note 385) used here: a **Cortex-M3** CPU plus CMSDK peripherals
QEMU `mps2-an385`	emulator of exactly this system: the code runs on the PC without the physical board
Clock	one **25 MHz** clock shared by CPU and peripherals; every timing value (UART speed, timer period) is computed from it
Peripheral region	addresses `0x40000000` – `0x5FFFFFFF`, reserved by the Cortex-M memory map for devices; each device occupies a 4 KiB window

Figure 4.1 — MPS2-AN385 peripheral region (0x4000_0000…)

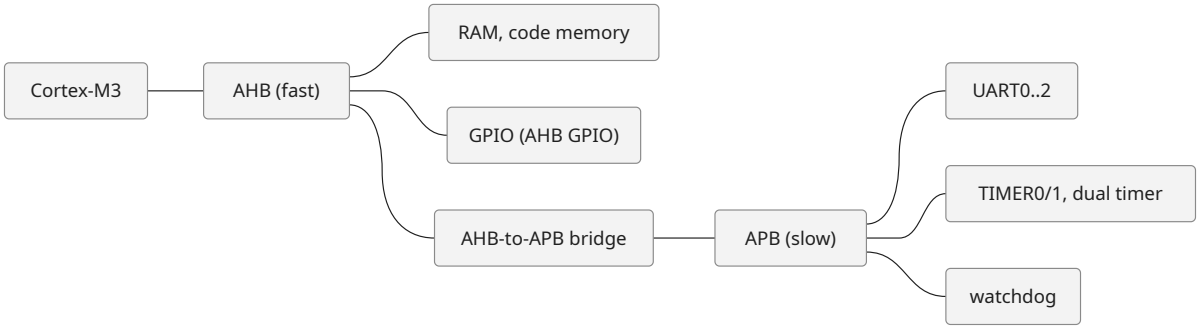


1.3 AHB and APB Buses

The CPU does not talk to every device in the same way: fast and slow devices sit on two different buses, connected by a bridge.

A **bus** carries every load and store from the CPU to a memory or a device.

Figure 4.2 — AHB and APB buses of the MPS2-AN385



Bus	Characteristics
AHB (Advanced High-performance Bus)	pipelined, one transfer per cycle: memories, GPIO
APB (Advanced Peripheral Bus)	simple, slow, low-power: rarely accessed registers (UART, timers, watchdog)
AHB-to-APB bridge	translates each access: a few extra cycles per APB register

For software **the bus is transparent**: only the timing differs. Software does not need to know which bus a device is on: every register access is the same load or store instruction, and the bridge routes it automatically. The only difference is speed: an APB access takes a few extra clock cycles.

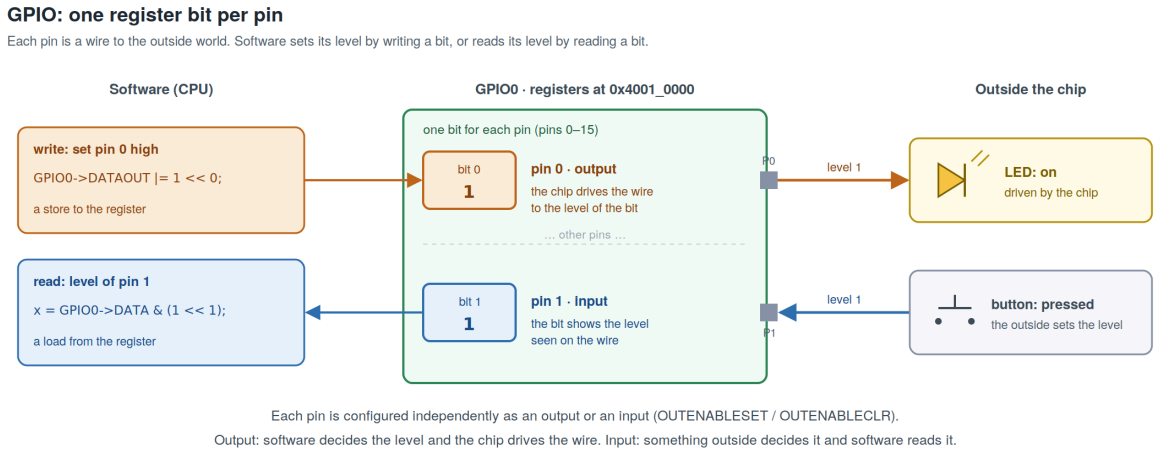
1.4 Standard I/O Interfaces and DMA

Microcontrollers exchange data with the outside world through a small set of standard **interfaces**; a DMA controller can additionally move data without using the CPU.

Interface	Wires	Operating principle	Typical use
GPIO	1 per pin	each pin is a digital input (level read) or output (driven to 0 or 1)	buttons, LEDs, chip-selects
UART	2 (TX, RX)	asynchronous: no clock wire, both sides agree on a **baud rate**	debug console, PC link
SPI	clock, data out, data in, 1 select per device	synchronous; the controller drives the clock, one bit out and one in per tick; **chip-select** picks the device	flash, displays (MHz)
I2C	2 (clock, data)	synchronous multi-drop; the controller first sends the 7-bit device **address**	sensors (100–400 kHz)
ADC / DAC	1 per channel	analog voltage to number (ADC) or number to voltage (DAC)	temperature, audio

1.4.1 GPIO

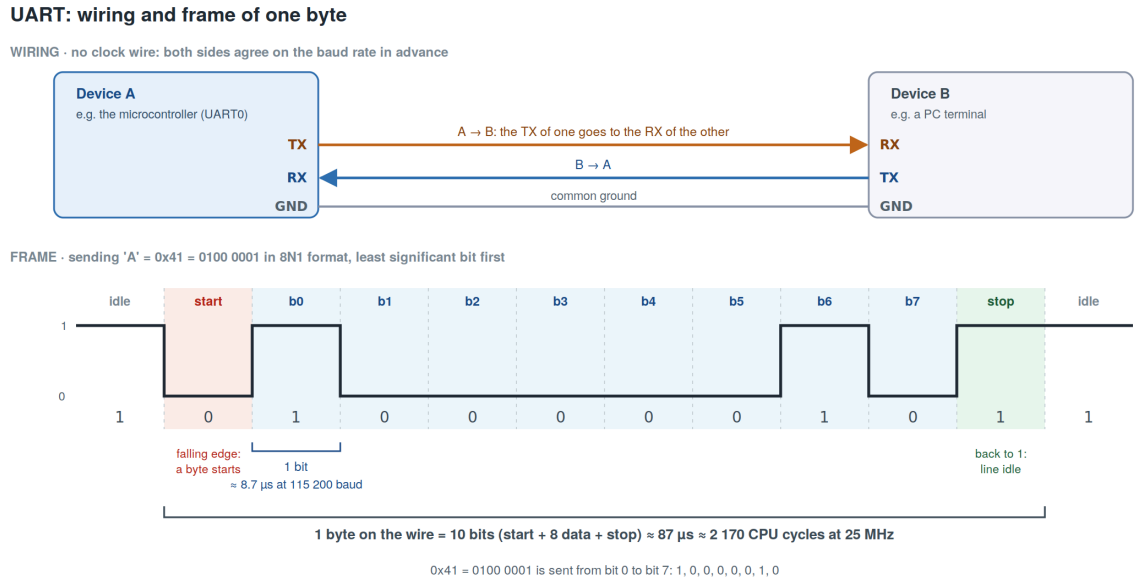
A **GPIO** (General-Purpose Input/Output) pin is the simplest interface: a **single wire** whose level the software can **read** (input) or **set** to 0 or 1 (output). When a pin is used as an output, the electrical circuit that drives it determines whether the pin can safely share its wire with other devices.



1.4.2 UART

A **UART** (Universal Asynchronous Receiver-Transmitter) sends bytes one bit at a time over a single wire per direction. It is **asynchronous**: there is no clock wire, so transmitter and receiver must agree in advance on the speed, the **baud rate** (bits per second), and each side times the bits with its own clock.

Element	Description
Wires	TX (transmit) and RX (receive), crossed between the two devices: the TX of one goes to the RX of the other; plus a common ground
Idle line	held at 1 when nothing is sent
Start bit	one 0 bit: the falling edge tells the receiver that a byte is starting and synchronizes its timing
Data bits	usually 8, sent **least significant bit first**
Parity bit	optional error check (none in the common **8N1** format: 8 data bits, no parity, 1 stop bit)
Stop bit	one 1 bit: returns the line to idle before the next byte



Transmission of `'A'` (`0x41` = `0100 0001`) in 8N1 format, bit 0 first:

```
idle  start  b0  b1  b2  b3  b4  b5  b6  b7  stop  idle
 1      0     1   0   0   0   0   0   1   0    1     1
```

Each byte therefore takes **10 bits** on the wire (start + 8 data + stop). At 115 200 baud one bit lasts about 8.7 µs, so one byte takes about **87 µs**: a long time for a CPU running at 25 MHz, which is why the way the CPU waits for the UART (polling or interrupts) matters.

1.4.3 DMA

Moving data between a peripheral and memory normally costs CPU time: the CPU must read each byte from the device and store it in memory, or the opposite. A **DMA controller** (Direct Memory Access) is a separate hardware unit that **performs these copies** on its own, **leaving the CPU free** to execute other code while the transfer is in progress.

Aspect	Description
Operation	copies source → destination for a count, using the bus while the CPU runs; usually signals completion with an interrupt
Directions	peripheral → memory (UART RX, ADC); memory → peripheral (SPI TX)
Costs	bus contention; the buffer must not be touched during the transfer
AN385	no DMA: the CPU moves every byte

2 Memory-Mapped I/O

On a Cortex-M, devices have **no special I/O instructions**: each device register is a **memory address**, and the CPU controls the device with ordinary loads and stores.

With **memory-mapped I/O** (MMIO), every peripheral register has a fixed physical address and is accessed with ordinary load and store instructions:

```
#define UART0_DATA (*(volatile uint32_t *)0x40004000u)

UART0_DATA = 'A';      /* a store to 0x40004000: the UART transmits 'A' */
```

The assignment compiles to a single `str` at `0x40004000`: the syntax is a variable access, the meaning is defined by the hardware.

2.1 Device Registers Are Not RAM

Although a device register is accessed like a variable, **it does not behave like memory:** reading or writing it can trigger actions in the hardware.

Property of a device register	Example
A **read** can have a side effect	consuming a received byte, acknowledging an event
A **write** can start an action	a transmission, a flag clear, a pin change
The value can **change without any store**	a status bit set when a byte arrives
The value read back can **differ** from the value written	read-only bits, self-clearing bits, reserved bits that must be written as zero or preserved

2.2 The `volatile` Qualifier

The C compiler assumes that **memory changes only when the program writes it**. For device registers this assumption is false, and `volatile` tells the compiler not to rely on it.

Under optimization the compiler caches values, removes repeated reads, and merges stores: all invalid for device registers.

> ⚠ **WHAT `VOLATILE` DOES AND DOES NOT DO**
>
> It forces every source access to be performed, in order with respect to other `volatile` accesses. It gives **no atomicity** and **no synchronization** between `main` and a handler.

2.3 Read-Modify-Write Hazards

A plain assignment to a register, `REG = value;`, is a single store: one instruction, which cannot be interrupted halfway. A compound assignment such as `|=`, `&=` (or operation like `x++`) looks like one operation too, but the CPU performs it in three steps:

```
CMSDK_GPIO0->DATAOUT |= (1u << 0);      /* set bit 0 of the output latch */
```

```
    ldr  r1, [r0]       /* 1. READ   the whole register into r1   */
    orr  r1, r1, #1     /* 2. MODIFY bit 0 inside the CPU         */
    str  r1, [r0]       /* 3. WRITE  the whole value back         */
```

Between the read and the write, the **register can change**, or the value written back can do more than intended.

2.3.1 Two Kinds of Register

To change one pin without touching the others, the GPIO (`0x40010000`) offers two kinds of register:

- **registers that hold all the pins together**, such as `DATAOUT` (the levels driven on the output pins): a store overwrites every pin, so changing one pin requires a read-modify-write;
- **set/clear pairs**, such as `OUTENABLESET` and `OUTENABLECLR` (make pins outputs or inputs): a store acts only on the bits written as 1 and ignores the bits written as 0, so a single plain store changes one pin and no read is needed.

```
CMSDK_GPIO0->OUTENABLESET = (1u << 0);     /* safe: one store, pin 0 becomes an output */
CMSDK_GPIO0->DATAOUT     |= (1u << 0);     /* risky: read-modify-write of all the pins */
```

2.3.2 When Read-Modify-Write Fails

Hazard	What goes wrong	Example
An interrupt in between	another context changes the register after the read; the write puts back the old value and cancels that change	`main` reads `DATAOUT` to set bit 0; an interrupt sets bit 1; `main` writes back the value it read, and bit 1 is lost
Write-1-to-clear bits	in some registers, writing a 1 *clears* a flag; writing back the value read clears every flag that was set	UART `INTSTATUS` reads `0b11` (TX and RX pending); `INTSTATUS
The read is not what was written	the value read describes something else; writing it back changes bits that were never meant to be touched	GPIO `DATA` reads the levels of all pins, inputs included; writing it back drives the outputs with the levels just observed

2.3.3 Safe Alternatives

- **Use a set/clear register** whenever the device has one (GPIO `OUTENABLESET` / `OUTENABLECLR` , NVIC `ISER` / `ICER`): no read at all.
- **Write only the bit, with a plain store**, in write-1-to-clear registers: `CMSDK_UART0->INTSTATUS = UART_INT_TX;` clears TX and leaves the other flags alone.
- **Protect the read-modify-write** when it cannot be avoided: mask interrupts around it (`irq_save()` … `irq_restore()` , see the section on critical sections), or do it before interrupts are enabled, as the boot code does for `CCR` .

2.4 Register Overlays with C Structures

Instead of one macro per register, a whole register block can be described by a C `struct` placed at the device's base address, so that every register becomes a named field.

```
typedef struct
{
    volatile uint32_t DATA;      /* 0x00 read = received byte, write = byte to send */
    volatile uint32_t STATE;     /* 0x04 status flags */
    volatile uint32_t CTRL;      /* 0x08 control flags */
    volatile uint32_t INTSTATUS; /* 0x0C interrupt status; write 1 to clear a bit */
    volatile uint32_t BAUDDIV;   /* 0x10 baud-rate divider */
} CMSDK_UART_TypeDef;

#define CMSDK_UART0   ( ( CMSDK_UART_TypeDef * ) 0x40004000UL )
```

Element	Meaning
Layout	fields at consecutive 4-byte offsets from the base address
`CMSDK_UART0`	a constant pointer, not a variable: `->STATE` is a load from `0x40004004`
Safety	`volatile` fields; bit names next to the `struct` (`UART_STATE_TXBF` is `1UL << 0`); `_Static_assert` + `offsetof` catch layout typos at compile time

The same technique describes SysTick, NVIC, SCB, timers, watchdog, GPIO, and FPGAIO.

2.5 Registers as a Hardware Protocol

Device registers must be used in a precise order, like the steps of a protocol; the **driver is the code that implements this order**.

Figure 4.3 — Register protocol of a device



Rule	Example
Configuration precedes enabling	speed before switching the transmitter on
A status check precedes every transfer	write `DATA` only when there is room, read it only when a byte is present
An event is acknowledged	clearing its flag lets the device report the next one

2.6 Peripherals under QEMU

QEMU **does not emulate every device of the board equally well**. At each address of the peripheral region

there can be:

- **a real model**: QEMU imitates the device, and its registers react as on the hardware. Writing a byte to the UART `DATA` register prints it on the terminal, status bits change, interrupts arrive. A driver for such a device can be tested in the emulator. Examples: UART0, timers, SysTick, NVIC.
- **a placeholder**: QEMU only reserves the address, so that accesses do not fault, but there is no logic behind it. A driver for such a device cannot be tested, because right and wrong code behave the same. Example: GPIO0.
  - Writes are ignored
  - reads return 0
- **nothing**: no device answers at that address. Example: `0x40080000`.

The QEMU monitor shows which case applies at each address:

```
printf 'info mtree -f\nquit\n' | qemu-system-arm -M mps2-an385 -display none -S -monitor stdio
-serial null | grep ' 00000000400' | sort -u
```

```
0000000040000000-0000000040000fff (prio 0, i/o): cmsdk-apb-timer
0000000040002000-0000000040002fff (prio 0, i/o): cmsdk-apb-dualtimer
0000000040004000-0000000040004fff (prio 0, i/o): uart
0000000040008000-0000000040008fff (prio 0, i/o): cmsdk-apb-watchdog
0000000040010000-0000000040010fff (prio -1000, i/o): cmsdk-ahb-gpio
0000000040020000-0000000040020fff (prio 0, i/o): pl022
0000000040022000-0000000040022fff (prio 0, i/o): arm_sbcon_i2c
0000000040028000-0000000040028fff (prio 0, i/o): mps2-fpgaio
```

A real model appears with `prio 0` and its name, a placeholder with `prio -1000` (here the GPIO), and an empty address does not appear at all.

Running QEMU with `-d unimp` prints a line for every access to a placeholder or to an empty address. If it prints nothing, every device the program uses is a real model, so any malfunction is in the code.

> **CORE VERSUS BOARD PERIPHERALS**
>
> **Core** peripherals (SysTick, NVIC) have addresses fixed by ARMv7-M and exist on every Cortex-M. **Board** peripherals (UART0, GPIO0, FPGAIO) exist where the board places them, modelled as faithfully as the emulator allows.

3 Device Drivers and Polled UART I/O

The first driver built on these concepts is the UART, the serial port used as console. On top of it, a small formatting library provides `kprintf` for kernel output.

3.1 CMSDK UART Register Interface

The UART *(Universal Asynchronous Receiver-Transmitter)* sends and receives bytes on a serial line. It is controlled through five registers at `0x40004000`.

Offset	Register	Access	Content
`0x00`	`DATA`	R/W	write: byte to send; read: received byte (clears RXBF)
`0x04`	`STATE`	R; bits 2–3 W1C	TXBF, RXBF, TXOR, RXOR
`0x08`	`CTRL`	R/W	TXEN, RXEN, TXIRQEN, RXIRQEN, overrun IRQ enables
`0x0C`	`INTSTATUS`	R/W1C	TX, RX, TXOR, RXOR interrupt pending
`0x10`	`BAUDDIV`	R/W	clock / baud rate, minimum 16

DATA 8-bit

UART DATA (0x00), low byte of the 32-bit register



Bits	Field	Description
7:0	DATA	Write: byte to send; read: received byte (clears RXBF)

STATE 8-bit

UART STATE (0x04), low byte of the 32-bit register



Bits	Field	Description
3	RXOR	Receive overrun: a byte was lost (write 1 to clear)
2	TXOR	Transmit overrun: a write was lost (write 1 to clear)
1	RXBF	Receive buffer full: a byte is waiting
0	TXBF	Transmit buffer full: the previous byte is still leaving

CTRL 8-bit = 0x03

UART CTRL (0x08) after uart_init, low byte



Bits	Field	Value	Description
5	RXORIRQEN	0	Receive-overrun interrupt enable
4	TXORIRQEN	0	Transmit-overrun interrupt enable
3	RXIRQEN	0	Receive interrupt enable
2	TXIRQEN	0	Transmit interrupt enable
1	RXEN	1	Receiver enable
0	TXEN	1	Transmitter enable

INTSTATUS 8-bit

UART INTSTATUS (0x0C), write 1 to clear a bit



Four facts about the UART shape the driver:

- **One byte of buffer per direction.** A byte written to `DATA` while the previous one is still leaving (`TXBF` = 1) is lost, and `TXOR` is set; a byte arriving while the previous one has not been read (`RXBF` = 1) is lost, and `RXOR` is set. The driver therefore checks `TXBF` before every write.
- **Off after reset.** `CTRL` is 0, so the transmitter and receiver are disabled and writes to `DATA` are silently ignored: `uart_init` must enable them first.
- **Speed on the real board.** `BAUDDIV` divides the 25 MHz clock: 25 000 000 / 115 200 ≈ 217 gives 115 200 baud.
- **Speed under QEMU.** The emulator ignores the line speed; any value from 16, the minimum, works, so the driver writes 16.

3.2 The Device Driver Abstraction

A driver separates *what* the rest of the system wants to do (send a byte) from *how* a particular device does it (which registers to touch, in which order). It is the only code that touches the device's registers; everything else calls its functions.

All character devices (UART, terminal, …) offer the same three operations. They can be collected in a `struct` whose fields are **pointers to functions**: each field holds the address of the function that performs that operation for one specific device.

```
struct char_driver
{
    void ( *init )( void );       /* bring the device up     */
    int  ( *read )( void );       /* one byte, or -1 if none */
    void ( *write )( char c );    /* send one byte           */
};

/* The UART driver fills the struct with its own functions */
static const struct char_driver uart0_driver = {
    uart_init,                    /* init  */
    uart_getc_polled,             /* read  */
    uart_putc_polled              /* write */
};

/* Generic code: works with any driver passed to it */
void say_hi( const struct char_driver *dev )
{
    dev->write( 'h' );            /* calls the write function of that driver */
    dev->write( 'i' );
}
```

`say_hi( &uart0_driver )` prints "hi" on the UART; passing another driver prints it on another device, without changing `say_hi`.

Aspect	Description
Interface	every driver offers the same operations (init, read, write); replacing the UART means replacing one driver file
Generalization	operating systems use this pattern for every class of device: Linux `struct file_operations` (`.open`, `.read`, `.write`, …) is filled in by each driver
This kernel	with a single console the `struct` is not needed: the kernel calls `uart_putc_polled` directly

3.3 Polled UART Driver

The simplest UART driver uses **polling**: the CPU repeatedly reads the status register until the device is ready, **without using interrupts**.

```
#include "drivers/uart.h"
#include "cmsdk.h"

void uart_init( void )
{
    CMSDK_UART0->BAUDDIV = 16;                          /* configure */
    CMSDK_UART0->CTRL = UART_CTRL_TXEN | UART_CTRL_RXEN;    /* enable */
}

void uart_putc_polled( char c )
{
    while ( CMSDK_UART0->STATE & UART_STATE_TXBF )          /* check status: room? */
    {
    }
    CMSDK_UART0->DATA = (uint32_t) (unsigned char) c;       /* move data */
}

int uart_getc_polled( void )
{
    if ( ( CMSDK_UART0->STATE & UART_STATE_RXBF ) == 0 )    /* check status: byte? */
    {
        return -1;
    }
    return (int) ( CMSDK_UART0->DATA & 0xFF );              /* reading DATA takes the byte */
}
```

Function	Protocol steps	Behaviour
`uart_init`	configure (`BAUDDIV`), enable (`CTRL`)	brings the UART to a known state
`uart_putc_polled`	check TXBF in a loop, write `DATA`	blocks until there is room

Function	Protocol steps	Behaviour
`uart_getc_polled`	check RXBF once, read `DATA`	never blocks: returns `-1` if no byte is waiting

Polled: status is checked by the CPU, not signalled by interrupts. The cast through `unsigned char` keeps a byte such as `0xE9` from becoming `0xFFFFFFE9` when `char` is signed.

> **ENABLE BEFORE MOVING DATA**
>
> Without `uart_init`, `CTRL` stays 0 and every byte is silently dropped (`-d guest_errors`: `transmit data write with Tx disabled`).

> **MOVE TO THE C APPENDIX**
>
> Section 3.4 (variadic functions) belongs in the appendix; leave a one-line reference in 3.5.

3.4 Variadic Functions and `<stdarg.h>`

A `printf`-like function accepts a variable number of arguments. C supports this with *variadic functions* and the macros of `<stdarg.h>`.

```
void kprintf( const char *fmt, ... )
{
    va_list ap;                     /* a cursor over the extra arguments    */
    va_start( ap, fmt );            /* start right after the last named one */
    int n = va_arg( ap, int );      /* take the next argument, as an int    */
    va_end( ap );                   /* done                                 */
}
```

Rule	Meaning
Argument count and types	unknown to the function: the **format string** supplies them; a mismatch is undefined behaviour
Default promotions	a `char` arrives as an `int`: `%c` uses `va_arg( ap, int )`
Freestanding	`<stdarg.h>` has only types and built-ins, no library code
Convention	a function taking a `va_list` is named `v…` (`vprintf`)

3.5 Building a Kernel printf over the UART

Without a C library there is no `printf`: the kernel builds its own, `kprintf`, which sends each character to the UART. The work is split in two:

- **formatting**: `vformat` turns the format string and its arguments into characters, one at a time;
- **output**: each character is passed to a function `out( c, ctx )`, which decides where it goes (for `kprintf`, the UART).

This way the same `vformat` can later serve other destinations.

```
static void put_string( void ( *out )( char c, void *ctx ), void *ctx, const char *s )
{
    while ( *s != '\0' )
    {
        out( *s, ctx );
        s++;
    }
}

void vformat( void ( *out )( char c, void *ctx ), void *ctx, const char *fmt, va_list ap )
{
    char num[12];   /* longest number + '\0' */

    while ( *fmt != '\0' )
    {
        if ( *fmt != '%' )              /* ordinary character: pass it on */
        {
            out( *fmt, ctx );
            fmt++;
            continue;
        }
        fmt++;                          /* a '%': look at the next letter */
        switch ( *fmt )
        {
        case 'c':
            out( (char) va_arg( ap, int ), ctx );   /* a char arrives as an int */
            break;
        case 's':
        {
            const char *s = va_arg( ap, const char * );
            put_string( out, ctx, s != 0 ? s : "(null)" );  /* no crash on null */
            break;
        }
        case 'd':
        {
            int v = va_arg( ap, int );
            uint32_t mag;
            if ( v < 0 ) { out( '-', ctx ); mag = 0u - (uint32_t) v; }
            else         { mag = (uint32_t) v; }
            utoa_dec( mag, num );
            put_string( out, ctx, num );
            break;
```

```
        }
        case 'u': utoa_dec( va_arg( ap, uint32_t ), num ); put_string( out, ctx, num ); break;
        case 'x': utoa_hex( va_arg( ap, uint32_t ), num ); put_string( out, ctx, num ); break;
        case '%': out( '%', ctx ); break;
        case '\0': out( '%', ctx ); return;          /* string ended right after '%' */
        default:  out( '%', ctx ); out( *fmt, ctx ); break;  /* unknown: as written */
        }
        fmt++;                          /* past the conversion letter */
    }
}
```

Aspect	Description
Conversions	`%c` , `%s` , `%d` , `%u` , `%x` (no `0x`), `%%` ; no width; unknown ones printed literally
`%d` magnitude	`0u - (uint32_t) v` , not `-v` : `-INT_MIN` overflows (UB), unsigned subtraction is always defined

```
static void kprintf_out( char c, void *ctx )
{
    (void) ctx;                         /* not used */
    uart_putc_polled( c );
}

void kprintf( const char *fmt, ... )
{
    va_list ap;
    va_start( ap, fmt );
    vformat( kprintf_out, 0, fmt, ap );
    va_end( ap );
}
```

`kprintf( "d=%d\n", -42 )` makes six `kprintf_out` calls. Polled, unbuffered, and interrupt-free, `kprintf` works in **any context**, fault handlers included.

3.6 Fault Handlers That Print the Cause

Without output, every fault ends in `Default_Handler` , an infinite loop that gives no clue about what went wrong. With `kprintf` , each fault can get its own handler, written in C, that replaces the weak default: it reads the fault status register of the core, prints it, and only then halts.

The CPU records the cause of a fault in one of two status registers, so each handler prints the one that matches its fault:

Register	Address	Content
CFSR (Configurable Fault Status)	`0xE000ED28`	one bit per cause of MemManage, BusFault, UsageFault

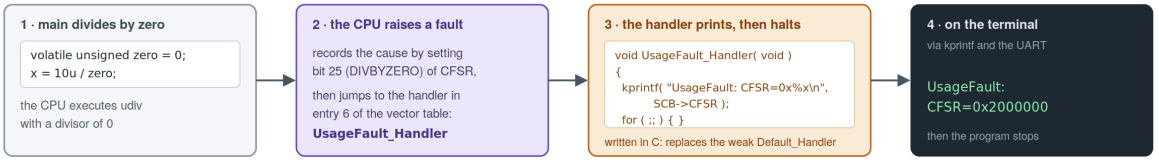
Register	Address	Content
HFSR (HardFault Status)	`0xE000ED2C`	why a HardFault occurred

DIVISION BY ZERO

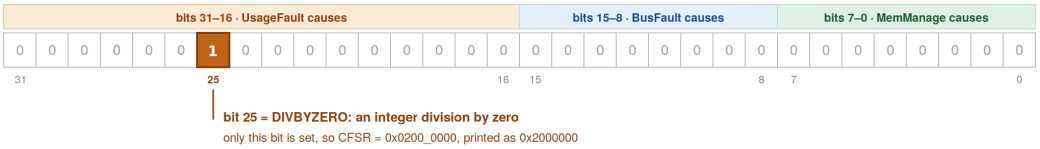
With `CCR.DIV_0_TRP` set, an integer division by a `volatile` zero (`volatile` prevents compile-time evaluation) prints `UsageFault: CFSR=0x2000000` : bit 25, `DIVBYZERO` .

A fault handler that prints the cause

Example: a division by zero, with CCR.DIV_0_TRP set by the boot code.



How to read 0x2000000: the CFSR register (0xE000ED28)



Every possible cause has its own bit: the printed value tells exactly what went wrong.
Without this handler the fault would end in Default_Handler, an infinite loop that prints nothing.

4 Polling and Interrupts

Polling wastes CPU time; interrupts let a device call the CPU only when needed. The hardware mechanism that enters and leaves a handler is described in the exception model of the chapter on the ARM Cortex-M3 architecture; this section compares the two approaches and shows what the mechanism means for driver code.

4.1 Polling versus Interrupts

There are two ways to find out when a device is ready: the CPU keeps asking (**polling**), or the device signals the CPU (**interrupt**). Each has its uses.

Aspect	Polling	Interrupts
CPU time	spent in the waiting loop even when nothing happens	spent only per event (entry, handler, exit)
Latency	up to one trip around the loop; grows with the devices checked	about 12 cycles of entry on Cortex-M3, nearly constant
Complexity	one loop	handler, shared data, priorities
CPU between events	busy	free, or asleep (`wfi`)
Suited to	certain, imminent events; contexts without interrupts (fault handlers)	rare or unpredictable events, long transfers

One byte at 115 200 baud lasts ~87 µs, about **2 170 cycles** at 25 MHz wasted by polling.

4.2 Interrupt Handlers Are Ordinary C Functions

On exception entry the Cortex-M3 pushes `r0` – `r3`, `r12`, `lr`, `PC`, and `xPSR` on the current stack and loads `EXC_RETURN` into `lr`. These are exactly the registers that the AAPCS allows a C function to clobber, and any compiled C function already preserves `r4` – `r11`. A device handler is therefore written as a plain C function whose name matches the vector table entry, and its ordinary return becomes an exception return. The register roles of the AAPCS, the stacked frame, `EXC_RETURN`, and tail-chaining are described in the Cortex-M3 chapter; the frame can be observed with gdb, as shown in the SysTick section below.

5 The SysTick Timer and Wraparound-Safe Time Keeping

An operating system needs a notion of time. SysTick, a **timer** built into every Cortex-M, generates a **periodic interrupt** (the *tick*) from which the kernel counts **milliseconds**.

5.1 SysTick Configuration

SysTick is the 24-bit down-counter built into every Cortex-M core. Its four registers (`CTRL` , `LOAD` , `VAL` , `CALIB` at `0xE000E010` – `0xE000E01C`), the choice of clock source, and the differences between SysTick and a device interrupt are described in the SysTick section of the chapter on the ARM Cortex-M3 architecture. The driver needs only three facts: one period lasts `LOAD + 1` cycles of the processor clock (25 MHz on `mps2-an385`); writing `CTRL = 7` starts the counter on the processor clock with the exception enabled; and SysTick is a core exception (number 15), so its priority is set in `SHPR3` and its handler needs no acknowledgement.

5.2 Reload Value and Tick Rate

The tick period is chosen by writing a reload value; it is computed from the clock frequency and the desired number of ticks per second.

From period = (`LOAD` + 1) / f: **`LOAD` = f / rate − 1**. At 25 MHz and 100 ticks/s, `LOAD` = 25 000 000 / 100 − 1 = **249 999** (`0x3D08F`), which fits in 24 bits; the longest period is 2^24 cycles ≈ 0.67 s.

Tick	Rate	Resolution	Exceptions/s	Note
1 ms	1 000 Hz	1 ms	1 000	in QEMU runs about 26% slow in wall-clock time
10 ms	100 Hz	10 ms	100	classic OS rate; exact wall-clock time in QEMU

A faster tick gives finer resolution at the cost of more exceptions.

5.3 SysTick Driver and Uptime Counter

The driver starts SysTick and counts elapsed milliseconds in a variable updated by the interrupt handler.

```
#define CPU_CLOCK_HZ 25000000UL     /* processor clock: 25 MHz */

static volatile uint32_t uptime_ms; /* changed by the handler */

void systick_init( void )
{
    uptime_ms = 0;
    SysTick->LOAD = CPU_CLOCK_HZ / ( 1000 / TICK_MS ) - 1;   /* 249 999 for TICK_MS = 10 */
    SysTick->VAL = 0;               /* any write: counter = 0 */
    SysTick->CTRL = SysTick_CTRL_ENABLE | SysTick_CTRL_TICKINT | SysTick_CTRL_CLKSOURCE;
}

void SysTick_Handler( void )
```

```
{
    uptime_ms += TICK_MS;           /* no acknowledge needed */
}

uint32_t systick_uptime_ms( void )
{
    return uptime_ms;               /* one 32-bit read: never torn */
}
```

`SysTick_Handler` overrides the weak default in slot 15; `uptime_ms` is `volatile` because the handler changes it, and its single aligned 32-bit read can never be **torn**.

5.4 Wraparound-Safe Elapsed Time

The kernel measures time with a `uint32_t` counter of milliseconds (`uptime_ms`). A 32-bit variable can count only up to $2^{32}-1$ (`0xFFFFFFFF`); one millisecond later it **wraps around** to 0. This happens after 2^{32} ms ≈ **49.7 days**, a realistic uptime for an embedded device, so code that measures elapsed time must stay correct across the wrap.

> **ANALOGY: A 12-HOUR CLOCK**
>
> From 11 o'clock to 2 o'clock, 3 hours pass, even though 2 is smaller than 11. Counting on the clock face, $(2-11) \bmod 12 = 3$. Unsigned arithmetic in C works the same way, on a "clock" of 2^{32} positions: subtraction is computed **modulo** 2^{32}, so `now - start` is always the true distance from `start` to `now` , even if `now` has wrapped past 0.

Two ways to test whether `delay` ms have passed since `start` :

Test	Across the wrap	Why
`now - start >= delay`	**correct**	the subtraction gives the real elapsed time, modulo 2^32
`now >= start + delay`	**wrong**	`start + delay` can itself wrap to a small number, so the comparison becomes true too early

WORKED EXAMPLE: `START = 0XFFFFFF00`, `DELAY = 300` MS

`start` is 256 ms before the wrap (`0xFFFFFFFF` + 1 = 0).

Moment	`now`	`now - start` (correct test)	`start + delay` (wrong test)
16 ms after start	`0xFFFFFF10`	16 → 16 < 300: **not yet** ✔	wraps to 44; `0xFFFFFF10` ≥ 44 → **elapsed ✘** (284 ms too early)
356 ms after start (after the wrap)	`0x00000064` (100)	256 + 100 = 356 → 356 ≥ 300: **elapsed ✔**	—

The subtraction always measures the real distance; the addition breaks as soon as `start + delay` passes `0xFFFFFFFF`.

5.5 Idle Loop with `wfi`

When there is nothing to do, the CPU should sleep instead of spinning. The `wfi` instruction puts it to **sleep until the next interrupt.**

```
uint32_t last = systick_uptime_ms();
for ( ;; )
{
    __asm volatile ( "wfi" );           /* sleep until an interrupt */
    uint32_t now = systick_uptime_ms();
    if ( now - last >= 1000 )           /* safe across wraparound */
    {
        last += 1000;
        kprintf( "uptime: %u ms\n", now );
    }
}
```

Element	Effect
`wfi`	sleeps until an exception is pending; the loop runs once per interrupt, at least every 10 ms
`last += 1000`	schedules each print 1 s after the previous one: no drift (unlike `last = now`)

5.6 Inspecting the Exception Frame in gdb

The stacked frame described in the exception model of the Cortex-M3 chapter can be observed directly with gdb, by stopping at the first instruction of `SysTick_Handler`.

```
(gdb) break *SysTick_Handler        # stop at the very first instruction
(gdb) continue
(gdb) x/8wx $sp                     # r0 r1 r2 r3 / r12 lr pc xPSR
```

```
0x203fffd0:  0x00000000  0x00000000  0x00000007  0x00000000
0x203fffe0:  0x00000000  0x0000050d  0x00000510  0x01000000
(gdb) p/x $lr
$1 = 0xfffffff9
(gdb) info symbol 0x510
main + 28 in section .text
```

Observation	Meaning
`break *SysTick_Handler`	stops before the prologue, while `sp` points at the frame
Stacked `PC` = `0x510`	instruction after `wfi` in `main`
Stacked `r2` = 7	value just written to `CTRL`
`lr` = `0xFFFFFFF9`	return to Thread mode, MSP

6 The NVIC and Interrupt-Driven UART Transmission

Device interrupts are managed by the **NVIC** (Nested Vectored Interrupt Controller). Using it, the UART can transmit in the background: the program puts bytes in a buffer, and an interrupt handler sends them one by one.

6.1 IRQ Lines and Handlers on the MPS2-AN385

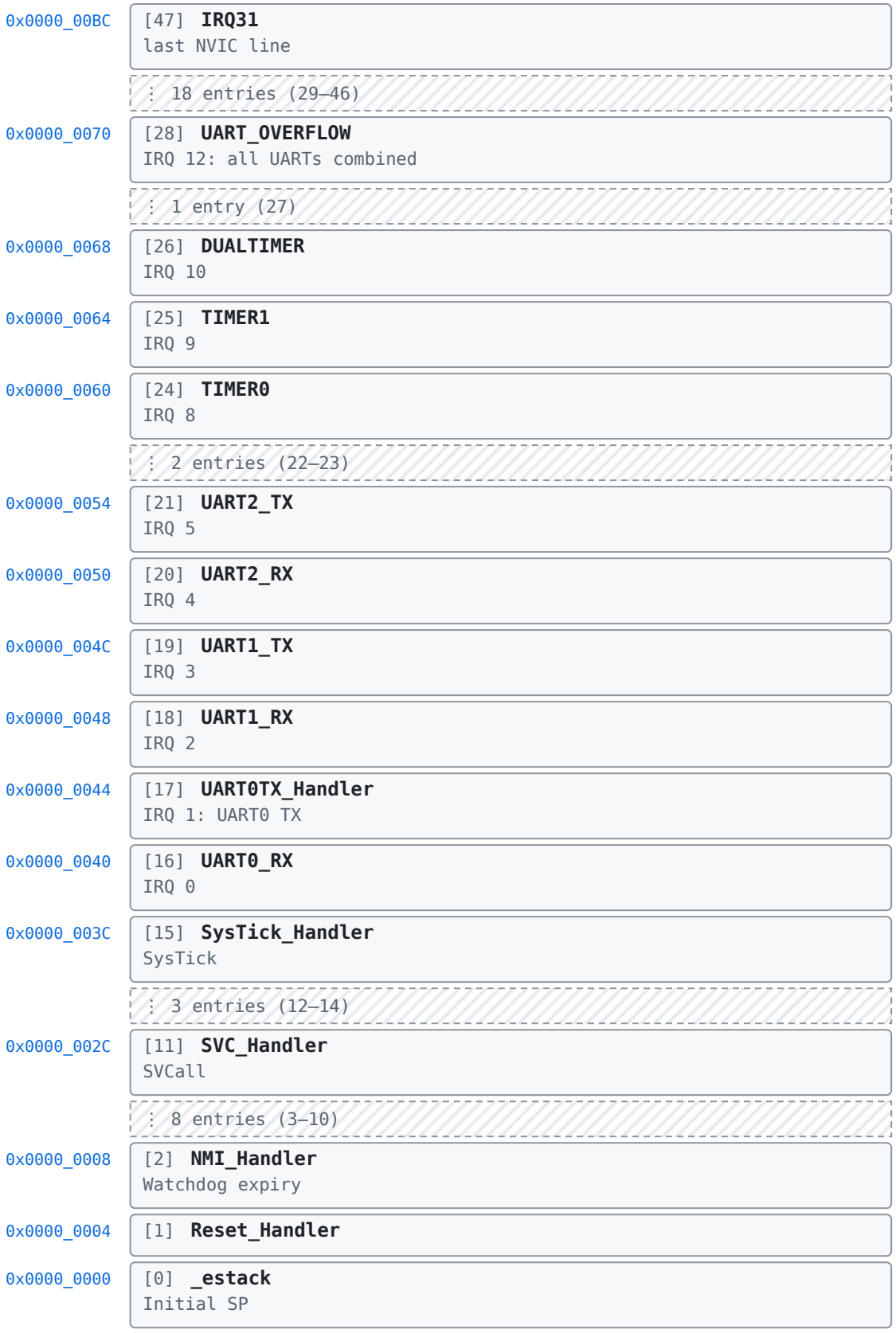
Device interrupts reach the CPU through the NVIC, whose registers are described in the Cortex-M3 chapter. The `mps2-an385` board connects 32 interrupt request lines, **IRQ 0–31**, to it. Every IRQ is also an exception with its own vector table slot: IRQ n is exception 16 + n, and its handler address is stored at offset 4 × (16 + n).

UART0 TX

IRQ 1 → exception 16 + 1 = 17 → slot address 4 × 17 = 68 = `0x44`, which holds the address of `UART0TX_Handler`.

IRQ	Source	IRQ	Source
0	UART0 RX	8	TIMER0
1	UART0 TX	9	TIMER1
2 / 3	UART1 RX / TX	10	dual timer
4 / 5	UART2 RX / TX	12	UART overflow (all UARTs; never fires in QEMU)

Figure 4.4 — Vector table with device interrupts (48 words, 0xC0 bytes)



The vector table grows from 16 to 48 words. As for the core exceptions, every device handler name is a `.weak` alias of `Default_Handler`, and a C function with the same name (for example `UART0TX_Handler`) replaces it at link time without editing the table.

The board's watchdog is connected to the **NMI** (exception 2) instead of an ordinary IRQ line. A watchdog is a timer that the software must reload periodically; if the program hangs and stops reloading it, the watchdog fires so that the system can recover. It uses the NMI because a hung program may have masked all ordinary interrupts: `PRIMASK` does not block the NMI, and its fixed priority is above every configurable one.

6.2 Enabling, Acknowledging, and Prioritising the UART Interrupt

```
NVIC->IPR[UART0TX_IRQn] = 0xC0;                              /* priority of IRQ 1 */
NVIC->ISER[UART0TX_IRQn / 32] = 1UL << ( UART0TX_IRQn % 32 ); /* enable IRQ 1 */
```

The first line writes the priority byte of IRQ 1; the second sets its bit in the enable register. Set and clear registers ignore 0 bits, so a plain assignment is enough and no `|=` is needed. The UART TX handler then runs only if three conditions hold at the same time:

1. the **device** may raise the interrupt (UART TX: `CTRL.TXIRQEN`, bit 2);
2. the **NVIC** line is enabled in `ISER`;
3. its priority is more urgent than that of the running code, and `PRIMASK` = 0.

NVIC lines are **level-sensitive**: a handler that does not clear the device flag is re-entered forever. The UART handler therefore starts by **acknowledging at the device**, with the write-1-to-clear store `CMSDK_UART0->INTSTATUS = UART_INT_TX;`.

Priorities decide which exception runs first and which can interrupt another. A lower value means more urgent, and only a **strictly more urgent** exception preempts the running one; the others stay **pending**. The kernel uses the following policy:

Exception	Priority
Faults	`0x00` or more urgent
SVCall	`0x80`
UART0 TX (IRQ 1)	`0xC0`
SysTick	`0xE0`

6.3 Races on Shared Data and Critical Sections

When `main` and a handler share data, an interrupt arriving in the middle of an update can corrupt it. A critical section prevents this by temporarily masking interrupts.

`volatile` forces real memory accesses but does not make an update indivisible:

```
/* count++ */
    ldr  r1, [r0]       /* r1 = count            (say 5)                    */
    adds r1, r1, #1     /* r1 = 6   <- the handler runs here, sets count = 6 */
    str  r1, [r0]       /* count = 6: the handler's update is lost          */
```

Operation on shared data	Safe?
Single aligned 32-bit read or write	yes
Read-modify-write (`++`, `+=`, `&=`, OR-assignment)	no: **lost update** (a **race**)
Update of several related variables	no

A **critical section** masks interrupts through `PRIMASK`, which C cannot name; two assembly functions provide nestable masking:

```
    .syntax unified
    .cpu cortex-m3
    .thumb
    .section .text,"ax",%progbits

/* uint32_t irq_save(void) */
    .global irq_save
    .type irq_save, %function
    .thumb_func
irq_save:
    mrs r0, primask             /* old = PRIMASK; r0 = return value */
    cpsid i                     /* PRIMASK = 1: interrupts masked */
    bx lr                       /* return old (in r0) */
    .size irq_save, . - irq_save

/* void irq_restore(uint32_t old) */
    .global irq_restore
    .type irq_restore, %function
    .thumb_func
irq_restore:
    msr primask, r0             /* PRIMASK = old (first argument, r0) */
    bx lr                       /* return */
    .size irq_restore, . - irq_restore
```

```
uint32_t a = irq_save();     /* PRIMASK 0 -> 1, a = 0 */
uint32_t b = irq_save();     /* PRIMASK already 1, b = 1 */
irq_restore( b );            /* PRIMASK stays 1: a plain cpsie here would unmask too early */
irq_restore( a );            /* PRIMASK back to 0 */
```

Rule	Reason
Restore the saved value, never `cpsie i`	sections **nest** correctly
Only the main code needs it	a handler cannot be interrupted by the main code
Short, never waiting	interrupts stay pending until `irq_restore`

6.4 Ring Buffer

A ring buffer is the data structure that connects the program (which produces bytes) and the interrupt handler (which consumes them).

A **ring buffer** is an array used as a FIFO: producer at `head`, consumer at `tail`, both wrapping.

“Pasted image 20261018202856.png” could not be found.

Operation	Expression
Next index	`( i + 1 ) & ( SIZE - 1 )`, equal to `% SIZE` for a power of two, with no division
Empty	`head == tail`
Full	`( ( head + 1 ) & ( SIZE - 1 ) ) == tail`

Each index has one writer. One slot stays unused, so 64 entries hold **63** bytes: otherwise full and empty would both satisfy `head == tail`.

6.5 Interrupt-Driven Transmitter

The interrupt-driven UART driver combines the previous pieces: the ring buffer, the critical section, and the TX interrupt handler.

```
#define TX_BUF_SIZE 64                  /* a power of two */

static char tx_buf[TX_BUF_SIZE];        /* bytes waiting to be sent */
static volatile unsigned tx_head;       /* next free slot: written by uart_write */
static volatile unsigned tx_tail;       /* oldest byte: written by the handler (and priming) */
static volatile int tx_busy;            /* 1 while the UART sends and its IRQ will come */

static int tx_empty( void ) { return tx_head == tx_tail; }
static int tx_full( void )  { return ( ( tx_head + 1 ) & ( TX_BUF_SIZE - 1 ) ) == tx_tail; }

static char tx_pop( void )              /* take the oldest byte; buffer not empty */
{
    char c = tx_buf[tx_tail];
    tx_tail = ( tx_tail + 1 ) & ( TX_BUF_SIZE - 1 );
    return c;
}

void uart_init( void )
{
    CMSDK_UART0->BAUDDIV = 16;
    CMSDK_UART0->CTRL = UART_CTRL_TXEN | UART_CTRL_RXEN;
```

```
    tx_head = 0;
    tx_tail = 0;
    tx_busy = 0;
    NVIC->IPR[UART0TX_IRQn] = 0xC0;                              /* below SVCall, above
SysTick */
    NVIC->ISER[UART0TX_IRQn / 32] = 1UL << ( UART0TX_IRQn % 32 ); /* enable IRQ 1 */
}

unsigned uart_write( const char *buf, unsigned len )
{
    unsigned accepted = 0;
    uint32_t saved = irq_save();               /* the handler must not run during the updates
*/

    while ( accepted < len && !tx_full() )      /* copy what fits */
    {
        tx_buf[tx_head] = buf[accepted];
        tx_head = ( tx_head + 1 ) & ( TX_BUF_SIZE - 1 );
        accepted++;
    }

    if ( !tx_busy && !tx_empty() )              /* idle: no IRQ will come, so prime it */
    {
        CMSDK_UART0->CTRL |= UART_CTRL_TXIRQEN; /* FIRST enable the TX interrupt ... */
        while ( CMSDK_UART0->STATE & UART_STATE_TXBF )
        {
        }                                       /* (a polled byte may still be leaving) */
        CMSDK_UART0->DATA = (uint32_t) (unsigned char) tx_pop(); /* ... THEN write */
        tx_busy = 1;
    }

    irq_restore( saved );
    return accepted;
}

void UART0TX_Handler( void )
{
    CMSDK_UART0->INTSTATUS = UART_INT_TX;       /* acknowledge at the device (W1C) */

    if ( !tx_empty() )
    {
        CMSDK_UART0->DATA = (uint32_t) (unsigned char) tx_pop(); /* send the next byte */
    }
```

```
    else
    {
        CMSDK_UART0->CTRL &= ~UART_CTRL_TXIRQEN;  /* nothing left: interrupt off */
        tx_busy = 0;                              /* transmitter idle */
    }
}
```

Component	Role
`uart_write` (**producer**)	critical section; copies what fits and **returns the count accepted**, never waits; **primes** an idle transmitter with the first byte
`UART0TX_Handler` (**consumer**)	acknowledges, sends the next byte or disables `TXIRQEN`; no critical section needed

PRIMING ORDER

The TX interrupt fires when a byte has been sent **and** `TXIRQEN` was set when that byte was written. Enabling it while idle raises nothing; enabling it after the write misses that byte. Correct order: first `TXIRQEN`, then the first byte.

160 BYTES THROUGH THE 64-BYTE BUFFER

`uart_write` is called 3 times (63, 63, 34 accepted); `UART0TX_Handler` runs **160 times**, the last one disabling `TXIRQEN`.

7 System Calls and the Kernel Boundary

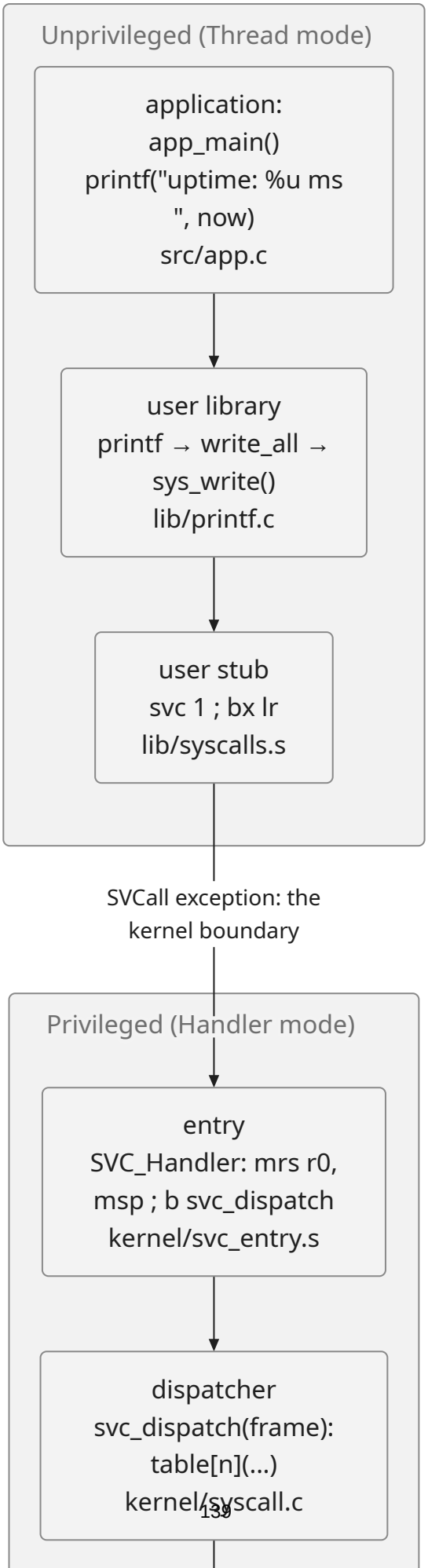
Application code can be made unprivileged, so that it cannot reconfigure the machine. It then needs a controlled way to ask the kernel for services: the system call, implemented with the `svc` instruction.

7.1 Privilege Model and Layered Structure

The Cortex-M distinguishes **privileged** and **unprivileged** execution. The system is organized in layers, with the application on the unprivileged side and the kernel and drivers on the privileged side.

Mechanism	Behaviour
`CONTROL.nPRIV` = 1 in Thread mode	execution becomes unprivileged; one-way (unprivileged code cannot clear the bit)
Unprivileged access to the System Control Space (SysTick, NVIC, SCB)	**BusFault**
`svc #n`	allowed; raises **SVCall**, whose handler runs in Handler mode, always privileged
Return from SVCall	resumes after `svc`, still unprivileged

Figure 4.5 — Layers on the two sides of the kernel boundary



Without an MPU the UART stays reachable from unprivileged code, so "all I/O through system calls" is a **design choice**, with one hard reason: `cpsid i` **is silently ignored unprivileged**, so `uart_write` is correct only when privileged.

The mechanism is the same as in a desktop operating system:

Aspect	Linux on a desktop	Bare-metal Cortex-M3
Restricted code	user-mode process	application in unprivileged Thread mode
Trap instruction	`syscall` (x86-64), `svc` (ARM)	`svc #n`
Privileged side	the kernel	`SVC_Handler` and the dispatcher
Service	`write()` puts bytes on the terminal	`sys_write()` sends bytes to the UART driver

The restricted side cannot do the job itself, so it traps into code that can, and that code decides what to do. This is the core of every operating system's protection model.

7.2 The System-Call ABI

Caller and kernel must agree on how a request is expressed at the machine level: which service is requested, where the arguments are, and where the result goes.

The **ABI** (Application Binary Interface): service number in the immediate of `svc #n`, arguments in `r0`–`r2`, result in `r0`.

Number	Service	Arguments	Result
1	`sys_write`	`buf`, `len`	bytes accepted, 0 to `len` (`uart_write`)
2	`sys_read`	none	one received byte, or `-1` (`uart_getc_polled`)
3	`sys_uptime`	none	milliseconds since start (`systick_uptime_ms`)
0, ≥ 4	none	none	`-1`

No service ever waits: the caller handles partial results.

7.3 Encoding of `svc #n` and Decoding from the Stacked Frame

The service number is not passed in a register but encoded inside the `svc` instruction itself; the kernel reads it, together with the arguments, from the stacked frame.

```
$ arm-none-eabi-objdump -d build/scratch-os.axf          one instruction per line
000008e0 <sys_write>:
 8e0:   df01    svc  1
 8e2:   4770    bx   lr

$ arm-none-eabi-objdump -s -j .text build/scratch-os.axf  raw bytes, memory order
 08e0 01df7047 02df7047 03df7047
```

svc_1 16-bit **= 0xDF01**

Thumb encoding of svc #1 (one halfword)



Bits	Field	Value	Description
15:8	`OPCODE`	0xDF (223)	0xDF identifies svc
7:0	`IMM8`	0x1 (1)	Service number n

Memory is **little-endian**: `objdump -d` shows the halfword `df01`, `objdump -s` the bytes `01 df`. With `uint32_t *frame` pointing at the stacked frame (see the exception model in the Cortex-M3 chapter; `frame[6]` = `pc`):

Item	Location	Reason
Service number `n`	`( ( uint8_t * ) frame[6] )[-2]`	the stacked `pc` points **after** the 2-byte `svc`
Arguments	`frame[0]` – `frame[2]`	stacked by the hardware: no copy needed
Result	**`frame[0]`**	the exception return pops it into the caller's `r0`

7.4 Assembly Entry Point `SVC_Handler`

To read the stacked frame, the kernel must know where it is at the very first instruction of the handler; this requires a short entry point in assembly.

The kernel needs **MSP** at entry to find the frame, but a C prologue moves `sp` first:

```
0000027a <svc_dispatch>:
 27a:  b580   push  {r7, lr}     @ sp = sp - 8
 27c:  b084   sub   sp, #16      @ sp = sp - 16
 27e:  af00   add   r7, sp, #0
```

`sp` has moved by a compiler-dependent 24 bytes. A handler that must find the frame needs an assembly entry:

```
    .global SVC_Handler         /* replaces the weak alias in the vector table */
    .type SVC_Handler, %function
    .thumb_func
SVC_Handler:
    mrs r0, msp                 /* r0 = MSP: frame = address of the 8 saved registers */
    b svc_dispatch              /* svc_dispatch(frame); lr still = EXC_RETURN */
    .size SVC_Handler, . - SVC_Handler
```

Instruction	Purpose
`mrs r0, msp`	frame address in `r0`, the first AAPCS argument of `svc_dispatch(uint32_t *frame)`

Instruction	Purpose
`b` (not `bl`)	**tail branch**: `bl` would overwrite `lr` and lose `EXC_RETURN`; with `b`, the normal return of `svc_dispatch` through `lr` performs the exception return directly

7.5 Dispatch Table and User-Side Stubs

On the kernel side, a table maps each service number to its implementation; on the application side, tiny assembly stubs turn a C call into an `svc`.

```
/* A service: up to 3 arguments (caller's r0-r2), one result (caller's r0). */
typedef uint32_t ( *syscall_fn )( uint32_t a0, uint32_t a1, uint32_t a2 );

static uint32_t sys_write_impl( uint32_t a0, uint32_t a1, uint32_t a2 )
{
    (void) a2;
    return uart_write( ( const char * ) a0, a1 );   /* bytes accepted, 0..len */
}

static uint32_t sys_read_impl( uint32_t a0, uint32_t a1, uint32_t a2 )
{
    (void) a0; (void) a1; (void) a2;
    return ( uint32_t ) uart_getc_polled();         /* byte, or -1 */
}

static uint32_t sys_uptime_impl( uint32_t a0, uint32_t a1, uint32_t a2 )
{
    (void) a0; (void) a1; (void) a2;
    return systick_uptime_ms();
}

static const syscall_fn table[] = {             /* entry 0 stays empty (0) */
    [SYS_WRITE]  = sys_write_impl,
    [SYS_READ]   = sys_read_impl,
    [SYS_UPTIME] = sys_uptime_impl,
};

#define TABLE_SIZE ( sizeof( table ) / sizeof( table[0] ) )

void svc_dispatch( uint32_t *frame )
{
    uint8_t n = ( ( uint8_t * ) frame[6] )[-2];  /* low byte of svc #n before the stacked pc
*/
```

```
    if ( n >= TABLE_SIZE || table[n] == 0 )
    {
        frame[0] = ( uint32_t ) -1;             /* unknown service */
        return;
    }
    frame[0] = table[n]( frame[0], frame[1], frame[2] );  /* result into the stacked r0 */
}
```

Element	Meaning
`syscall_fn`	one signature for all services, so they fit one array
Designated initializers	each entry at its number; entry 0 stays empty; `TABLE_SIZE` follows automatically
Bounds and null check	unknown service answers `-1`

```
/* int sys_write(const char *buf, unsigned len): buf in r0, len in r1 */
    .global sys_write
    .type sys_write, %function
    .thumb_func
sys_write:
    svc #1                      /* svc(SYS_WRITE, buf, len) */
    bx lr                       /* return r0 (set by the kernel) */
    .size sys_write, . - sys_write
```

`sys_read` (`svc #2`) and `sys_uptime` (`svc #3`) are identical. The number is part of the encoding, so it is a literal kept in sync by hand with the C header. In gdb, `x/2bx ((unsigned *)$sp)[6] - 2` at `SVC_Handler` shows `0x03 0xdf` (`svc #3`), and `finish` from `svc_dispatch` lands on the stub's `bx lr` : the return was the exception return.

7.6 User-Space `printf` , `write_all` , and `sleep_ms`

Unprivileged code cannot use `kprintf` , so the application gets its own `printf` , built only on system calls.

Aspect	`kprintf` (kernel)	`printf` (user)
Each character goes to	`uart_putc_polled`	a 64-byte buffer on the stack
When the buffer is full, or at the end	—	`write_all(buf, used)`
Path to the UART	direct register access (privileged)	`sys_write` → `svc #1`

```
#define PRINTF_BUF_SIZE 64      /* characters per sys_write */

void write_all( const char *buf, unsigned len )
{
    while ( len > 0 )
```

```
    {
        int n = sys_write( buf, len );  /* the kernel takes what fits */
        buf += n;
        len -= n;
        if ( n == 0 )
        {
            __asm volatile ( "wfi" );   /* full: sleep until an interrupt */
        }
    }
}

struct printf_ctx
{
    char buf[PRINTF_BUF_SIZE];  /* characters not sent yet */
    unsigned used;              /* how many of buf are in use */
    int total;                  /* characters produced so far */
};

static void printf_out( char c, void *ctx )    /* store one char, flush when full */
{
    struct printf_ctx *p = ctx;
    p->buf[p->used] = c;
    p->used++;
    p->total++;
    if ( p->used == PRINTF_BUF_SIZE )
    {
        write_all( p->buf, p->used );
        p->used = 0;
    }
}

int printf( const char *fmt, ... )
{
    struct printf_ctx p;                    /* on the stack, about 72 bytes */
    p.used = 0;
    p.total = 0;
    va_list ap;
    va_start( ap, fmt );
    vformat( printf_out, &p, fmt, ap );
    va_end( ap );
    write_all( p.buf, p.used );             /* send what is left */
    return p.total;
}
```

```c
void sleep_ms( uint32_t ms )
{
    uint32_t start = sys_uptime();
    while ( sys_uptime() - start < ms )     /* safe across wraparound */
    {
        __asm volatile ( "wfi" );
    }
}
```

Element	Design point
64-byte buffer	one system call per 64 characters
`write_all`	handles **partial writes**; on 0 accepted, `wfi` (allowed unprivileged) until the TX interrupt frees room
`sleep_ms`	the **waiting happens in Thread mode**, where interrupts run

> ⚠ **HIDDEN `MEMSET` IN A FREESTANDING BUILD**
>
> `struct printf_ctx p = { .used = 0, .total = 0 };` fails to link (`undefined reference to 'memset'`): GCC zero-fills the array by calling `memset` , absent without a C library.

7.7 Kernel Start and Unprivileged Application

`main` initializes the system while still privileged, then drops privilege and hands control to the application, which from then on uses only system calls.

```c
int main( void )
{
    uart_init();                                /* 1. console first */
    kprintf( "scratch-os: kernel started\n" );  /* 2. polled print */
    systick_init();                             /* 3. 10 ms tick */
    kprintf( "dropping to unprivileged mode\n" );
    __asm volatile (            /* 4. CONTROL = CONTROL | 1; (nPRIV = 1) */
        "mrs r0, control    \n" /* r0 = CONTROL */
        "orr r0, r0, #1     \n" /* r0 |= 1: set nPRIV */
        "msr control, r0    \n" /* CONTROL = r0: now unprivileged */
        "isb                \n" /* refetch under the new rules */
        : : : "r0", "memory" );
    app_main();                                 /* 5. never returns */
    for ( ;; ) { }                              /* not reached */
}

void app_main( void )
```

```
{
    printf( "app: hello from unprivileged code\n" );
    uint32_t last = 0;                          /* when the uptime was last printed */

    for ( ;; )
    {
        uint32_t now = sys_uptime();
        if ( now - last >= 1000 )               /* one second passed? */
        {
            printf( "uptime: %u ms\n", now );
            last += 1000;
        }
        int c = sys_read();                     /* never waits: -1 = no key */
        if ( c != -1 )
        {
            printf( "key: %c\n", c );
        }
        sleep_ms( 10 );
    }
}
```

Element	Reason
Order: console, tick, drop	printing first; the tick serves `sys_uptime` and `sleep_ms`
`isb`	discards instructions fetched under privileged rules
Clock comparison	counting iterations would drift

```
scratch-os: kernel started
dropping to unprivileged mode
app: hello from unprivileged code
uptime: 1000 ms
key: a
key: b
uptime: 2000 ms
```

The first two lines come from polled `kprintf`, the rest through `printf` → `sys_write` → `svc #1` → `uart_write` → `UART0TX_Handler`. In gdb, `$control` is `0x1` even inside `sys_write_impl` (privileged only because in Handler mode), and `$xpsr & 0x1ff` = 11 (SVCall).

7.8 Kernel Rules: No `svc` Inside the Kernel, Never Wait

Two rules keep the kernel correct; breaking either one stops the system, as the following failures show.

No system calls in kernel code. An `svc` is **synchronous** and cannot stay pending: if not more urgent than the

running code, it **escalates** to HardFault (HFSR bit 30, **FORCED**, e.g. `HFSR=0x40000000`).

Where `svc` executes	Current priority	Result
`app_main`, `main` (Thread mode)	none (lowest)	taken
`SysTick_Handler` / `UART0TX_Handler`	`0xE0` / `0xC0`	taken: SVCall is more urgent and nests
`SVC_Handler` and anything it calls	`0x80`	HardFault
A fault handler	`0x00` or more urgent	HardFault
Between `irq_save` and `irq_restore`	masked (`PRIMASK` = 1)	HardFault

The kernel never waits. If `sys_write_impl` loops until all bytes are accepted, a 130-byte write prints one byte and freezes: only `UART0TX_Handler` (`0xC0`) can drain the buffer, but the CPU is in SVCall (`0x80`), more urgent, so TX and SysTick (`0xE0`) stay pending forever. Hence `sys_write` returns the accepted count and `write_all` waits in **Thread mode**.

> **TWO KERNEL INVARIANTS**
>
> Never block inside a system call (partial results, waiting in user space). Never issue a system call from the kernel (output via polled `kprintf`).

1 Why an Operating System Manages Memory

Memory is a shared resource: the kernel and every task or process it runs all need it at the same time. With respect to memory, an operating system has four jobs.

- **Placement**: decide where the code, data, and stack of each program go, so that they do not overlap. Two programs that both use address `0x20000004` for their own data would destroy each other's values.
- **Sharing**: let programs use the same memory when they should, for example one copy of library code for every program, or a buffer handed to a device.
- **Protection**: stop a program from reading or writing memory that is not its own, whether by mistake (a bad pointer, a stack that grows too far) or on purpose.
- **More programs than memory**: when the programs together need more memory than there is, keep in RAM only the parts in use and store the rest elsewhere.

A flat firmware image, like the kernel built in the previous chapters, does only the first job, and only partly. The linker places every variable, but it places them file after file, with no border between the kernel's data and the data of each task.

```
20000004 B uptime_ms      kernel: millisecond counter
20000008 b tx_buf         kernel: UART transmit buffer
20000054 b a_lines        task A: printed lines
20000058 b b_seconds      task B: reported seconds
```

The output of `arm-none-eabi-nm -n` shows four variables packed together in `.bss`, just above `0x20000000`, a few bytes apart. Nothing marks `a_lines` as "task A only": if task B computed the address `0x20000054` and wrote there, it would change A's variable, and nobody would notice.

1.1 Static Versus Dynamic Allocation

Placement can be decided at two different times. With **static allocation**, every address and every size is fixed at link time by the linker script and the variable definitions; the map file shows the full layout of memory before the program runs, and no holes can appear because nothing is ever freed. The price is flexibility: a size cannot change at run time, and objects whose size is known only at run time cannot be created.

With **dynamic allocation**, memory is handed out at run time from a **heap**, through calls such as `malloc( n )` and `free( p )`. The layout then depends on the order of the calls, which is usually unpredictable. After many allocations and frees, the free memory is split into small holes between blocks still in use.

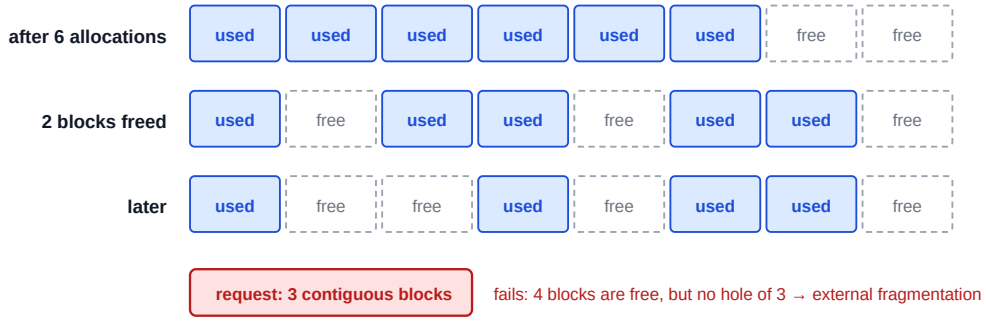


Figure 5.1: *External fragmentation. Four blocks are free, but no three of them are contiguous, so a request for three blocks fails.*

This is **external fragmentation**: enough free space exists, but in the wrong shape. Dynamic allocation has two more problems in an embedded system. It can fail at run time, when the heap is exhausted or too fragmented, and a device in the field usually has nobody to read the error. Its execution time is also not constant, because `malloc` may have to search many blocks, which is a problem for code with deadlines. For these reasons the kernel described here allocates everything statically.

1.2 The Memory-Management Spectrum

Memory-management schemes range from very simple to very capable. Two questions classify each of them: is the address used by the program **translated** into another address before it reaches memory, and does the hardware **protect** memory by stopping forbidden accesses?

Scheme	Translation	Protection	Where
Flat	No	No	Simple firmware: one address space for everybody
Static partitions	No	No	A fixed slice of RAM per task, decided at link time
MPU	No	Yes	Microcontrollers: hardware checks every access against a few regions
Paging (MMU)	Yes	Yes	Linux, Windows, macOS: each process has its own address space
Virtual memory	Yes	Yes	Paging plus a disk: programs can use more memory than the RAM

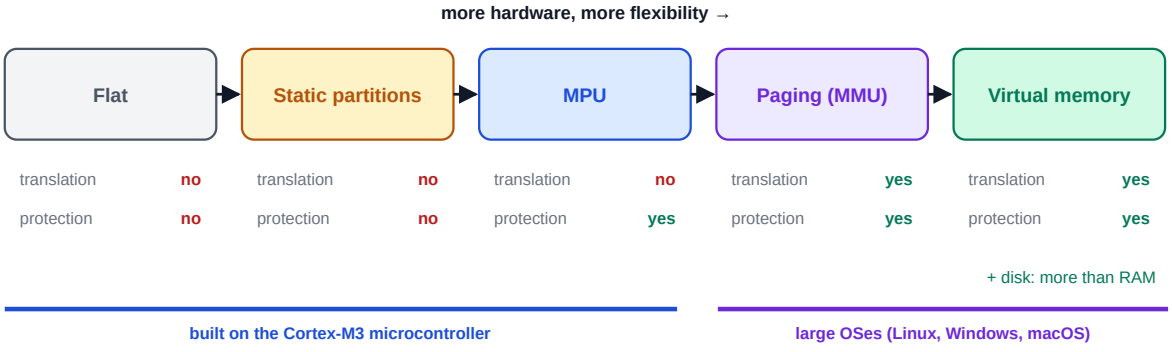


Figure 5.2: *The memory-management spectrum. Each step to the right needs more hardware and gives more flexibility.*

The first three steps can be built on a Cortex-M3 microcontroller; paging and virtual memory need an MMU and are described here as theory, because they explain what a microcontroller does differently.

1.3 Unprotected Memory: A Rogue Task

```
extern volatile uint32_t uptime_ms;     /* the kernel's counter */


printf( "B: writing to the kernel's uptime counter\n" );
uptime_ms = 0x12345678;                 /* a store into kernel memory */
```

Task B declares the kernel's counter with `extern`, so the linker gives it the real address, and then stores a value into it. The store succeeds: one second later B prints an uptime of 305 420 896 ms, because `0x12345678` is 305 419 896 and the kernel keeps counting from there. Task B has changed the kernel's clock for everyone.

Nothing stopped the write even though task B runs unprivileged. Unprivileged mode blocks only some instructions (such as `msr` to `CONTROL`) and access to the System Control Space; it does not check ordinary loads and stores to RAM. On the `mps2-an385` board the memories themselves have no notion of owner either: the bus answers every load and store from any code. A bug in one task can therefore silently break the kernel or another task, and the damage shows up far from the code that caused it. Stopping it requires hardware that checks every access, which is the job of an MMU or an MPU.

2 Paging and Virtual Memory

Large operating systems isolate processes with **paging**. A program uses **virtual addresses** (VA), the addresses in its pointers and instructions; the memory chips see **physical addresses** (PA), the addresses on the bus. Paging maps one to the other.

2.1 Pages, Frames, and the MMU

The virtual address space is divided into equal blocks called **pages**, often 4 KiB, and physical memory into **frames** of the same size. Any page can be placed in any frame. A **page table**, one per **process**, records which frame holds each page. Each entry also has a **valid** bit (is the page in a frame at all?) and permission bits: may the page be read, written, or executed, and may user code access it or only the kernel?

The **MMU** (Memory Management Unit) is the hardware that translates every address and checks its permissions on every load, store, and instruction fetch. This must be done in hardware, because doing it in software for every access would be far too slow. Only the **page number** is translated; the **offset** inside the page is copied unchanged, because a page and its frame have the same size. With 4 KiB pages the offset is the low 12 bits of the address ($2^{12} = 4096$).

2.2 Two-Level Page Tables

With 32-bit addresses and 4 KiB pages, the page number has 20 bits. A single flat table would need 2^{20} entries of 4 bytes, that is 4 MiB per process, even for a tiny program. Real systems therefore split the page number again and use two levels. A register called **PTBR** (page-table base register) holds the address of the current process's **page directory**; each directory entry points to a second-level page table.

```
0x00403007 = 0000 0000 0100 0000 0011 0000 0000 0111
           = 0000000001 | 0000000011 | 000000000111
             dir = 0x001  tbl = 0x003  offset = 0x007
```

The top 10 bits select the directory entry, the next 10 bits the table entry, and the low 12 bits are the offset. To translate `0x00403007`, the MMU reads directory entry 1 at `PTBR + 1 × 4`, which gives the table address, say `0x00021000`. It then reads entry 3 of that table at `0x00021000 + 3 × 4 = 0x0002100C`, which gives the frame, say `0x0009A000`, with the valid bit set. The physical address is the frame plus the offset: `0x0009A007`.

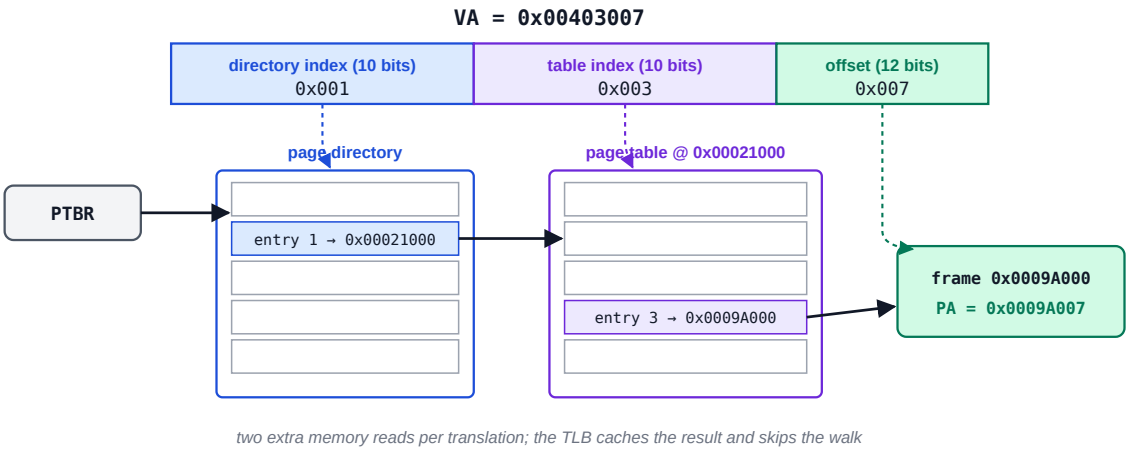


Figure 5.3: *Translation of 0x00403007 with a two-level page table. Each level costs one extra memory read.*

Two levels pay off because a second-level table exists only where memory is used. A small process needs one directory and a few tables instead of a flat 4 MiB table made almost entirely of invalid entries.

2.3 The TLB and the Effective Access Time

The walk costs two extra memory reads for every access the program makes, which would make memory three times slower. The **TLB** (Translation Lookaside Buffer) is a small, fast cache inside the MMU that keeps recent translations (page number → frame, with permissions). On a **TLB hit** the frame comes straight from the TLB; on a **miss** the MMU walks the tables and stores the result. When the OS changes a page table or switches to another process, it must **flush** the stale TLB entries, otherwise a process could reach the frames of the previous one.

The cost of paging is measured by the **effective access time** (EAT), the average time of one memory access including translation. With memory access time m, hit ratio h, and a two-level table, a hit costs m and a miss costs $3m$:

$$EAT = h \cdot m + (1 - h) \cdot 3m = m\,(3 - 2h)$$

The slowdown over plain memory is $2(1-h)$: every 1% of misses makes memory 2% slower. With $m = 10$ ns, a hit ratio of 0.80 gives 14 ns (+40%), while 0.99 gives 10.2 ns (+2%). Processors with 64-bit addresses use four table levels, so a miss costs $5m$ and the slowdown doubles to $4(1-h)$.

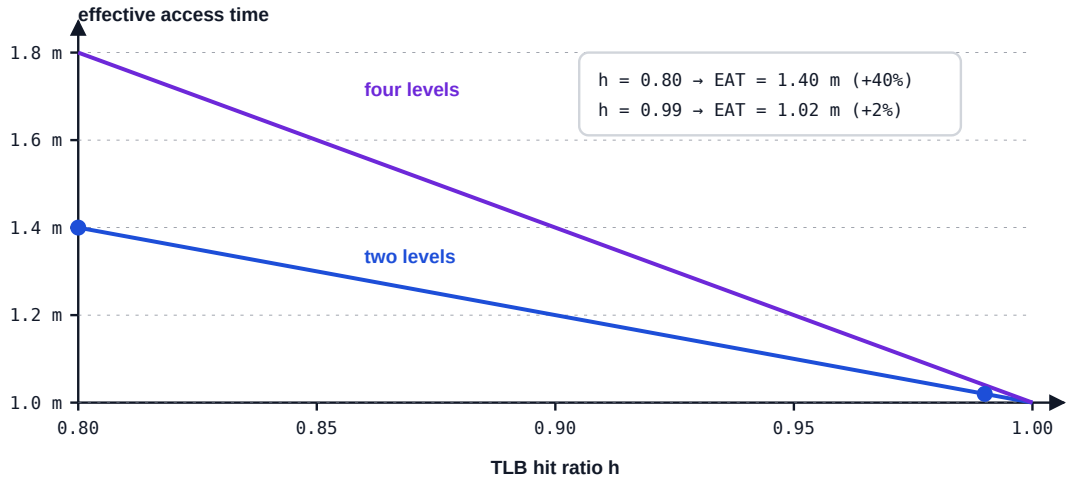


Figure 5.4: *Effective access time as a function of the TLB hit ratio, for two-level and four-level page tables.*

The hit ratio is high in practice because of **locality**: a loop that reads an array misses once on the first word of each 4 KiB page and then hits on the other 1023 words. The **TLB reach** (entries × page size, for example 64 × 4 KiB = 256 KiB) tells how much memory can be accessed without misses. After a flush the TLB is empty, so the same code runs slower right after a context switch. For a real-time task this variation is **jitter**: the time of an access depends on what ran before.

2.4 Page Faults

When the walk finds an entry whose valid bit is clear, or the access breaks the entry's permissions, the MMU raises a **page fault**:

1. the MMU stops the access and raises an exception, saving the faulting address;
2. the OS checks its records of the process's memory: if the address is not part of the program, or the access is not allowed, the program has a bug and the OS terminates it;
3. if the address is legal but the page is not in RAM, the OS finds a free frame, reads the page from disk, fills in the page-table entry, and sets the valid bit;
4. the handler returns and the hardware **restarts** the faulting instruction, which now succeeds.

The instruction is restarted, not skipped, because its load or store never happened. A page fault that reads from disk costs milliseconds, while a memory access costs nanoseconds. With page-fault rate p:

$$EAT = (1 - p) \cdot m + p \cdot t_{fault}$$

With $m = 200$ ns and $t_{fault} = 8$ ms, one fault every thousand accesses makes the machine about forty times slower, and keeping the slowdown under 10% requires fewer than one fault every 400 000 accesses. A single fault also stops the program for milliseconds at an unpredictable moment, which a real-time system cannot accept.

2.5 Virtual Memory

Paging combined with the disk is **virtual memory**. It gives **isolation**: each process has its own page table, so the same virtual address lands in different frames for different processes, and a process cannot even name another process's memory. It also gives **more memory than RAM**: pages not in use wait on disk and are brought back by page faults. Virtual memory therefore handles both protection and the fourth job of the OS, but it needs an MMU, page tables in RAM, and a disk.

A microcontroller has none of these, and it has good reasons not to: an MMU costs silicon area and energy, TLB misses make timing unpredictable, and there is no disk to page to. The Cortex-M3 offers instead an optional **MPU**, which checks addresses against a few regions without translating them. Its regions, permissions, and registers are described in the MPU section of the chapter on the ARM Cortex-M3 architecture. The rest of this chapter builds protection with it, in two steps: first static partitions, which give each task a block of memory with the shape the MPU needs, then the MPU itself.

3 Static Task Partitions

A **partition** is a slice of RAM that belongs to a single task and holds only that task's stack and variables.

3.1 Partition Rules

The partitions follow five rules.

- **They are fixed**: the linker chooses them at link time (static allocation), and no task is created, grown, or moved at run time.
- **Each one is 4 KB and aligned to 4 KB**: the size is a power of two and the start address a multiple of it, such as `0x20000000` or `0x20001000`. This is exactly the shape of an MPU region.
- **They cover RAM only**: code, constants, and string literals stay in FLASH, shared by the kernel and all tasks. Both tasks call the same `printf`, and one copy of its code is enough.
- **The stack is at the bottom**: the stack takes the lowest 1 KB, and the task's variables come right above it.
- **The kernel keeps the rest**: the kernel's `.data` and `.bss` come after the partitions, and the kernel stack stays at the top of RAM.

The position of the stack matters. The stack grows down, so a stack that grows too far leaves the partition below its start instead of overwriting the task's own variables. With the MPU enabled, that first out-of-partition store causes a fault; with the stack at the top, an overflow would silently overwrite the task's variables, and no protection hardware could tell that write from a legitimate one.

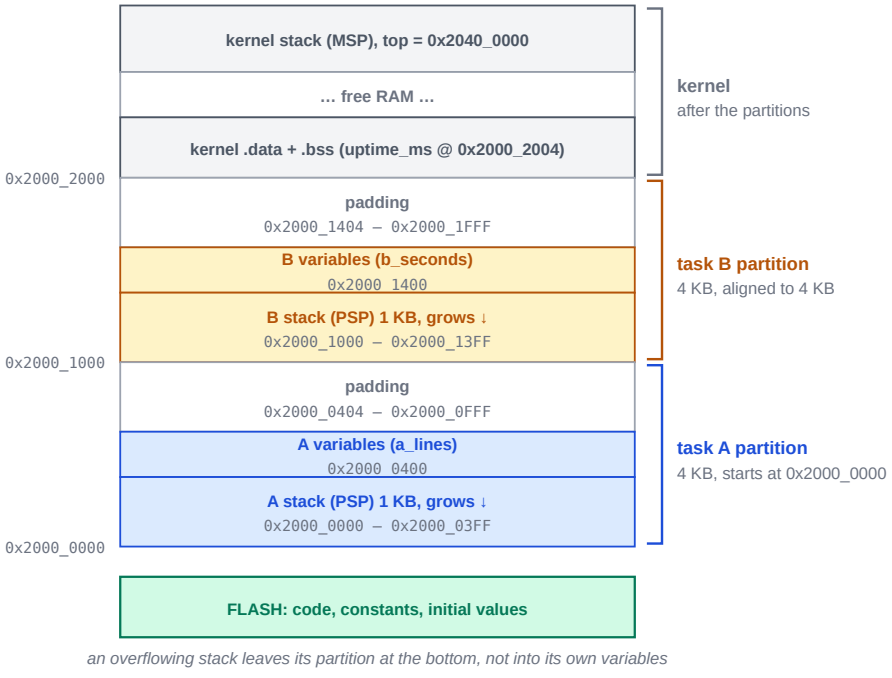


Figure 5.5: *RAM layout with two task partitions. Each 4 KB partition holds a 1 KB stack at the bottom, the task's variables, and padding.*

Power-of-two sizes have a cost: a task that needs 1.1 KB still gets 4 KB. On a microcontroller with a few hundred KiB of RAM and a few tasks, this waste is usually acceptable.

3.2 Linker Script Constructs for Partitions

```
.task_a_data : { ... } > RAM AT > FLASH       /* runs in RAM, initial bytes stored in FLASH */
_task_a_data_load = LOADADDR(.task_a_data);  /* the FLASH address of those bytes           */
.task_a_stack ALIGN(ORIGIN(RAM), 4096) (NOLOAD) : { ... } > RAM
*task_a.o(.data .data.* .bss .bss.* COMMON)  /* these input sections, from task_a.o only */
```

The partitions reuse the linker-script constructs of the boot chapter and add placement by file.

Construct	Meaning
`> RAM AT > FLASH`	The section runs in RAM (VMA) but its initial bytes are stored in FLASH (LMA), as for `.data`
`LOADADDR(section)`	The load address of a section, here the FLASH address of the task's initial values
`(NOLOAD)`	Reserves addresses but stores nothing in the image: right for stacks and padding
`ALIGN(x, 4096)`	`x` rounded up to a multiple of 4096; written before the colon, it sets the section start
`. = start + 4096`	Moves the location counter to the end of the partition
`*task_a.o(...)`	Takes the listed input sections from the object file `task_a.o` only
`ASSERT(cond, "msg")`	Makes the link fail if `cond` is false

A file-name pattern in front of the parentheses selects input sections from one object file only, so every variable written in `src/task_a.c` goes to A's partition without any special marking in the source. Three details make the rules work. First, **the first matching rule wins**: the generic `*(.data*)` and `*(.bss*)` rules of the kernel would also match `task_a.o`, so the partition rules must come before them in the script. Second, the task's `.bss` is placed inside the loaded section together with `.data`: its zeros are then stored in FLASH (a few bytes), and a single copy loop initialises every task variable. Third, the first partition must use `ALIGN(ORIGIN(RAM), 4096)`, because right after `.text` the location counter still holds a FLASH address.

3.3 The Partition Block

```
    /* Task A partition, 4 KB: [ stack 1 KB | A's .data + .bss | padding ] */
    .task_a_stack ALIGN(ORIGIN(RAM), 4096) (NOLOAD) : {
        _task_a_part_start = .;            /* partition start = stack bottom */
        . = . + TASK_STACK_SIZE;           /* reserve the 1 KB stack */
        _task_a_stack_top = .;             /* the stack grows down from here */
    } > RAM
    .task_a_data : {
        . = ALIGN(4);
        _task_a_data_start = .;            /* run address (RAM) */
        *task_a.o(.data .data.* .bss .bss.* COMMON)   /* all of task_a.o's variables */
```

```
        . = ALIGN(4);
        _task_a_data_end = .;
    } > RAM AT > FLASH                      /* runs in RAM, image stored in FLASH */
    _task_a_data_load = LOADADDR(.task_a_data);   /* FLASH address of the image */
    .task_a_pad (NOLOAD) : {
        . = _task_a_part_start + PART_SIZE;   /* jump to the partition end */
        _task_a_part_end = .;
    } > RAM

    /* Task B partition: the same three sections, in the next 4 KB. */
    .task_b_stack ALIGN(4096) (NOLOAD) : { ... } > RAM
    .task_b_data : { ... } > RAM AT > FLASH
    .task_b_pad (NOLOAD) : { ... } > RAM

    /* Link-time guards: each task's variables must fit in its 4 KB. */
    ASSERT(_task_a_data_end <= _task_a_part_end, "task A data does not fit in its partition")
    ASSERT(_task_b_data_end <= _task_b_part_end, "task B data does not fit in its partition")
```

Each partition is made of three output sections, placed before the kernel's `.data` and `.bss`, with `PART_SIZE = 0x1000` and `TASK_STACK_SIZE = 0x400`. `.task_a_stack` starts at the first 4 KB boundary at or above `0x20000000`, marks the start of the partition, reserves 1 KB, and marks the stack top at `0x20000400`; it is `NOLOAD` because a stack has no initial content. `.task_a_data` follows immediately and collects every variable of `task_a.o`; the two `ALIGN(4)` keep both ends on a word boundary, because the copy loop copies whole words. `.task_a_pad` stores nothing and only moves the location counter to the end of the partition, so the next partition starts after A's full 4 KB. Task B repeats the three sections, with `ALIGN(4096)` alone because the location counter is already in RAM. The two `ASSERT` lines catch a task that grows too big at build time rather than at run time.

3.4 Checking the Layout

```
00000dd0 A _task_a_data_load
20000000 B _task_a_part_start
20000400 B _task_a_stack_top
20000400 d a_lines
20001000 B _task_a_part_end
20001000 B _task_b_part_start
20001400 B _task_b_stack_top
20001400 d b_seconds
20002000 B _task_b_part_end
20002004 B uptime_ms
```

The `nm -n` output confirms the layout: task A owns `0x20000000` – `0x20001000` and task B `0x20001000` – `0x20002000`, both 4 KB and 4 KB-aligned, and the kernel's `uptime_ms` has moved after the partitions, to `0x20002004`. `a_lines` and `_task_a_stack_top` have the same address, `0x20000400`, but they do not overlap. The stack top is an exclusive bound: the Cortex-M stack is full descending, so with `sp = 0x20000400` the first

push writes at `0x200003FC`, and the word at `0x20000400` is free for the first variable. The map file shows the same layout from the linker's point of view, with a FLASH load address for each `.task_X_data` section and none for the `NOLOAD` stacks.

3.5 Initialising Partition Variables

```
void partition_init( const struct task *t )
{
    const uint32_t *src = t->data_load;     /* initial values, in FLASH */
    uint32_t *dst = t->data_start;          /* the variables, in RAM */
    while ( dst < t->data_end )
        *dst++ = *src++;                    /* copy one word, move both on */
}
```

The variables of a task run in RAM but start in FLASH, so the kernel must copy them before the task runs for the first time. `partition_init` is the same loop that `Reset_Handler` runs for the kernel's `.data`, executed once per task with the addresses stored in its `struct task`, which the kernel fills from the linker symbols. No separate zeroing loop is needed, because the task's `.bss` is part of the loaded section.

3.6 Partitions Arrange but Do Not Enforce

Running the rogue task again after the change gives exactly the same result as before: task B still corrupts `uptime_ms`, now at `0x20002004`. The linker decides only *where* things are; at run time a store to `0x20002004` is an ordinary store, and nothing compares it with B's partition. What has changed is the shape of memory: each task's memory is now a single aligned 4 KB block, so one hardware rule per task is enough to describe what that task may touch.

4 Starting and Ending Tasks

A task runs in unprivileged Thread mode on its own stack. Starting and ending such a task requires the two stack pointers of the Cortex-M3 and the exception return mechanism, both described in the chapter on the ARM Cortex-M3 architecture.

4.1 Kernel Stack and Task Stacks

Each task's stack is a **PSP** stack inside its own partition, while the kernel and every exception handler use the **MSP** stack at the top of RAM. A running task has `CONTROL = 3`: `nPRIV = 1` (unprivileged) and `SPSEL = 1` (PSP). The stack must lie inside the partition because it holds the task's local variables, return addresses, and the exception frame that the hardware pushes on every interrupt; with the stack inside the partition, the MPU can restrict the task to its own 4 KB. Because Handler mode always uses MSP, the kernel's own work never runs on a task's stack.

4.2 Entering a Task with a Fake Exception Frame

The kernel cannot start a task with a function call. When it starts a task it is running inside the SVC handler, in Handler mode, privileged, and on MSP; a call would keep that mode, privilege, and stack, and the `svc` would never end. The only hardware mechanism that changes mode, privilege, and stack in a single step is the **exception return**. The kernel therefore **fakes an interruption**: it writes on the task's stack the frame that the hardware would have pushed if the task had been interrupted just before its first instruction, points PSP at it, and returns from the `svc` with `EXC_RETURN = 0xFFFFFFFD` (Thread mode, PSP).

Word	Address (task A)	Register	Value written
`frame[0..4]`	`0x200003E0` – `0x200003F3`	`r0` – `r3`, `r12`	0
`frame[5]`	`0x200003F4`	`lr`	`task_exit`
`frame[6]`	`0x200003F8`	`pc`	`t->entry & ~1`, the address of `task_a`
`frame[7]`	`0x200003FC`	`xPSR`	`0x01000000`, Thumb bit only

```
__asm volatile (
    "msr psp, %0        \n" /* PSP = frame */
    "mrs r1, control    \n" /* r1 = CONTROL */
    "orr r1, r1, #1     \n" /* r1 |= 1: set nPRIV */
    "msr control, r1    \n" /* CONTROL = r1 */
    "isb                \n" /* make the new CONTROL take effect now */
    : : "r" ( frame ) : "r1", "memory" );
return EXC_RETURN_THREAD_PSP;              /* 0xFFFFFFFD */
```

`task_launch` first records the current task, sets the MPU region for its partition, and copies its variables with `partition_init`. It then writes the 8-word frame at `t->stack_top - 8` and runs the assembly block, which is needed because C cannot name PSP or `CONTROL`. The returned value travels back to `SVC_Handler`, which ends the exception with it. The hardware pops the 8 words from PSP and continues at the popped `pc`, `task_a`, in

Thread mode, unprivileged, on PSP.

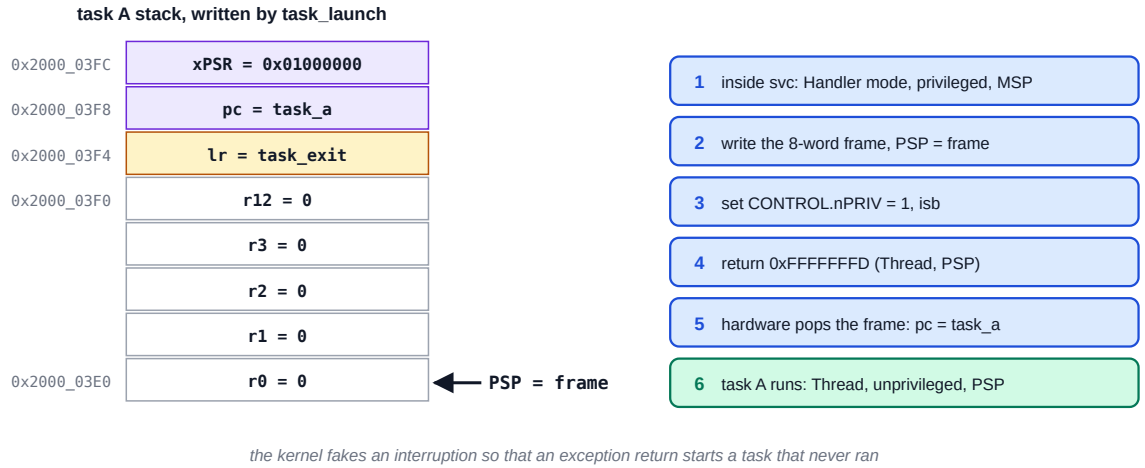


Figure 5.6: *Starting a task. The kernel writes a fake exception frame on the task's stack, and the exception return "resumes" a task that has never run.*

Several details of the frame are essential. Setting `nPRIV` inside the handler has no effect until the return, because Handler mode is always privileged; after the return `CONTROL` is 3, since the exception return with `0xFFFFFFFD` sets `SPSEL`. The `xPSR` word must have bit 24 (the Thumb bit) set, otherwise the first instruction raises a UsageFault (`INVSTATE`). The `pc` word has bit 0 cleared, because it must be a real instruction address rather than a code pointer. Finally, `lr` holds `task_exit` , so a task that simply returns from its function lands in the exit path.

4.3 Choosing the Stack in SVC_Handler

```
    tst lr, #4                  /* flags = lr & 4: is bit 2 set? */
    ite eq                      /* if ((lr & 4) == 0) ... else ... */
    mrseq r0, msp               /*     frame = MSP;  // came from main() */
    mrsne r0, psp               /* else frame = PSP;  // came from a task */
    mov r1, lr                  /* exc_return = lr: second argument */
    bl svc_dispatch             /* r0 = svc_dispatch(frame, exc_return) */
    bx r0                       /* return_from_exception(r0) */
```

With tasks, `svc` can arrive from two places: from a task, on PSP, and once from `main` , on MSP. Bit 2 of `EXC_RETURN` tells which stack holds the stacked frame. `tst lr, #4` sets the Z flag when the bit is 0, and `ite eq` (if-then-else) makes exactly one of the two following `mrs` instructions execute. The entry then calls `svc_dispatch` with the frame and the `EXC_RETURN` value, copying `lr` first because `bl` overwrites it, and ends the exception with the value the dispatcher returns. In pseudo-C: `frame = (lr & 4) ? PSP : MSP; lr = svc_dispatch(frame, lr); return_from_exception(lr);` .

Returning a value is what lets a system call return *somewhere else*. Ordinary services (`SYS_WRITE` , `SYS_READ` , `SYS_UPTIME`) write their result into `frame[0]` and return `exc_return` unchanged, so the exception goes back to the caller. `SYS_START_TASKS` , issued once by `main` with `svc #5` , returns into the first task; `SYS_EXIT` returns into the next task or back to `main` . Each `svc` still ends at once: the kernel never waits inside a handler.

4.4 Ending a Task

```
sys_exit:
    svc #4                      /* svc(SYS_EXIT) */
    b sys_exit                  /* never reached; if it were, ask again */

task_exit:
    bl sys_exit                 /* sys_exit(); it does not come back */
    b task_exit                 /* never reached; if it were, try again */
```

A task ends either by calling `sys_exit()` or by returning from its function, in which case it lands in `task_exit`, the address placed in `frame[5]`. On the kernel side, `SYS_EXIT` from a task calls `task_exit_current`:

```
uint32_t task_exit_current( void )
{
    kprintf( "task %s exited\n", tasks[current].name );
    if ( current + 1 < ( int ) NUM_TASKS )
        return task_launch( current + 1 );      /* start the next task */
    current = -1;                               /* no task runs any more */
    /* asm block: CONTROL = CONTROL & ~1;  clear nPRIV, then isb */
    return EXC_RETURN_THREAD_MSP;               /* 0xFFFFFFF9: back to main() */
}
```

When task A ends, the function launches task B: it moves the MPU region to B's partition and returns B's fake frame with `0xFFFFFFFD`, so the exception returns straight into B. When the last task ends, the kernel clears `nPRIV` and returns `0xFFFFFFF9` (Thread mode, MSP), so `main` continues after its `svc #5`. Clearing `nPRIV` is essential: `main` runs in Thread mode, and with `nPRIV` still at 1 it would run unprivileged and fault on its first access to the UART. It could not fix this by itself, because unprivileged code cannot write `CONTROL`.

5 Protecting Tasks with the MPU

With the partitions in place, two MPU regions are enough to describe what the running task may touch. The region encoding (`RBAR` , `RASR` , `SIZE` , `AP` , `XN`) is described in the MPU section of the Cortex-M3 chapter.

5.1 Two Regions per Task

```
void mpu_init( void )
{
    /* Region 0 = FLASH: base 0, 4 MB = 2^22 B, read-only, code may run. */
    MPU->RBAR = 0x00000000 | MPU_RBAR_VALID | 0;          /* = 0x00000010 */
    MPU->RASR = MPU_RASR_AP( MPU_AP_RO ) | MPU_RASR_C
              | MPU_RASR_SIZE( 22 ) | MPU_RASR_ENABLE;    /* = 0x0602002B */
    /* MPU on. PRIVDEFENA: the kernel keeps the default memory map. */
    MPU->CTRL = MPU_CTRL_ENABLE | MPU_CTRL_PRIVDEFENA;     /* = 5 */
    __asm volatile ( "dsb" ::: "memory" );
    __asm volatile ( "isb" ::: "memory" );
}

void mpu_set_task_region( uint32_t base, uint32_t size )
{
    uint32_t log2_size = __builtin_ctz( size );           /* 4096 -> 12 */
    /* Region 1 = the partition: read-write, never executable (XN). */
    MPU->RBAR = base | MPU_RBAR_VALID | 1;                 /* B: 0x20001011 */
    MPU->RASR = MPU_RASR_XN | MPU_RASR_AP( MPU_AP_FULL_RW )
              | MPU_RASR_S | MPU_RASR_C
              | MPU_RASR_SIZE( log2_size ) | MPU_RASR_ENABLE; /* 0x13060017 */
    __asm volatile ( "dsb" ::: "memory" );
    __asm volatile ( "isb" ::: "memory" );
}
```

`mpu_init` runs once, before the first task starts. It makes **region 0** cover the 4 MB of FLASH as read-only and executable for everyone, and enables the MPU with `PRIVDEFENA` , so that privileged code (the kernel, `main` , and every handler) keeps the default memory map. `mpu_set_task_region` is called by `task_launch` for every task. It points **region 1** at the partition of the task about to run, as read/write and execute-never. `__builtin_ctz` (count trailing zeros) computes $\log_2$ of the power-of-two size, and the `SIZE` macro applies the encoding. The `dsb` and `isb` barriers make sure that no later access is checked against the old regions.

The "MPU state" of a task is therefore a single value, the base of region 1. Everything else (the other task's partition, kernel data, kernel stack, and peripherals) has no region and is forbidden to unprivileged code. A correct program prints exactly the same lines as before: its code and constants are in FLASH, its variables, stack, and the `printf` buffer (a local variable) are in its partition, and every access to the UART happens inside the privileged kernel through system calls.

5.2 MemManage Reports

```
uint32_t cfsr = SCB->CFSR;
kprintf( "MemManage: CFSR=0x%x\n", cfsr );
if ( cfsr & SCB_CFSR_MMARVALID )                /* MMFAR is valid only then */
    kprintf( "  address 0x%x\n", SCB->MMFAR );
if ( cfsr & SCB_CFSR_DACCVIOL )
    kprintf( "  cause: data access\n" );
if ( cfsr & SCB_CFSR_IACCVIOL )
    kprintf( "  cause: instruction fetch\n" );
if ( cfsr & SCB_CFSR_MSTKERR )
    kprintf( "  cause: stacking\n" );
```

`MemManage_Handler` runs privileged and on MSP, so it can always print. It reads `CFSR` once, so that every test sees the same value, prints `MMFAR` only when `MMARVALID` is set, and tests each cause separately, because several bits can be set at the same time. With the MPU enabled, the rogue store never happens: the report reads `CFSR = 0x82` (`MMARVALID` + `DACCVIOL`) with address `0x20002004`, which `arm-none-eabi-nm` identifies as `uptime_ms`. The fault points at the exact instruction and variable involved, instead of letting the damage show up later and elsewhere.

A **stack overflow** is caught in the same way. A recursion that needs about 4 KB of stack in task B, which has only 1 KB, produces `CFSR = 0x92` with address `0x20000FDC`: `0x80 + 0x10 + 0x02`, that is `MMARVALID` + `MSTKERR` + `DACCVIOL`. The address is 36 bytes below B's partition, inside A's partition, which has no region while B runs. The stacking cause appears because the CPU also tried to push the exception frame on the overflowing PSP; the frame is lost, but the handler runs on MSP and its report is still correct. Without the MPU, the same program keeps corrupting memory and ends much later with a misleading `UsageFault` (`INVSTATE`), far from the real cause.

5.3 Why the Kernel Must Check Pointers

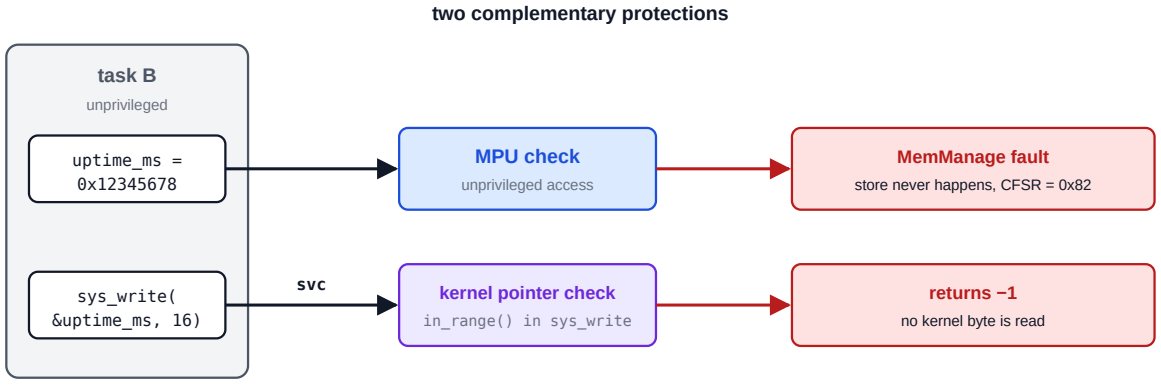
```
int r = sys_write( (const char *) &uptime_ms, 16 );
```

The MPU stops what a task does by itself, but not what a task asks the kernel to do. A task that passes a kernel address to `sys_write` makes the kernel read that memory on its behalf: the handler runs privileged, `PRIVDEFENA` gives it the whole map, and the call prints 16 bytes of kernel data (the counter and the start of the UART buffer). The kernel must therefore check every pointer it receives against what *that task* could access by itself.

```
static int in_range( uint32_t p, uint32_t len, uint32_t start, uint32_t end )
{
    return p >= start && p <= end && len <= end - p;
}
```

```
/* in sys_write_impl: accept the buffer only inside the caller's
 * partition or inside FLASH; the kernel itself (no task) is trusted. */
const struct task *t = task_current();
if ( t != 0
     && !in_range( a0, len, t->part_start, t->part_end )
     && !in_range( a0, len, 0x00000000, 0x00400000 ) )
    return ( uint32_t ) -1;
```

`in_range` checks that `[p, p + len)` lies entirely inside `[start, end)`. The obvious test `p + len <= end` is wrong, because `p + len` can wrap around 2^{32}: with `p = 0x20001F00` and `len = 0xFFFFFF00`, the sum wraps to `0x20001E00` and the check would pass, letting the kernel read almost all of memory. Once `p <= end` is known, `end - p` cannot wrap, so `len <= end - p` is always correct. `sys_write_impl` accepts a buffer only if it lies entirely in the caller's partition or entirely in FLASH, where string constants live; otherwise it returns −1 before reading anything. The task receives an error it can handle, not a fault, and the task's own `printf` still works because it formats its text in a buffer on the task's stack.



the MPU stops what a task does itself; the kernel stops what a task asks the privileged kernel to do

Figure 5.7: *Two complementary protections. The MPU stops direct accesses by unprivileged code; the kernel's pointer checks stop indirect accesses made through system calls.*